Master's Thesis

**MSc degree in Automatic Control and Robotics (MUAR)**

# VLM-based high-level action planner for mobile robots

July 6, 2026


**Author:** Víctor Marín Jiménez

**Director:** Vicenç Puig Cayuela


**Academic year:** 2025-2026

Escola Tècnica Superior
d'Enginyeria Industrial de Barcelona

## Abstract

This thesis explores the use of a visual-language model (VLM) based approach for the high-level action planning of a mobile robot, instantiated in the case of an autonomous vehicle. The approach is integrated in a hierarchical framework that decouples the perception, high-level planning and low-level control into connected modules. These modules communicate with each other using ROS2.

The framework operates as follows: the perception module fuses the vehicle's sensor data, and generates a structured scene context that grounds the VLM model in its surroundings. The decision-making module combines this sensorial data with an image, from the vehicle's frontal camera, into a prompt. Then, it queries the VLM and obtains a discrete driving action. The behavior of the model is specified through natural language and sent to the VLM as the system's prompt. Finally, the low-level control module translates the high-level action into a specific vehicle behavior through CARLA's Traffic Manager, and wraps it in a dedicated layer of safeguards that prevents dangerous maneuvers arising from models hallucinations or inference latency.

The central hypotheses of this thesis are, first, that the proposed architecture is capable of guiding the vehicle taking well-grounded mid-term decisions, while the low-level control layer acts as a safety substrate that keeps navigation viable through the model's long latency periods; and second, that the framework is model agnostic, with the quality of the decisions taken scaling with the capability of the model. Furthermore, the thesis investigates whether the visual input improves the navigation planning by providing real-world grounding, enhancing the model's reasoning.

The framework is validated in CARLA simulator, considering a single navigation environment, Town10HD, which reproduces an inner-city urban setting that challenges the agent's reasoning capabilities. Two VLM models are proposed for the study: Qwen3.5:2B, as an on-device baseline model, and Claude Haiku 4.5 as an upper-bound cloud reference. Five scenarios of increasing complexity are considered: a free run, a run with ambient traffic present, a road obstruction, a pedestrian suddenly crossing, and the approach of an ambulance. The decisions of the VLM-based approach are subsequently evaluated both qualitatively, through visual inspection, and quantitatively through key metrics such as valid-call rate, fallback rate and hallucination instances.

The results obtained showed how both models are capable of successfully guiding the vehicle. Despite the instability shown by the baseline, the control layer and its safeguards kept the resulting driving viable and close to that of the reference, supporting the role of the architecture as a model-agnostic safety substrate. The difference in output quality between the two models (91.5% valid calls for the baseline versus 100.0% for the reference) is absorbed before it reaches the vehicle. The tests are consistent with our underlying hypothesis, decision quality scales with model capability. Yet the outcome divergence remains mild because the control layer compensates for the baseline model, showing that the framework itself bridges the gap between both models. This compensation, however, has a clear boundary, as the safety layer rejects malformed or unstable outputs, but it cannot correct a decision that is well-formed and wrong. This fact was clearly observed in the pedestrian scenario, when the baseline model moved recklessly towards a pedestrian still in the front cone while emitting a valid action that no safeguard could flag. Finally, contribution of visual support could not be isolated under the current design and a dedicated ablation study is required to draw a firm conclusion on this matter.

# Contents

**List of Abbreviations** **11**

**1 Introduction** **13**
1.1 Hypotheses . . . 15
1.2 Objectives . . . 15
1.3 Scope . . . 16
1.4 Contributions . . . 17
1.5 Thesis Outline . . . 17

**2 State-of-the-art** **19**
2.1 High-level Planning: Traditional Methods . . . 19
2.2 Learning Methods . . . 19
2.3 World Models . . . 20
2.4 LLM / VLM Fundamentals . . . 21
2.5 LLMs / VLMs Applied To Autonomous Driving And Robotics . . . 23
2.5.1 Language models as high-level planners . . . 23
2.5.2 Language-based navigation . . . 23
2.6 Closed-loop planning and correction . . . 23
2.7 Hierarchical language-based planning in driving . . . 24
2.8 Research gaps addressed . . . 24

**3 Methodologies** **25**
3.1 System Architecture . . . 25
3.1.1 Perception module . . . 25
3.1.2 Decision-making module . . . 39
3.1.3 Low-level control module . . . 45
3.2 Experimental setup . . . 52
3.2.1 Models compared . . . 52
3.2.2 Reproducibility . . . 52
3.2.3 Scenarios . . . 53
3.2.4 Visualization . . . 53
3.2.5 Captured data . . . 54
3.2.6 Metrics . . . 54
3.3 Platform and environment setup . . . 55
3.3.1 Simulation: CARLA . . . 55
3.3.2 Middleware: ROS2 and the CARLA bridge . . . 55
3.3.3 Software environments: pixi . . . 57
3.3.4 Hardware and software summary . . . 57
3.3.5 Launch configuration file . . . 57

**4 Results** **59**
4.1 Aggregate quantitative comparison . . . 59
4.2 Results per scenario . . . 61
4.2.1 Free run . . . 61
4.2.2 Traffic . . . 64
4.2.3 Obstacle . . . 67
4.2.4 Pedestrian . . . 68

4.2.5 Ambulance . . . 71
4.3 Visual modality . . . 73
4.4 Study limitations . . . 74

**5 Discussion** **77**
5.1 Framework limitations . . . 78

**6 Planning and Scheduling** **79**

**7 Economical and environmental impact** **81**
7.1 Economical study . . . 81
7.2 Environmental study . . . 81

**8 Conclusions** **83**

**9 Future work** **85**

**10 Appendix** **87**
10.1 Source code . . . 87

## List of Figures

1 Comparison of autonomy approaches: (a) Traditional *Behavior Trees* require structured, rigid, and predefined logic [1]. In contrast, Deep learning Models demonstrate high flexibility without prior specific training (zero-shot), both in (b) reasoning and planning of complex tasks via text (LLM), and in (c) semantic understanding and description of dynamic environments from images (VLM). (d) Image of Madrid's M-30, used for the VLM analysis shown in (c). This image is sourced from "EFE" . . . . . 14
2 Abstracted hierarchical architecture framework diagram. . . . . 15
3 Conceptual view of a transformer processing a sequence. Each input token is mapped to an embedding. A self-attention layer lets every token weigh the relevance of the others (line width suggests the attention weight), and a stack of such blocks feeds a final output layer that produces a probability distribution over the next token in the chain. The highest-probability token is returned as the model's output. . . . . 22
4 Complete system architecture. CARLA exposes its sensors and actuators through the `carla-ros-bridge`, the only boundary between the simulated vehicle and the ROS2 side. The perception node (`scene_context_node`) packs the sensor data into a structured scene context, published on `/scene_context`. The decision node (`prompt_manager`) combines that context with the behavior prompt and the front camera image and queries the VLM through an API, publishing the validated action on `/llm/output`. The control node (`action_dispatcher`) filters the action through the safety layer and executes it via the Traffic Manager's pre-configured behaviors, built on a PID controller. The dashed arrow closes the loop, returning actuation commands to the vehicle. . . . . 26
5 Perception pipeline. The `scene_context_node` aggregates the self-data and surroundings-data pseudo-sensors exposed by the carla-ros-bridge, processes them into the relevant magnitudes (rounding kinematics and deriving the braking distance, filtering and discretizing the surrounding objects and traffic lights, and mapping raw values to readable labels), assembles them into an XML scene context, and publishes it at 1 Hz on the `/scene_context` topic. . . . . 27
6 Schematic illustration of the mentioned two ways in which the input context degrades language-model performance, reproduced qualitatively from the findings of the cited works. (a) When the relevant information is placed at different positions within a long context, accuracy is highest at the beginning and the end and drops in the middle [29]. (b) Adding irrelevant context to the prompt lowers accuracy, an effect that is more pronounced for smaller, less capable models [28]. The curves are schematic and reproduce the papers' reported trends, not the exact measurements. . . . . 28
7 ROS2 computation graph of the framework. Ellipses are nodes and rectangles are topics, with arrows indicating publication and subscription. The `carla_ros_bridge` publishes the sensor topics (prefix `/carla/hero/`, abbreviated as `.../`) and the front camera image. The `scene_context_node` subscribes to the sensor topics and publishes `/scene_context`, consumed by both the `prompt_manager` and the `action_dispatcher`. The camera image is delivered directly to the `prompt_manager`, which publishes the validated action on `/llm/output`; the `action_dispatcher` consumes it, publishes the committed action on `/action_committed`, and returns the control commands to the bridge (dashed arrow). . . . . 31

8 Schematic illustration of detection ranges around the ego vehicle. Objects reported by the sensors are expressed relative to the ego and filtered in two stages. Those within DISTANCE_THRESHOLD are kept and ordered by proximity, of which only the MAX_OBJECTS (five) closest are retained (green). Objects within range but beyond the five closest are discarded by the count limit (orange triangles), and objects beyond DISTANCE_THRESHOLD are discarded by distance (grey crosses). The object within EGO_SELF_RADIUS is removed as the ego itself. Traffic lights follow an analogous distance filter with TL_THRESHOLD. Those in the rear sector (shaded area) are additionally discarded, as a light behind the ego cannot govern its forward motion. . . . . 34
9 The eight named directions used in the scene context, shown as $45°$ sectors around the ego vehicle (top view). FRONT is centred on the ego's orientation ($0°$), positive angles lie to the left and negative angles to the right, following the convention of Table 4. . . . . 37
10 Operation of the speed ramp. The planner issues a new target speed at intermittent decision instants (Inference time and latency are not exactly fixed) (orange markers), producing the step commanded target. The 10 Hz ramp moves the actual speed toward that target gradually, at a maximum rate of RAMP_RATE $= 3$ m/s$^2$ and updated every 100 ms, so that the vehicle's motion stays smooth between the infrequent decisions of the planner rather than jumping abruptly at each new command. . . . . 46
11 Architecture of CARLA's Traffic Manager and the parts the framework disables. The Traffic Manager runs an iterative control loop (localization, collision, traffic-light, and motion-planning stages) on top of a PID controller, producing the low-level command array sent to the simulator. The framework keeps its low-level execution (lane keeping, commanded-speed following, discrete maneuvers, and collision detection) but disables its decision autonomy. The traffic-light compliance is left to the VLM, autonomous lane changes are turned off, and the random seed is fixed to zero for reproducibility. Adapted from CARLA's Traffic Manager documentation. . . . . 47
12 Town10HD, the inner-city urban map used for all experiments. It concentrates a variety of road features relevant to the scenarios, including single- and multi-lane roads, intersections and junctions of different types, traffic lights, and a dense urban layout, which makes it possible to exercise the planner on diverse situations within a single, consistent environment. . . . . 56
13 Free-run scenario. Qwen3.5:2b. The model issues a CHANGE_LANE_RIGHT action without an appropriate reason. This figure is an example of how the baseline model issued unwarranted lateral maneuvers. . . . . 61
14 Free-run scenario. Qwen3.5:2b. Imprecise traffic-light stop of the baseline model in the free-run scenario: the vehicle comes to a halt far short of the red light. (a) General view, with the stopping position marked on the road; (b) detail showing that the light is still distant at the moment the vehicle stops. . . . . 63
15 Traffic scenario. Claude Haiku 4.5. Correctly placed traffic-light stop in the traffic scenario: the vehicle halts close to the red light, at an appropriate stopping distance. (c) General view; (d) detail of the light directly ahead. . . . . 63
16 Traffic scenario. Claude Haiku 4.5. Issuing of the TURN_LEFT action. The model correctly identifies that the car is turning but fails to identify the correct orientation. Moreover, it issues the action at a curving road rather than at a junction. . . 65

17 Traffic scenario, Claude Haiku 4.5. Graduated sequential response to a vehicle stopped at a red light: (a) FOLLOW_LANE while the leading vehicle is still far; (b) SLOW_DOWN as the gap closes; (c) STOP when the vehicle is close; (d) SLOW_DOWN once the leading vehicle sets off again. The model modulates speed progressively with the distance to the obstacle. . . . . 65
18 Obstacle scenario. Claude Haiku 4.5. Cautious approach to the stationary vehicle blocking the lane. The model approaches with successive SLOW_DOWN actions at a safe distance from the obstacle (a, b) and comes to a controlled STOP behind it (c), anticipating the blockage rather than reacting to it at close range. . . . . 67
19 Obstacle scenario. Qwen3.5:2B. Approach to the same blockage: the baseline keeps FOLLOW_LANE at speed until close to the obstacle (a), then attempts to go around it with two CHANGE_LANE_RIGHT actions (b, c) before returning to FOLLOW_LANE (d). The reaction comes late and the maneuver, as the rationale shows, is not part of a deliberate plan. . . . . 69
20 Pedestrian scenario. Claude Haiku 4.5. Graduated response to the crossing pedestrian: (a) FOLLOW_LANE while the pedestrian is still far; (b) SLOW_DOWN as the pedestrian begins to cross and the distance becomes unsafe; (c) STOP when the pedestrian is directly ahead, issued shortly after the Traffic Manager has already begun braking; (d) SLOW_DOWN once the pedestrian leaves the lane but remains nearby. The reasoning matches the action at each step. . . . . 69
21 Pedestrian scenario. Qwen3.5:2b. Committed sequence in front of the crossing pedestrian: (a) FOLLOW_LANE on approach; (b) SLOW_DOWN; (c) STOP in front of the pedestrian; (d) FOLLOW_LANE resuming once the pedestrian is no longer in the lane. Behind this apparently safe sequence, the baseline stopped compulsively throughout the run and, at one point with the pedestrian 6.5 m still ahead, briefly commanded FOLLOW_LANE instead of braking or proceeding slowly. Finally when the pedestrian is clearly outside the front cone the vehicle continues with FOLLOW_LANE . . . . . 70
22 Ambulance scenario, LiDAR point cloud shown for visualization only. The LiDAR is *not* part of the framework and does not feed the planner: the ego relies solely on the pseudo-sensors that build the scene context (Section 3.1.1), and this view is used here merely to make the ambulance visible to the reader. The point cloud shows a vehicle immediately behind the ego, the emergency vehicle, which remains in that position throughout the scene without overtaking. Because it approaches from the rear, outside the frontal camera's field of view, and is reported only as a generic CAR in the rear sectors of the object channel, the planner never receives the information a competent driver would need to make way for the ambulance. . . . . 72
23 Front camera frame corresponding to the reference model's call cited above (free run, call #17). The palm trees and the surrounding buildings that the model reports in its analysis are both present in the image, neither of which appears in the textual scene context, confirming that the model attends to and incorporates the camera image rather than reasoning from the text alone. . . . . 75
24 The Gantt chart shows the initial planning and scheduling for the development of the thesis. . . . . 79

## List of Tables

2 Selected pseudo-sensors, grouped by category, with the magnitude each one provides and its role in the planning decision. Speed is always obtained in $m/s$. Orientation (*yaw*) is always obtained from a quaternion. . . . . 29
3 Perception parameters used in the scene-context processing. . . . . 35
4 Mapping from the relative angle $\theta_{\text{rel}}$ to the eight named directions. Positive angles lie to the left of the ego. . . . . 36
5 Mapping from the object classification codes of `derived_object_msgs/Object` to the labels used in the scene context. . . . . 38
6 Mapping from the traffic-light state codes of `carla_msgs/CarlaTrafficLightStatus` to the labels used in the scene context. . . . . 38
7 Translation of each high-level action into a default target speed and a Traffic Manager command. Actions without a discrete command affect only the target speed. 49
8 Minimum commitment duration per action, during which a different non-emergency action is ignored. . . . . 51
9 Safety-layer parameters of the action dispatcher. . . . . 51
10 Models compared over the five scenarios. . . . . 52
11 Hardware and software configuration used for all experiments. . . . . 57
12 Aggregate metric comparison over the five scenarios. . . . . 59
13 Action distribution aggregated over the five scenarios, comparing the actions *requested* by the planner (from the debug files) against those *committed* by the dispatcher after the safety layer (from the dispatcher logs). The obstacle scenario is counted once (second run), consistently with Table 14. . . . . 60
14 Decision quality per scenario and model (Qwen3.5:2B vs Claude Haiku 4.5). The obstacle scenario, run twice, is reported per run. The aggregate row counts each scenario once, using the second obstacle run. . . . . 73
15 Control-layer events per scenario and model (Qwen3.5:2B and Claude Haiku 4.5), from the dispatcher logs. The obstacle scenario is reported per run. . . . . 74
16 Approximate project budget. . . . . 81

## List of Abbreviations

| | |
|---|---|
| **ALSM** | Agent Lifecycle and State Management (CARLA Traffic Manager) |
| **API** | Application Programming Interface |
| **BT** | Behavior Tree |
| **CNN** | Convolutional Neural Network |
| **CoT** | Chain-of-Thought |
| **CPU** | Central Processing Unit |
| **CUDA** | Compute Unified Device Architecture |
| **DDS** | Data Distribution Service |
| **FSM** | Finite State Machine |
| **GNSS** | Global Navigation Satellite System |
| **GPU** | Graphics Processing Unit |
| **H1, H2** | First and Second Hypotheses |
| **HTML** | HyperText Markup Language |
| **IMU** | Inertial Measurement Unit |
| **JPEG** | Joint Photographic Experts Group |
| **JSON** | JavaScript Object Notation |
| **LiDAR** | Light Detection and Ranging |
| **LLM** | Large Language Model |
| **PID** | Proportional-Integral-Derivative (controller) |
| **QoS** | Quality of Service |
| **RGB** | Red-Green-Blue |
| **RL** | Reinforcement Learning |
| **ROS2** | Robot Operating System 2 |
| **RQ** | Research Question |
| **TACL** | Transactions of the Association for Computational Linguistics |
| **TL** | Traffic Light |
| **TM** | Traffic Manager (CARLA) |
| **TMLR** | Transactions on Machine Learning Research |
| **VLM** | Vision-Language Model (also Visual Language Model) |
| **VRAM** | Video Random Access Memory |
| **XML** | eXtensible Markup Language |

# 1 Introduction

Autonomous navigation can be defined as the capability of an agent to successfully travel in a given environment, from point A to point B, without any kind of external guidance nor assistance. Throughout the navigation, an autonomous agent must be capable of taking optimal high-level decisions, whatever the surrounding conditions might be, in order to complete the task.

Autonomous robots are already being successfully implemented into closed, deterministic environments, such as a warehouse plant, where the intervention of human agents is minimum, the terrain is easily navigable, and other agents' actions are highly predictable. However, current methods of high-level planning, including behavior trees, finite state machines, and other rule-based systems, struggle in environments where uncertainty is high, or when a certain degree of complexity is introduced. These methods cover only a predefined and finite set of situations so, upon a non considered scenario, they do not generalize and become blocked or default to a generic fallback, failing to progress in their actions. Autonomous vehicles faithfully illustrate this. These often require to navigate through spatially complex environments where the surrounding agents add high uncertainty and a degree of complexity to the task planning. A fixed rule-based system is limited to navigate these kinds of scenarios using generic and deterministic set of behaviors, and therefore cannot anticipate every situation an open environment presents. Two scenarios illustrate this need clearly: the approach of an ambulance and the overtaking of a bicycle. Both require a genuine understanding of how the real world and how its actors behave to navigate optimally. For example, the coming of an emergency vehicle on duty requires, when possible, the maximum flexibility of the vehicle obstructing the emergency vehicle's path, which has to detect the emergency vehicle and move out of the way, sometimes even needing to break the traffic rules. Another illustrative example is the overtaking of a bicycle, which requires flexibility and thorough analysis of surrounding conditions to decide the opportune time frame of execution to safely overtake the cyclist. Capturing this kind of world knowledge in a fixed rule set is unmanageable, these scenarios demand a planner that is adaptable enough to reason about unseen situations rather than enumerating them in advance.

Large Language Models (LLMs) offer a promising route toward this capability. They are a category of deep learning models trained on immense amounts of data, capable of processing and generating natural language. These models not only excel at handling sequences of words but also are capable of producing plausible outputs to problems never seen before. Furthermore, the vast training data turns LLMs into wide generic knowledge sources, making them adaptable to many tasks in a zero- or few-shot manner. Therefore, LLMs appear as suitable candidates for navigation task planning, potentially overcoming the rigidity of rule-based systems. However, language alone leaves the planner blind to its surroundings. Visual Language Models (VLMs) extend LLMs with visual processing, providing the real-world grounding that the planning task requires. These capabilities make VLMs interesting candidates for high-level planning in open environments such as the road. Delegating driving decisions to a probabilistic model in a loop, in which safety is critical, is far from trivial, as hallucinations or inference latency can translate into dangerous maneuvers. This tradeoff, between the flexibility a VLM brings and the reliability autonomous driving demands, is the concern that motivates this thesis, which studies whether a VLM can act as the high-level planner of an autonomous vehicle when a dedicated control layer safeguards its decisions.

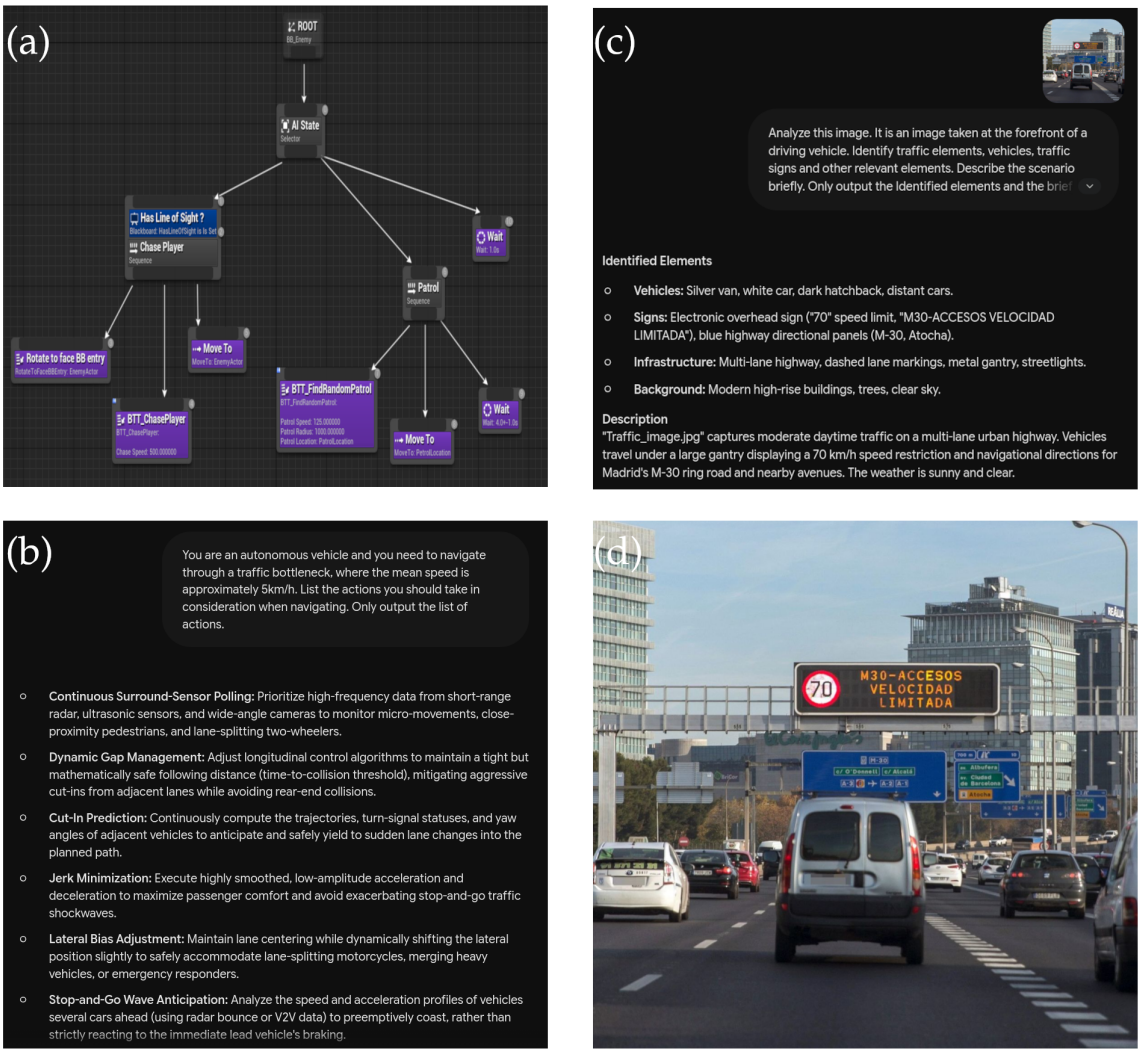


Figure 1: Comparison of autonomy approaches: (a) Traditional *Behavior Trees* require structured, rigid, and predefined logic [1]. In contrast, Deep learning Models demonstrate high flexibility without prior specific training (zero-shot), both in (b) reasoning and planning of complex tasks via text (LLM), and in (c) semantic understanding and description of dynamic environments from images (VLM). (d) Image of Madrid's M-30, used for the VLM analysis shown in (c). This image is sourced from "EFE"

## 1.1 Hypotheses

Three claims guide this work. The first two are formulated as hypotheses to be tested while the third is posed as an open question, since the experimental setup of this thesis does not allow it to be settled.

The first hypothesis (H1) states that the architecture can guide the vehicle by taking well-grounded mid-term driving decisions, while the low-level control layer acts as a safety substrate that keeps navigation viable through the model's long latency periods. This implies that the vehicle completes the navigation scenarios without executing dangerous maneuvers, and that the safeguards absorb the model's instabilities and inference delays rather than letting them propagate to the vehicle behavior and interrupt navigation continuity.

The second hypothesis (H2) explores whether the proposed framework is model-agnostic (i.e. it operates with interchangeable models without modification) and the quality of its decisions scales with the capability of the model employed. This implies that a higher-capability model produces measurably better decision quality than a smaller one.

Finally, the research question (RQ) asks whether the visual input improves the planning by providing real-world grounding that the textual scene context cannot express. This is treated as an open question rather than a hypothesis because every run is executed with the image and no text-only condition is available for contrast. The current design cannot isolate the contribution of the visual modality, and a dedicated ablation study would be required to answer it.

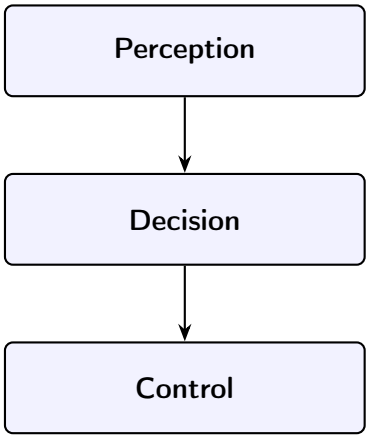


Figure 2: Abstracted hierarchical architecture framework diagram.

## 1.2 Objectives

This thesis aims to design, develop and validate a VLM-based high-level planner for an autonomous vehicle within a hierarchical framework with a dedicated safety layer. This general objective is decomposed into a set of specific objectives that span the construction of the framework, its integration and validation in simulation, and the empirical study of its behavior and of the role of its inputs. The specific objectives are the following:

- To structure the framework as three decoupled ROS2 modules in charge of the perception, high-level planning, and low-level control, respectively.
- To fuse the perception into a structured scene context representation that grounds the

VLM in its surroundings.

- To design the prompt in a way that accommodates the structured scene context while guiding the model's behavior to output specific high-level actions.
- To implement a system that translates high-level actions into low-level control actions.
- To implement a safety control layer that handles the model hallucinations and inference latency periods, ensuring continuity of the navigation.
- To integrate and validate the architecture in CARLA simulator within reproducible experimental conditions.
- To evaluate the framework's behavior and robustness across five scenarios of increasing complexity, using key metrics that quantitatively assess the results.
- To evaluate model dependency of the framework across two models, a baseline model and an upper-bound model.
- To assess the contribution of the visual input to the action planning.

## 1.3 Scope

This work covers the design, development, validation, and evaluation of a VLM-based high-level action planner for mobile robots. The application domain is autonomous driving, and CARLA is therefore used as the simulation environment. The framework is tested solely in simulation, in order to isolate the planner from the risks and the added complexity that real hardware would imply.

The thesis focuses on the high-level reasoning capabilities of the VLM as a task planner, while most of the low-level control, including trajectory generation and actuation, is delegated to CARLA's Traffic Manager. Perception is also abstracted, relying on the pseudo-sensors provided by the carla-ros-bridge package, which exhibit simulator ground truth, rather than real perception pipelines such as computer vision detection.

Although the application is an autonomous vehicle, navigation is purely reactive: the vehicle takes a decision over an instant on each inference cycle, with no predefined route or target destination, which isolates its local decision-making capacity.

Regarding the models, two VLMs are studied, an on-device baseline and an upper-bound cloud reference, as two representative points along the capability range. The aim is to characterize how decision quality varies with model capability, not to rank models. Both are used in a zero-shot manner, without any task-specific fine-tuning or any full prompt exemplification, so as to evaluate general off-the-shelf capability rather than the performance of a specialized model.

Furthermore, the evaluation is conducted on a single inner-city urban environment (Town10HD), chosen to keep the reproducibility of the experiments within a representative scenario.

Finally, the framework is built on standard Python libraries and ROS2 nodes, and the validity

of the code logic is verified through unit tests with pytest.

## 1.4 Contributions

The contributions of this work fall into two groups: the engineering of the framework itself and the empirical findings drawn from its evaluation.

On the engineering side, the central contribution is a functional hierarchical framework that integrates a VLM as the high-level planner of an autonomous vehicle within a ROS2 and CARLA pipeline, connecting perception, decision-making, and low-level control end-to-end. Within this framework, a dedicated safety layer is proposed: a set of safeguards, including an action whitelist, a STOP override, an action commitment mechanism, a speed ramp, a stale output watchdog, and a safe conversion of the model's numeric outputs. These keep the vehicle's behavior within a viable pathway and ensure continuity of the navigation despite the instabilities and the latency of the model. The framework also provides a model-agnostic provider abstraction that allows different VLMs, served either locally or through cloud APIs, to be interchanged without any modification of the architecture. Finally, a reproducible experimental setup is contributed, comprising 5 different deterministic scenarios, with a fixed spawn point, fixed seeds, and distance-based triggers, together with a set of parsing and evaluation tools that turn the raw model outputs into quantitative metrics, allowing the experiments to be repeated and measured consistently.

On the empirical side, the evaluation provides evidence that the control layer allows a small, unstable on-device model to navigate viably, absorbing its unstructured outputs, hallucinated actions, and inference delays rather than letting them propagate to the navigation. It also offers a characterization of how decision quality scales with model capability across the five scenarios. Although the higher-capability model produces measurably cleaner decisions, the gap in driving outcome remains small because the control layer compensates for the weaker model. Lastly, the study further contributes with a characterization of a sensing limitation of the framework, regarding its inability to handle emergency vehicles, which the data points to the single front-facing camera coverage and the absence of an emergency category in the object channel rather than to a reasoning failure of the models.

## 1.5 Thesis Outline

The rest of this document is organized as follows. Section 2 reviews the state of the art in high-level planning for autonomous navigation, covering traditional rule-based methods, learning-based approaches, and the recent use of LLMs and VLMs as planners. Section 3 describes the methodology: the platform and environment, the three-module architecture, and the experimental setup. Section 4 presents the results of the five-scenario campaign, both quantitatively and qualitatively. Section 5 discusses these results in relation to the hypotheses and examines the limitations of the framework. Section 7 addresses the project budget and its sustainability and ethical implications. Finally, Section 8 draws the conclusions and Section 9 outlines future work.

# 2 State-of-the-art

## 2.1 High-level Planning: Traditional Methods

High-level planning for autonomous navigation has traditionally been addressed with explicit, deterministic decision structures. Finite state machines (FSMs), behavior trees, and rule-based systems encode the agent's behavior as a fixed set of states or nodes and the transitions between them, so that at every instant the current situation is matched against a predefined condition and the corresponding behavior is executed. Each of these structures encodes the decision logic explicitly. A finite state machine represents behavior as a finite set of discrete states, each corresponding to a mode of operation such as "follow lane" or "stop at intersection", together with transition rules that move the agent from one state to another when a condition on the perceived situation is met. A rule-based system generalizes this idea to a set of condition-action rules that are matched against the current state to select the next action [2, 3]. Behavior trees organize the logic differently, as a tree of tasks that is traversed from the root at a fixed rate, or tick, with internal nodes (such as sequence and fallback) composing the execution of their children and leaf nodes performing the actual checks and actions [4]. This periodic traversal is what makes them reactive, since the whole tree is re-evaluated on every tick [4]. In all three cases the designer writes out, ahead of time, which situations the agent will encounter and how it should respond to each.

This paradigm guided the first generation of urban autonomous driving. In the 2007 DARPA Urban Challenge most teams controlled high-level behavior with finite state machines [5], and the winning vehicle, Boss, structured its behavioral layer as a hierarchical state machine that decomposed driving into top-level behaviors such as lane driving, intersection handling, and zone maneuvering [6]. Behavior trees were later introduced as a more modular and reactive alternative, and have since spread from computer-game AI to many branches of robotics [4].

The strength of these methods is that they are interpretable, verifiable, and deterministic. Their logic is written out explicitly, which makes their decisions auditable and their guarantees easy to reason about, and it is for this reason that they remain the standard in structured and controlled environments. Their weakness appears when complexity and uncertainty grow. Because they enumerate a finite, predefined set of situations, they do not generalize to circumstances that were not anticipated during the design of the robot, and upon encountering an unforeseen scenario they either block or fall back to a generic behavior. Finite state machines in particular are known to produce erratic behaviors as complexity increases, where adding a new case tends to break existing functionality [4]. This rigidity is the fundamental limitation of the deterministic approach in open, uncertain environments such as urban driving, and it is what motivates the turn towards methods that learn their behavior from data rather than having it enumerated in advance.

## 2.2 Learning Methods

The rigidity of deterministic decision structures motivated a shift towards methods that learn their behavior from data rather than having it enumerated in advance. Three broad families fall under this paradigm. In imitation learning, a policy is trained to reproduce expert demonstrations, mapping observed situations to the actions a human operator copied. In reinforcement learning (RL), an agent instead learns a policy by interacting with the environment and optimizing a scalar reward in a feedback loop, discovering behaviors that maximize long-term return. Finally, in end-to-end driving, a single model is trained to map raw sensor input directly

to control commands, collapsing perception, planning, and control into one learned function. The three families differ in what they learn from and how. Imitation learning treats driving as a supervised problem. Given a dataset of situations paired with the actions an expert took, it fits a function that reproduces those actions on new, similar situations. Reinforcement learning has no such labelled examples but instead the agent tries actions, receives a scalar reward that measures how good the outcome was, and gradually adjusts its policy, the mapping from situations to actions, so as to maximize the reward accumulated over time. End-to-end driving refers not to a distinct learning signal but to the scope of the learned function. Rather than dividing the pipeline into separate perception, planning, and control stages, a single model is trained to produce control commands directly from sensor input, so that any intermediate representation it needs is discovered during training rather than designed manually.

An early example is the work of Bojarski et al., who trained a convolutional neural network to map the pixels of a single frontal facing camera directly to steering commands, and showed that the network learned useful road features from the steering signal alone, without any explicit lane-detection stage [7]. Reinforcement learning has also been applied across the full range of driving tasks, from lane keeping and merging to full urban navigation, as surveyed by Kiran et al. [8].

These methods promise that, by learning from data, they generalize to situations that a fixed rule set could not anticipate. Their weakness is complementary to their strength. The learned policy is a black box whose decisions are not interpretable, which makes it hard to verify and to certify for safety, and training it typically demands large amounts of data. Imitation learning carries an additional, well-characterized failure mode because future observations depend on the policy's own past actions. Small errors accumulate and drive the agent into states that were never covered by the expert demonstrations, where its behavior degrades [9]. Reinforcement learning avoids this particular problem by learning from interaction, but shifts the difficulty onto the design of the reward function, which must capture the implicit understanding of good driving and is itself hard to specify. This last difficulty has recently been approached by using LLMs to generate or evolve reward functions from natural-language descriptions of the task [10, 11], an early sign of language models entering the driving pipeline, though in a role different from the high-level planning studied in this work.

These learning-based methods thus trade the interpretability of the rule-based approach for flexibility, gaining generalization at the cost of transparency and verifiability. The framework proposed in this thesis is placed deliberately between the two paradigms, delegating high-level reasoning to a learned model, the VLM, while confining it behind a deterministic control layer that keeps the parts of the system that must be verifiable outside the black box. The following blocks review the language-based models that make this high-level reasoning possible.

## 2.3 World Models

A more recent and still emerging line of work replaces the reactive stimulus-response mapping with an explicit model of the world. A world model learns a compact internal representation of the environment together with its dynamics, this is translated to how the scene is expected to evolve as a function of the agent's actions, so that the agent can predict future states and plan over them rather than only reacting to the present observation. The idea was popularized by Ha and Schmidhuber, who trained a generative latent model of an environment and showed that a controller could be learned entirely inside the imagined scenarios of that model, without fur-

ther interaction with the real environment [12]. In autonomous driving the paradigm has been taken up by large generative world models such as GAIA-1, which, conditioned on past frames together with text or action inputs, generates plausible future driving scenes, learning a predictive model of the road environment directly from video [13]. World models are conceptually close to the model of the plant used in model-based control, but learned from data and operating on high-dimensional visual observations rather than on a hand-derived state. They are relevant to this work as a possible route towards scalability and the transfer from simulation to the physical world, and are revisited among the directions for future development (Section 9).

## 2.4 LLM / VLM Fundamentals

Large language models, and their visual extension, are the central tool of this work, as the planner at the core of the proposed framework is a vision-language model. Since the intended audience of this document comes primarily from a control and robotics background, this block introduces the concepts needed to follow the thesis, to provide a better general understanding of the framework's "brain".

These models operate on language. A large language model (LLM) is a deep-learning model trained on vast amounts of text with a self-supervised objective, predicting and concatenating the highest most probable next token in a sequence [14]. From this objective, emerges the ability to produce and manipulate natural language, together with a wide storage of common knowledge that lets the model be applied to many tasks in a zero or few-shot manner, meaning that, they can be applied to perform tasks only from a task description and few or no examples at all given in the input, without specific re-training for the task [14]. Several of these capabilities are not present in smaller models and appear only as the model is scaled up [15], a property directly relevant to the model comparison carried out in this work. In practice the model is steered through a prompt, a piece of natural language that describes the task and the desired behavior, so that adapting an LLM to a new problem often consists in rewriting instructions rather than retraining the weights of the model.

The architecture that made this possible is the transformer [16] (Figure 3). Its central component is the attention mechanism, which, when computing the representation of a given token, lets the model weigh the relevance of every other token in the sequence and draw information selectively from the most relevant ones. Because this weighing is applied across the whole input at once rather than step by step, the transformer captures long-range dependencies between distant parts of the text more effectively than the recurrent architectures that preceded it, and does so in a form that parallelizes well and scales to very large models and datasets.

A vision-language model (VLM) extends this design to visual input. Rather than receiving text alone, the model also takes an image, which is processed by a vision encoder into a sequence of vectors lying in the same representation space as the text tokens. These visual vectors are then processed together by the same attention mechanism, so that the model reasons over image and text within a single sequence. PaLM-E illustrates this approach in a robotics setting, injecting encoded camera observations directly into the language model alongside the textual input, and using the combined representation for embodied reasoning and planning [17]. It is precisely this joint processing that provides the real-world grounding a purely textual planner lacks, and that makes VLMs candidates for high-level planning in visually rich environments such as the road.

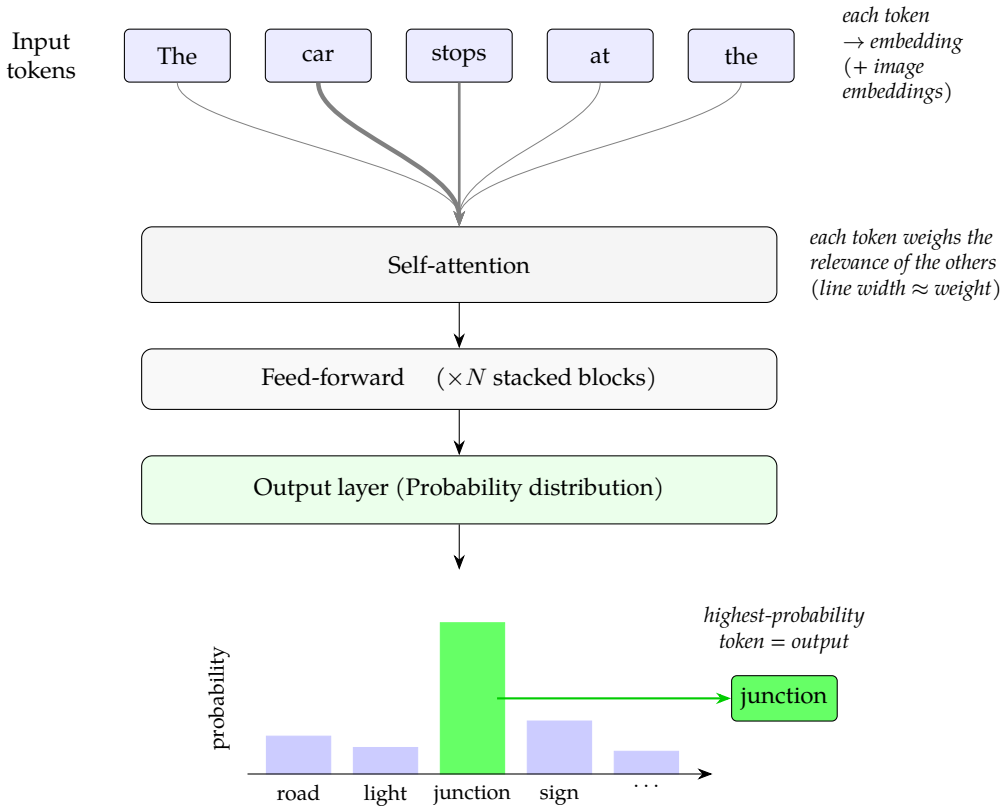


Figure 3: Conceptual view of a transformer processing a sequence. Each input token is mapped to an embedding. A self-attention layer lets every token weigh the relevance of the others (line width suggests the attention weight), and a stack of such blocks feeds a final output layer that produces a probability distribution over the next token in the chain. The highest-probability token is returned as the model's output.

## 2.5 LLMs / VLMs Applied To Autonomous Driving And Robotics

The fundamentals of the previous block make LLMs and VLMs candidates for a specific role, the high-level planner of a physical agent. This block reviews how they have been used in that role, moving from general robotics towards autonomous driving, and from flat designs towards the hierarchical, safeguarded structure adopted in this work.

### 2.5.1 Language models as high-level planners

There are several frameworks that use the LLM not to actuate the robot but to reason about what to do, delegating execution to lower-level primitives. SayCan is a very representative framework of this application in robotics. The language model proposes natural-language actions from its semantic knowledge of the task, while a set of pretrained low-level skills, scored by their affordances, constrains those proposals to what is actually feasible in the current situation [18]. The language model supplies the "what", the robot's skills supply the "can". This division, a probabilistic planner proposing and a grounded layer filtering, is the same principle on which the present work relies on. Other variants change the interface between the two levels. Code as Policies has the LLM emit executable programs that call perception and control primitives [19], while PaLM-E uses a single multimodal model to produce high-level plans from interleaved image and text [17]. At the opposite extreme, RT-2 collapses the hierarchy entirely, training a single vision-language-action model that maps images and instructions directly to low-level robot commands [20]. This end-to-end route is powerful but returns to the black box issue, providing a very hard to verify regime, already discussed earlier, and it is precisely the design this work avoids by keeping planning and control separate.

### 2.5.2 Language-based navigation

Closer to the driving domain, a line of work applies LLMs and VLMs to navigation. NavGPT performs zero-shot vision and language navigation by reasoning explicitly over textual descriptions of the visual scene, the navigation history, and the available directions, decomposing the instruction into sub-goals and tracking progress [21]. VELMA operates in real Street View imagery, verbalizing the agent's trajectory and surroundings into language that the LLM reasons over to decide the next move [22]. These systems show LLMs handling sequential, perception-grounded decisions in navigation, though in discrete graph-like environments rather than the continuous, dynamic traffic that driving implies.

## 2.6 Closed-loop planning and correction

A recurring topic is that a language model's plan cannot be trusted in open-loop, since insufficient grounding leads to hallucinated or infeasible steps. Several works close the loop around the planner. Bhat et al. propose BrainBody-LLM, which splits planning across two language models, one for high-level reasoning and one for low-level control, and feeds execution errors back to the planner to correct them, reporting that state feedback mitigates the hallucinations that ungrounded planning produces[23]. Sharma et al. use natural-language feedback to correct a planner's cost function after a failure, recovering from planning errors that the original specification could not handle [24]. Both share this work's principle, that the raw output of a language planner must be checked and corrected before or during execution, though here that role is played by a deterministic control layer rather than by another language model or a human in the loop.

## 2.7 Hierarchical language-based planning in driving

The design closest to this thesis places a language model at the top of a hierarchical driving stack. Lin Li et al. augment a hierarchical reinforcement-learning framework with an LLM that acts as an expert high-level planner, interpreting textual descriptions of the driving scenario and generating a long-term goal that a lower-level RL policy then executes, and report improved generalization and interpretability over flat RL [25]. This is the same separation adopted in this thesis, with one decisive difference, their low-level layer is itself learned, whereas the present work keeps it deterministic so that the verifiable part of the system stays outside the black box. Jinning Li et al. had already proposed a safe hierarchical planning framework in which a high-level reinforcement-learning coordinator selects among low-level controllers with safety guarantees [26]. Taken together, these works establish both the hierarchical structure and the language-based high-level planner that this thesis combines, and set up the question of what, specifically, remains to be contributed, addressed in the next block.

## 2.8 Research gaps addressed

The works reviewed above establish the two ingredients this thesis builds on, a language model acting as a high-level planner, and a hierarchical decomposition that separates that planner from execution. What remains open is the specific combination of the two, and the role assigned to the layer that safeguards the planner.

In most of the reviewed systems, whatever constrains the language planner is itself learned or probabilistic, or depends on a human. SayCan filters the planner's proposals through learned affordances [18], BrainBody-LLM corrects one language model with another [23], Sharma et al. relies on human language feedback [24], and the hierarchical driving framework of Li et al. places a learned reinforcement-learning policy underneath the LLM planner [25]. This thesis instead pairs the probabilistic planner with a deterministic control layer, so that the part of the system responsible for safety is verifiable and independent of the model's reliability. The contribution is not the use of a VLM as a planner, already demonstrated elsewhere, but the study of a deterministic safety layer acting as a substrate that keeps navigation viable in spite of an unreliable planner, which is the object of the first hypothesis (H1).

Two further aspects distinguish this work. First, the same framework is evaluated with a baseline model and a near-frontier, upper-bound reference model without any change to the architecture, characterizing how decision quality scales with model capability within a fixed design, the question posed by the second hypothesis (H2). The reviewed works each commit to a single model and do not isolate this effect. Second, the framework is built as three decoupled ROS2 modules, perception, decision, and control, which confines the probabilistic reasoning of the VLM to a single stage and keeps the components around it deterministic, an engineering structure that also makes the planner replaceable and the safety layer auditable in isolation.

These gaps define what follows. The next section describes the methodology that realizes this design, the three-module hierarchical architecture, the perception front-end that grounds the VLM, the decision module that queries it, and the low-level control module in which the deterministic safety layer lives.

# 3 Methodologies

## 3.1 System Architecture

The proposed system is structured as a hierarchical framework, composed of three decoupled modules, each responsible for one area of the autonomous vehicle's pipeline: perception, high-level action planning, and low-level control. This hierarchical and modular structure is a common design choice in autonomous robotic systems [27, 26], as it tackles the problem separating it into independent areas that can be developed, tested, and replaced in isolation. In this work, the decoupling additionally confines the probabilistic, and potentially unreliable, reasoning of the VLM to the planning stage, while keeping perception and low-level control as deterministic components around it.

Each of these modules is implemented as a ROS2-structured python package and its source contains a single ROS2 node that governs the logic of its respective area. The modules communicate exclusively through ROS2 topics, publishing and subscribing to the messages that carry data along the pipeline. This communication design is what makes the modules decoupled, allowing each one to only depend on the format of the data it receives, not on the internal implementation of the others. In this way, any module can be modified or substituted as long as it respects the topic format.

The framework, however, does not interact with the vehicle directly but through CARLA simulator, which cannot communicate with ROS2 natively. This connection is provided by the carla-ros-bridge package, which acts as a translator between the two endpoints. It provides data from CARLA's sensors and actuators as ROS2 messages, through topics, allowing the nodes to read the vehicle's state and surroundings and to send control commands as if they were standard ROS2 messages. The bridge is therefore the boundary between the simulated vehicle and the ROS2 side of the framework, and the pseudo-sensors it provides are the only source of perception data used by the system.

The data flow across the framework is sequential. The perception module packs the data from the vehicle's surroundings into a structured scene context; the decision-making module turns that context, together with a front camera image, into a high-level driving action through the VLM, and the low-level control module translates that action into vehicle behavior while safeguarding it against unsafe commands. Figure 4 shows the diagram of the architecture described. The following subsections describe each of these three modules in detail.

### 3.1.1 Perception module

The perception module is responsible for sensing the ego vehicle's environment and building a structured context that is subsequently sent to the decision-making module. Its logic is governed by the node 'scene_context_node.py'. This node aggregates the data from the pseudo-sensors provided by the carla-ros-bridge, which provide a ROS2 interface to the simulator's ground-truth state, and therefore allows the system to obtain real-time data from the surroundings of the ego vehicle. Then, the data is processed to obtain the relevant magnitudes for the structured scene context, some of which are abstracted to high-level perception concepts in order to reduce the reasoning burden on the model. Finally, the processed data is structured into XML format and published at 1 Hz on the `/scene_context` topic. Figure 5 depicts the complete flow described.

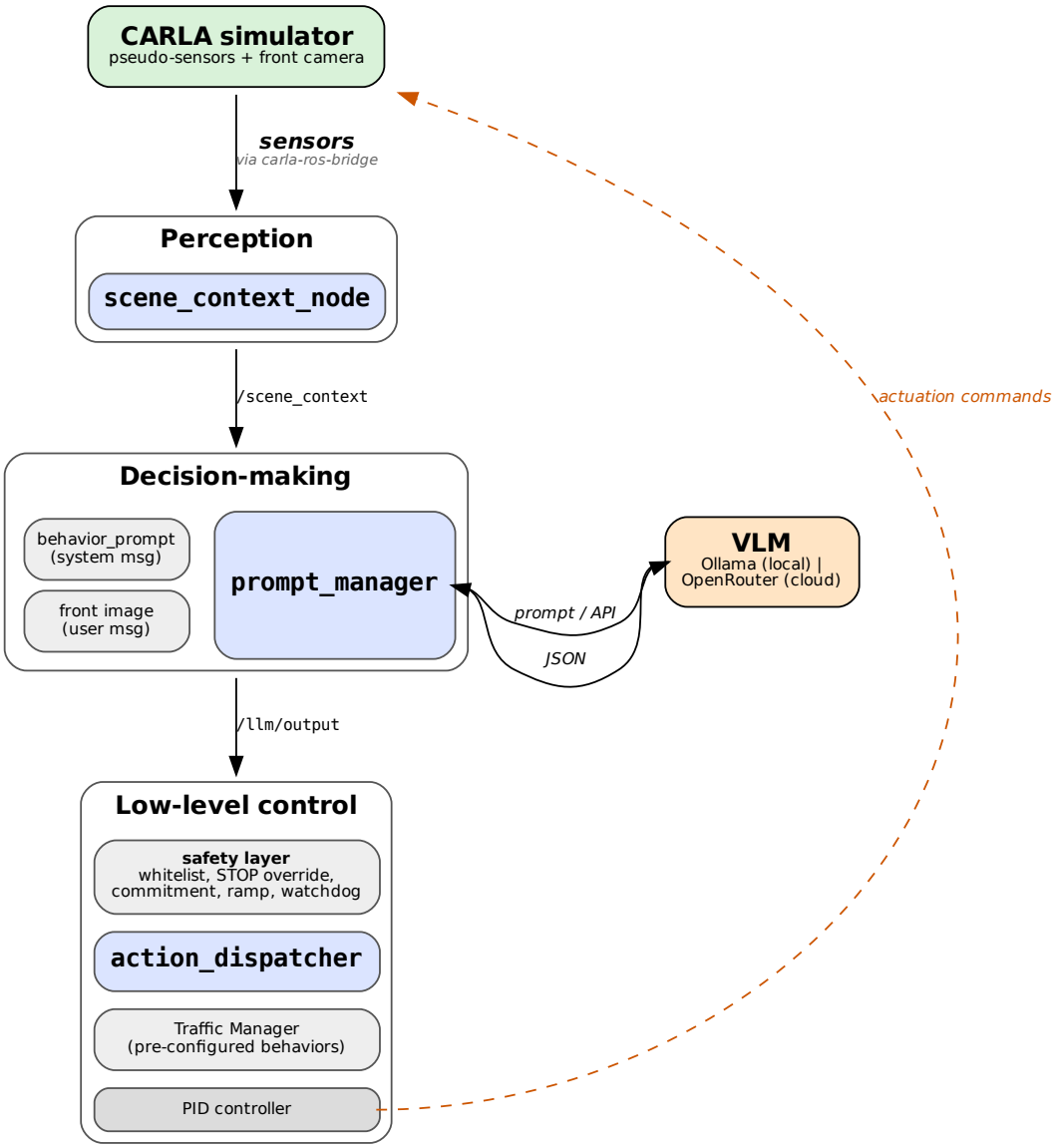


Figure 4: Complete system architecture. CARLA exposes its sensors and actuators through the `carla-ros-bridge`, the only boundary between the simulated vehicle and the ROS2 side. The perception node (`scene_context_node`) packs the sensor data into a structured scene context, published on `/scene_context`. The decision node (`prompt_manager`) combines that context with the behavior prompt and the front camera image and queries the VLM through an API, publishing the validated action on `/llm/output`. The control node (`action_dispatcher`) filters the action through the safety layer and executes it via the Traffic Manager's pre-configured behaviors, built on a PID controller. The dashed arrow closes the loop, returning actuation commands to the vehicle.

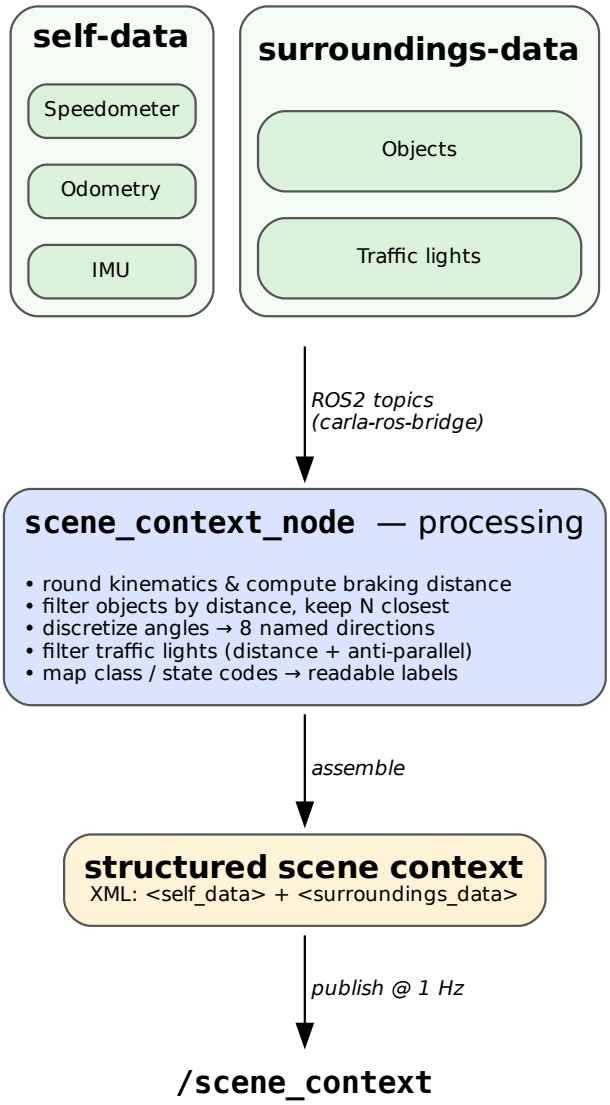


Figure 5: Perception pipeline. The `scene_context_node` aggregates the self-data and surroundings-data pseudo-sensors exposed by the carla-ros-bridge, processes them into the relevant magnitudes (rounding kinematics and deriving the braking distance, filtering and discretizing the surrounding objects and traffic lights, and mapping raw values to readable labels), assembles them into an XML scene context, and publishes it at 1 Hz on the `/scene_context` topic.

#### 3.1.1.1 Pseudo-sensors

Although the carla-ros-bridge exposes a broad set of pseudo-sensors, only a subset of them is included in the framework, in order to limit the information load on the model and to keep an adequate level of abstraction for the high-level planner. As explained in Section 2, LLMs and VLMs can process a limited number of tokens at a time, the context window, whose size depends on the model. Every additional piece of data in the input prompt consumes tokens, and upon exceeding the context window, the input begins to degrade the model's behavior in two distinct ways. First, even within the context window, irrelevant or redundant data introduces noise that distracts the model's reasoning and can lead to hallucinations, an effect that is more pronounced in smaller or less capable models such as the baseline used in this thesis [28] (Figure 6). Long contexts also tend to be exploited unevenly, with relevant information being used less reliably as the prompt grows [29] (Figure 6). Second, when the input exceeds the context window, the two providers behave differently: cloud models reject the request, whereas the local model, served through Ollama, silently truncates the prompt from the beginning. In practice, this truncation discards the behavior specification at the start of the prompt, leaving the model without instructions and returning empty or invalid outputs. This effect was observed directly during the development of the thesis, feeding the front image at a higher resolution produced more visual tokens than the baseline's context window could hold, causing the request to fail, whereas a resolution of 320×180 kept the prompt within the limit and restored valid responses.

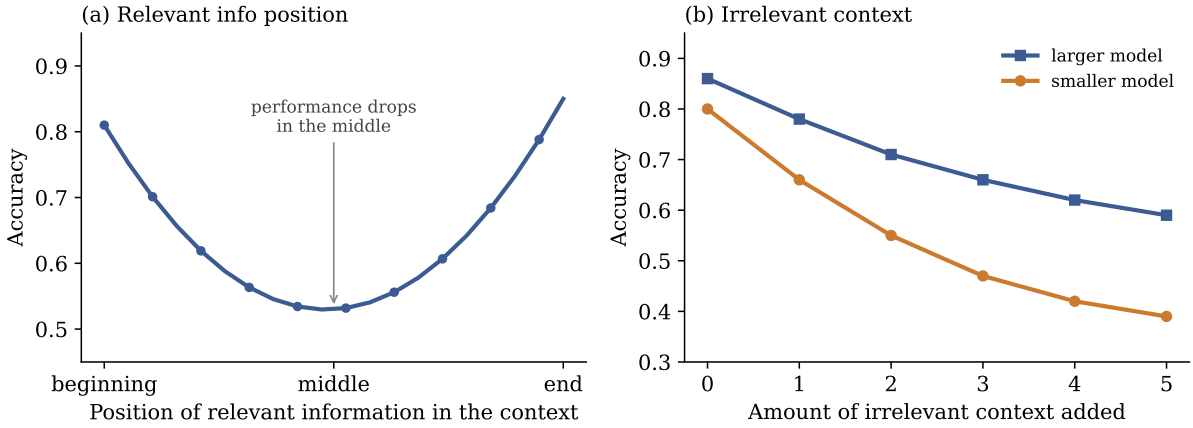


Figure 6: Schematic illustration of the mentioned two ways in which the input context degrades language-model performance, reproduced qualitatively from the findings of the cited works. (a) When the relevant information is placed at different positions within a long context, accuracy is highest at the beginning and the end and drops in the middle [29]. (b) Adding irrelevant context to the prompt lowers accuracy, an effect that is more pronounced for smaller, less capable models [28]. The curves are schematic and reproduce the papers' reported trends, not the exact measurements.

On the other hand, the planner is designed to only decide high-level actions, not trajectories, so it requires a limited kind of information: what is around the vehicle, where, and at what speed. Richer sources such as the LiDAR or a depth camera would not add information relevant to high-level decisions, while introducing an additional processing layer for the framework to resolve before the model can reason. Moreover, CARLA provides pseudo-sensors such as the traffic-lights and objects sensors, that lower the complexity of the perception pipeline and deliver clean, precise information, which allows the planner's performance to be isolated and

evaluated without perception errors.

For the reasons above, a reduced set of pseudo-sensors was selected. They fall into two groups, following the structure of the scene context: those describing the ego vehicle's own state (*self-data*) and those describing its environment (*surroundings-data*). Table 2 summarizes the selected sensors, the magnitude each one provides, and its role in the planning decision. The front camera is handled separately by the decision-making module (Section 3.1.2) and is therefore not part of the scene context.

Table 2: Selected pseudo-sensors, grouped by category, with the magnitude each one provides and its role in the planning decision. Speed is always obtained in $m/s$. Orientation (*yaw*) is always obtained from a quaternion.

| **Sensor** | **Group** | **Magnitude** | **Role in the decision** |
|---|---|---|---|
| Speedometer | self-data | Ego speed | Basis to speed up, hold speed, or brake |
| Odometry | self-data | Ego pose (position, yaw) | Relative distances and angles to other actors |
| IMU | self-data | Acceleration, angular velocity | Instantaneous dynamics of the ego |
| Objects | surroundings-data | Nearby agents (class, position, speed) | What, where, and how fast around the ego |
| Traffic lights | surroundings-data | Position, orientation and state of relevant lights | Compliance with traffic-light rules |

Oppositely, a number of pseudo-sensors offered by the bridge were deliberately left out, for one of two reasons. The first group includes sensors that would feed a low-level perception pipeline that the framework abstracts: LiDAR, semantic LiDAR, radar, the depth camera, and the semantic segmentation camera all provide raw or pre-segmented data that would require an additional processing stage to recover the identity and location of the surrounding agents. This information is already delivered cleanly through the objects sensor, so incorporating these sensors would introduce further complexity to the perception pipeline, which lies outside the scope of this work.

The second group comprises sensors whose information is not relevant to a purely reactive, high-level planner. The GNSS sensor provides global geolocalization and the map sensor provides road-network information. Both are only useful for goal-oriented navigation, where a route towards a target destination must be planned. Since the planner studied here decides instant by instant with no predefined route (Section 1.3), this data would add irrelevant information to the current decision. However, these two sensors are a natural starting point for extending the framework towards goal-oriented navigation, and therefore are left as future work (Section 9).

#### 3.1.1.2 Synchronous and latched topics

To obtain the data from the pseudo-sensors, the node scene_context_node.py subscribes to the topics published by the carla-ros-bridge. These topics are published by 'bridge.py', which is in

turn launched by carla_ros_bridge.py, the launch file that brings up the package's nodes. Each pseudo-sensor exposes its information in a single topic, except the traffic-lights sensor, which exposes its static information (position and orientation) in one topic and light state in another. The node keeps the last value received from each subscribed topic and assembles them into the scene context.

Keeping only the last value of each topic raises a question of temporal coherence. Since the pseudo-sensors do not all publish at the same rate, a context assembled from independently arriving messages could, in principle, combine data from different instants. To avoid this, CARLA is run in synchronous mode. In this mode the simulator advances deterministically, tick by tick, with the client controlling when the world steps forward, so that the sensors produce their data for the same simulation tick and remain referred to the same simulated instant. This consistent origin is preserved by the node, so the assembled context represents a coherent state rather than a mix of moments.

However, the consistency does not extend to every topic. Most pseudo-sensors, including odometry, speedometer, IMU, and objects, publish continuously (in synchronous mode), so the node always holds a value tied to the current simulation step. The traffic-lights information, by contrast, is delivered through a *latched* topic, meaning that rather than being published at a fixed rate, the last message is retained and updated only when the state changes. As a result, between updates the node may reason over a traffic-light state that is not aligned with the current tick. The implications of this refresh rate for traffic-light compliance are discussed in Section 5.1.

The latched behavior is not a property of the data itself but of the Quality of Service (QoS) policy with which the topic is configured. ROS2 lets each publisher and subscriber negotiate a QoS profile that governs how messages are delivered, and one of its settings, the *durability* policy, determines whether nodes that subscribe later than initialization receive messages published before they connected. By default, topics use a `VOLATILE` durability, under which a subscriber only receives messages sent after it subscribes, while any earlier messages are lost. The traffic-lights topics, instead, are configured with `TRANSIENT_LOCAL` durability and a history depth of one (Listing 1, lines 2-3) which makes the publisher retain the last message and deliver it immediately to any subscriber that connects afterwards [30].

This policy is what allows the node to hold a valid traffic-light state even though the information is published only when it changes. Without it, a subscriber created between two updates, or at initialization, would receive nothing until the next change, leaving the scene context without traffic-light data for long intervals. The node therefore subscribes to the two traffic-light topics with a matching `TRANSIENT_LOCAL` profile, as shown in Listing 1.

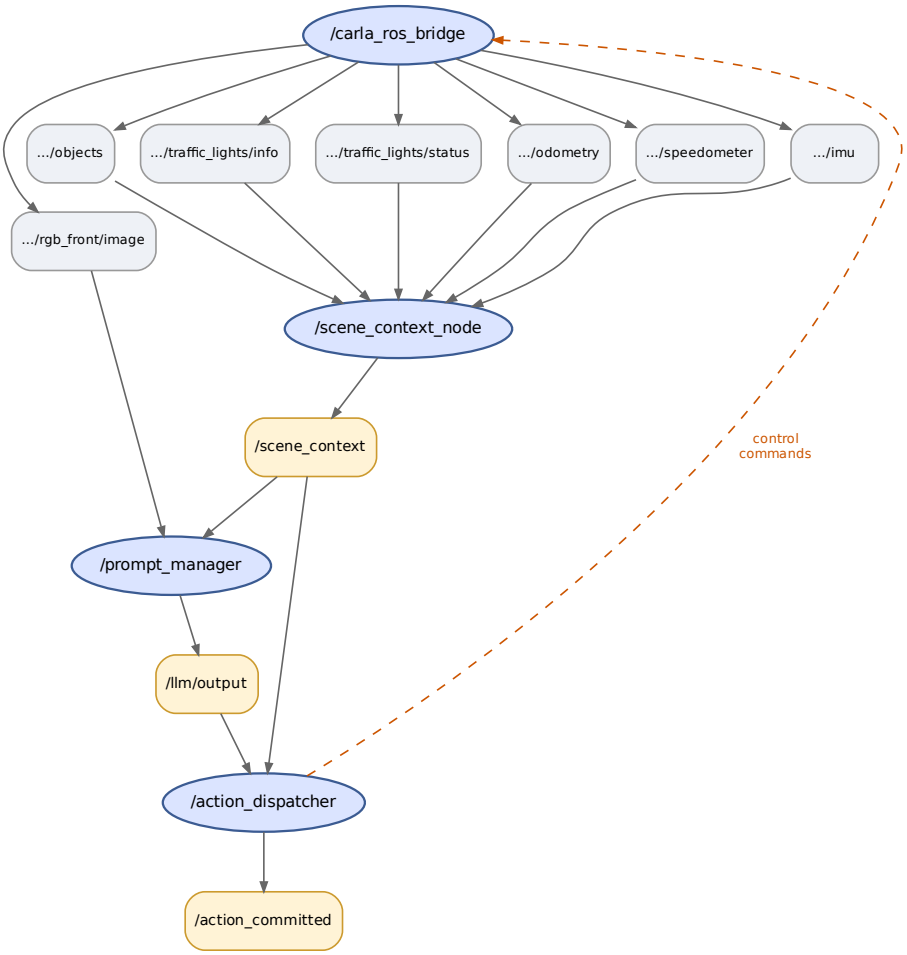


Figure 7: ROS2 computation graph of the framework. Ellipses are nodes and rectangles are topics, with arrows indicating publication and subscription. The `carla_ros_bridge` publishes the sensor topics (prefix `/carla/hero/`, abbreviated as `.../`) and the front camera image. The `scene_context_node` subscribes to the sensor topics and publishes `/scene_context`, consumed by both the `prompt_manager` and the `action_dispatcher`. The camera image is delivered directly to the `prompt_manager`, which publishes the validated action on `/llm/output`; the `action_dispatcher` consumes it, publishes the committed action on `/action_committed`, and returns the control commands to the bridge (dashed arrow).

```
qos_latched = QoSProfile(
    depth=1,
    durability=DurabilityPolicy.TRANSIENT_LOCAL
)

# Traffic-light subscriptions (latched topics)
self.create_subscription(
    CarlaTrafficLightInfoList,
    '/carla/traffic_lights/info',
    self.tl_info_callback,
    qos_latched
)
self.create_subscription(
    CarlaTrafficLightStatusList,
    '/carla/traffic_lights/status',
    self.tl_status_callback,
    qos_latched
)
```

Listing 1: Subscription to the latched traffic-light topics using a `TRANSIENT_LOCAL` QoS profile.

#### 3.1.1.3 Data processing

Once the latest data from every topic is available, the node does not publish it as received but processes it into the magnitudes that compose the scene context. Two goals guide this processing, which include extracting the information that is actually relevant to a high-level decision, and simplifying it to reduce the reasoning load on the model, particularly the baseline one.

Regarding the ego vehicle, the raw kinematic values are rounded to a fixed precision: speed and yaw to two decimal places, the accelerations along the $x$ and $y$ axes and the angular velocity about the $z$ axis to three, and the braking distance, introduced below, to one. These precisions are enough for a high-level decision while keeping the numeric tokens short. From these values the node also derives the braking distance, an estimate of the distance the vehicle would need to come to a stop from its current speed, computed as:

$$d_{\text{brake}} = \frac{v^2}{2 \cdot a_{\text{dec}}} \cdot k_{\text{safety}} \tag{1}$$

where $v$ is the ego speed, $a_{dec}$ a deceleration parameter, and $k_{safety}$ a safety margin (see Table 3 for equivalent script parameters).

$$d_{\text{brake}} = \frac{v^2}{2 \cdot g \cdot \mu} \tag{2}$$

Equation 2 presents the real physical kinematic relationship, where $v$ is the vehicle speed, $g$ is the gravity and $\mu$ is the friction coefficient. The actual equation used in the node, equation 1, is just a substitution of the deceleration parameter, adjusted to the acceleration limitation implemented and with the addition of a safety parameter that multiplies the value so the model has more margin to reason over it.

Note that Equation 1 is not a physically accurate stopping-distance model (Equation 2). The deceleration parameter is deliberately set equal to the control layer's speed ramp rate (`RAMP_RATE`), so that the estimate reflects the rate at which the vehicle is actually slowed down by the low-level controller rather than a real braking dynamic. This keeps the reference consistent with the system's own behavior. The safety margin (`SAFETY_MARGIN` = 1.5) multiplies the estimate so that the planner reacts slightly earlier than the bare value would dictate, absorbing the uncertainty in this simplified reference and giving the model additional room to reason over, at the cost of somewhat earlier braking. Replacing this equation with a proper physical model is left as future work (Section 9). The braking distance is computed in the node rather than left to the model, so that the VLM receives an already computed safety reference instead of having to perform the calculation itself. While this was not measured directly in this work, the smaller baseline is expected to handle such calculation less reliably, in line with its sensitivity to context noted above [28] [29].

The information about surrounding objects requires more processing, since the bridge reports every actor in the scene in absolute coordinates. The node first converts these absolute positions into distances relative to the ego vehicle, and discards the ego itself by removing any object within a small radius (`EGO_SELF_RADIUS`). The remaining objects are filtered by distance, keeping only those within `DISTANCE_THRESHOLD`, and then ordered by proximity, of which only the `MAX_OBJECTS` closest are retained (Figure 8). This last step bounds the size of the context and prevents distant, irrelevant traffic from saturating the prompt, in line with the selection criterion discussed above. For each retained object, the node also computes its relative angle with respect to the ego's orientation. The way this angle is turned into a named direction is described in Section 3.1.1.4, together with the rest of the geometric conventions.

The three parameters governing this stage were chosen to keep the object channel both relevant and compact. `EGO_SELF_RADIUS` (0.5 m) only needs to be large enough to capture the ego's own reported position and small enough not to discard a genuinely nearby actor, a condition half a metre comfortably satisfies. `DISTANCE_THRESHOLD` (50 m) sets the horizon beyond which an object no longer conditions a high-level decision at the low urban speeds used here, so that distant traffic is left out before it can add noise to the context. Finally, `MAX_OBJECTS` (5) bounds the number of retained actors, since every object adds tokens to the prompt and an oversized context degrades the reasoning of the baseline model in particular (Section 3.1.1.1), retaining only the five closest keeps the relevant surroundings represented while preventing the prompt from growing unnecessarily, given the scale of the scenarios. These values were adjusted during development, verifying that the retained elements were sufficient to represent the scenarios without saturating the prompt.

Traffic lights are processed analogously: they are filtered by distance, keeping those within `TL_THRESHOLD`, and orientation, so that only the lights actually governing the ego's lane are retained (Figure 8). The traffic-light threshold (`TL_THRESHOLD` = 80 m) is deliberately larger than the object threshold. A traffic light must be detected earlier than a generic object, since the vehicle needs enough distance to slow down and come to a stop for a red light, whereas an object only conditions the immediate action once it is close. The wider horizon ensures a relevant light enters the context in time for the planner to react to it. The value was tuned empirically. Both the orientation criterion and the discarding of lights that fall behind the ego are detailed in Section 3.1.1.4.

The discretization of angles into a small set of named directions intends to reduce token cost and

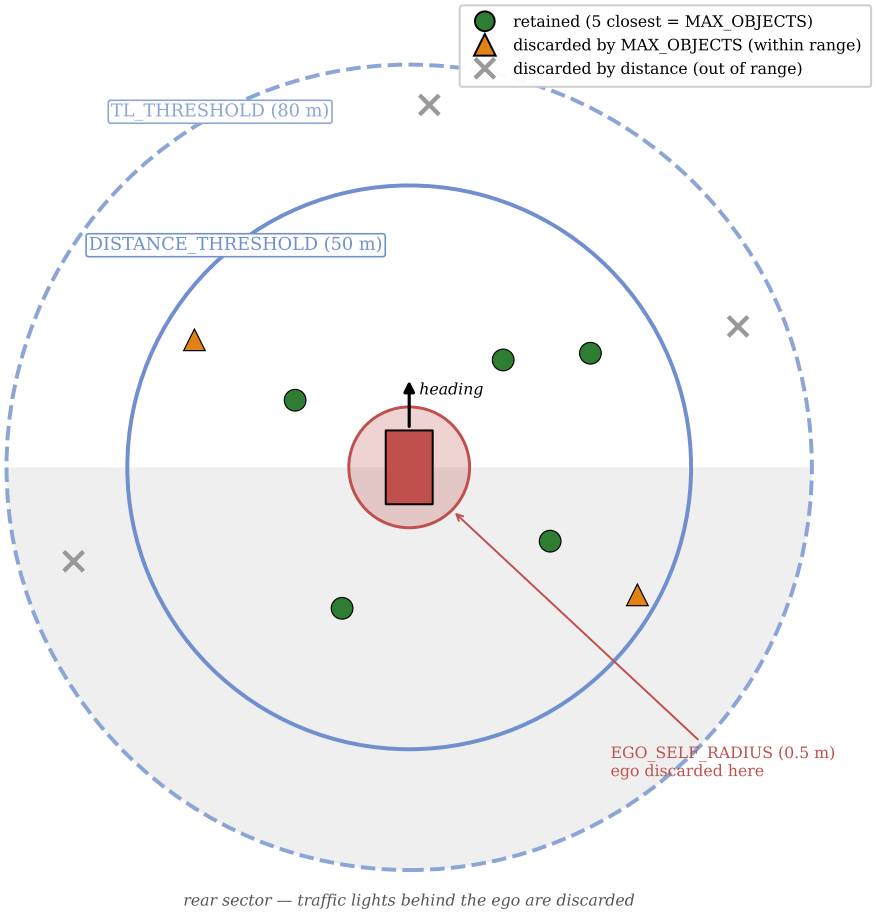


Figure 8: Schematic illustration of detection ranges around the ego vehicle. Objects reported by the sensors are expressed relative to the ego and filtered in two stages. Those within `DISTANCE_THRESHOLD` are kept and ordered by proximity, of which only the `MAX_OBJECTS` (five) closest are retained (green). Objects within range but beyond the five closest are discarded by the count limit (orange triangles), and objects beyond `DISTANCE_THRESHOLD` are discarded by distance (grey crosses). The object within `EGO_SELF_RADIUS` is removed as the ego itself. Traffic lights follow an analogous distance filter with `TL_THRESHOLD`. Those in the rear sector (shaded area) are additionally discarded, as a light behind the ego cannot govern its forward motion.

remove variation that would add noise to the model's context. This effect is especially relevant for the baseline model, which again, as noted earlier, is more sensitive to noisy or oversized contexts [28][29]. Orientations (*yaw*), both from the traffic lights and the ego vehicle, are provided through their respective channels inside quaternions and are obtained after being converted to Euler angles. Orientations exposed in degrees in the context to simplify the data for the VLM, but kept in radians internally for the geometrical calculations.

Table 3: Perception parameters used in the scene-context processing.

| Parameter | Value | Description |
|---|---|---|
| `DECEL` | 3.0 $m/s^2$ | Deceleration parameter used in the braking-distance estimate (set equal to `RAMP_RATE`) |
| `SAFETY_MARGIN` | 1.5 | Multiplicative safety factor in the braking-distance estimate |
| `EGO_SELF_RADIUS` | 0.5 m | Radius used to discard the ego vehicle from the object channel |
| `DISTANCE_THRESHOLD` | 50 m | Maximum distance for an object to be considered |
| `MAX_OBJECTS` | 5 | Maximum number of objects retained, ordered by proximity |
| `TL_THRESHOLD` | 80 m | Maximum distance for a traffic light to be considered |

#### 3.1.1.4 Geometrical conventions

All directional information in the scene context is expressed relative to the ego vehicle's orientation rather than in global coordinates, since the planner reasons from the vehicle's own point of view, without any map of global coordinates. The reference is therefore the ego's orientation, this means that an angle of 0°points straight ahead, and the sign of the angle encodes the side, with positive angles to the left of the ego and negative angles to the right.

For any surrounding object, the node first computes the absolute angle from the ego to the object with the four-quadrant arctangent of their relative position, and then subtracts the ego's yaw to obtain the relative angle, normalized to the interval $(-180, 180]$:

$$\theta_{\text{relative}} = \text{normalize}\left(\text{atan2}(p_y - e_y,\ p_x - e_x) - \psi_{\text{ego}}\right) \tag{3}$$

where $(p_x, p_y)$ is the object's position, and $(e_x, e_y)$ is the ego's position, and $\psi_{ego}$ is the ego's yaw.

Rather than passing this continuous angle to the model, the node maps it to one of eight named directions, each spanning a 45°sector centered on a cardinal or inter-cardinal angle relative to the ego. This discretization is the main abstraction step mentioned earlier, which replaces a precise but harder to interpret value such as -43.2°with a high-level concept such as `FRONT-RIGHT`, which is more direct and actionable for the planner and consumes fewer tokens. The eight sectors are defined in Table 4 and illustrated in Figure 9.

Table 4: Mapping from the relative angle $\theta_{\text{rel}}$ to the eight named directions. Positive angles lie to the left of the ego.

| Direction | Angular range (degrees) |
|---|---|
| FRONT | $(-22.5, 22.5]$ |
| FRONT-LEFT | $(22.5, 67.5]$ |
| LEFT | $(67.5, 112.5]$ |
| BEHIND-LEFT | $(112.5, 157.5]$ |
| BEHIND | $> 157.5$ or $\leq -157.5$ |
| BEHIND-RIGHT | $(-157.5, -112.5]$ |
| RIGHT | $(-112.5, -67.5]$ |
| FRONT-RIGHT | $(-67.5, -22.5]$ |

The same relative angle convention is used to decide which traffic lights are relevant to the ego. Two geometric criteria are applied. First, a traffic light whose direction falls in the rear sectors (any direction containing the string `BEHIND`) is discarded, as a light behind the vehicle cannot govern its forward motion and introduces noise inside the model's context.

Second, among the remaining lights, only those actually facing the ego's lane are kept. This is determined not from the light's position but from its orientation. A traffic light governs the ego's lane when its orientation is approximately anti-parallel to the ego's, that is, when the two face each other. Formally, the difference between the traffic light's yaw and the ego's yaw is compared against 180°, and the light is retained only if this difference lies within a tolerance (`TL_FACING_TOLERANCE`) of being exactly opposite:

$$\left| \, |\psi_{\text{tl}} - \psi_{\text{ego}}|_{\text{norm}} - 180^\circ \, \right| \leq \texttt{TL_FACING_TOLERANCE} \tag{4}$$

This criterion discards the traffic lights that regulate the transverse traffic at an intersection, which often fall within the ego's frontal cone by position but face other directions, and keeps only the light that applies to the ego's own lane. Both conventions, the sign of the relative angle and the anti-parallel orientation test, were verified empirically in simulation rather than assumed, as described in Section 3.1.3.5.

#### 3.1.1.5 Scene context construction

The processed magnitudes are assembled, within the `scene_context_node`, into the scene context. A single XML string is built by concatenating the corresponding tags, which is then wrapped in a `std_msgs/String` message and published. The XML format is a choice driven by structural clarity as, unlike unformatted text, XML delimits every field and group with explicit opening and closing tags, which removes any ambiguity about where each piece of data begins and ends and lets the model cleanly separate the ego state from the surrounding objects, and one object from the next. This kind of explicit, self-describing delimitation is in fact the structure recommended by Anthropic for prompting the Claude family of models [31], on which the cloud configuration of this work relies.

Other formats were considered. HTML was discarded as too verbose, spending a large number of tokens on markup while adding little structure that the simpler XML tags do not require.

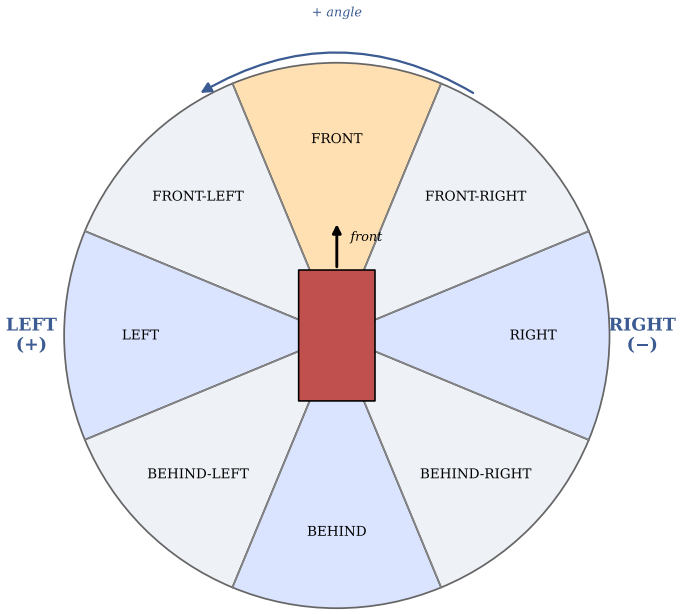


Figure 9: The eight named directions used in the scene context, shown as $45°$ sectors around the ego vehicle (top view). `FRONT` is centred on the ego's orientation $(0°)$, positive angles lie to the left and negative angles to the right, following the convention of Table 4.

JSON was also evaluated and is, for this purpose, highly comparable to XML [32]. XML was ultimately preferred because its open and closing tags give the least ambiguous separation between fields, at the cost of token efficiency. This is, however, negligible for a context of this size. Markdown is the most token-efficient of the formats considered, and is instead used for the behavior specification of the prompt ( 3.1.2), where the content is instructional prose. The environment data, being strongly hierarchical, is better served by XML, or JSON alternatively.

The context is organized into two top-level blocks that mirror the ego/surroundings distinction followed throughout the module. The `<self_data>` block holds the ego vehicle's own state, including speed, braking distance, yaw, and the acceleration and angular-velocity components, while the `<surroundings_data>` block holds the environment, grouped into `<traffic_lights>` and `<nearby_objects>`. Each `<object>` carries its identifier, class, distance, speed, and direction, together with the relative angle from which that direction is derived. Each `<traffic_light>` carries its identifier, state, distance, and direction. The `<traffic_lights>` block additionally exposes a `last_updated` attribute, which reports how many seconds ago the latched traffic-light data was refreshed (Section 3.1.1.3), or `unknown` if it has not yet been received. This makes that information explicit to the model rather than hiding it, so it can reason accordingly.

The `class` and `state` fields are not emitted as the raw numeric values provided by the bridge but mapped to their respective human-readable labels, again to keep the context interpretable. Table 5 gives the mapping for object classes, derived from the `derived_object_msgs/Object` message, and Table 6 the mapping for traffic-light states, derived from the `carla_msgs/CarlaTrafficLightStatus` message. An example of a complete scene context, taken from an actual run,

is shown in Listing 2. The resulting XML string is published at 1 Hz once per cycle on the `/scene_context` topic, where it is consumed by the decision-making module described in the following section.

The scene context is published at 1 Hz to match the planner's processing speed. Because the planner takes several seconds to make a decision, faster updates would waste computational resources on ignored data, while slower updates would provide stale information. This specific rate ensures the data remains fresh without overloading the system.

Table 5: Mapping from the object classification codes of `derived_object_msgs/Object` to the labels used in the scene context.

| Value | Label | Value | Label |
|---|---|---|---|
| 0 | UNKNOWN | 6 | CAR |
| 1 | UNKNOWN_SMALL | 7 | TRUCK |
| 2 | UNKNOWN_MEDIUM | 8 | MOTORCYCLE |
| 3 | UNKNOWN_BIG | 9 | OTHER_VEHICLE |
| 4 | PEDESTRIAN | 10 | BARRIER |
| 5 | BIKE | 11 | SIGN |

Table 6: Mapping from the traffic-light state codes of `carla_msgs/CarlaTrafficLightStatus` to the labels used in the scene context.

| Value | Label |
|---|---|
| 0 | RED |
| 1 | YELLOW |
| 2 | GREEN |
| 3 | OFF |
| 4 | UNKNOWN |

```
<self_data>
  <speed>5.33</speed>
  <braking_distance>7.1</braking_distance>
  <yaw>89.29</yaw>
  <acceleration>
    <x>-0.197</x>
    <y>-1.630</y>
  </acceleration>
  <angular_velocity>
    <z>-0.051</z>
  </angular_velocity>
</self_data>
<surroundings_data>
  <traffic_lights last_updated="24.5s">
    <traffic_light>
      <id>46</id>
      <state>RED</state>
      <distance>76.2</distance>
      <direction>RIGHT</direction>
    </traffic_light>
  </traffic_lights>
  <nearby_objects>
    <object>
      <id>74</id>
      <class>CAR</class>
      <distance>14.1</distance>
      <speed>7.15</speed>
      <direction>BEHIND-LEFT</direction>
      <relative_angle>150.2</relative_angle>
    </object>
    <object>
      <id>65</id>
      <class>CAR</class>
      <distance>33.2</distance>
      <speed>6.02</speed>
      <direction>FRONT-RIGHT</direction>
      <relative_angle>-53.8</relative_angle>
    </object>
  </nearby_objects>
</surroundings_data>
```

Listing 2: Example of a complete scene context produced in one cycle of an actual run (Traffic run, Claude Haiku 4.5)

### 3.1.2 Decision-making module

The decision-making module is the core of the framework, where the VLM acts as the high-level action planner. It receives the scene context produced by the perception module and the frontal RGB camera image, queries the model, and turns its response into a discrete driving action that is passed on to the low-level control module. Its logic is governed by the node `prompt_manager.py`.

This is the only module in which the probabilistic component of the system operates. As established in Section 3.1, this is precisely why it is decoupled, confining the VLM to a module, between a deterministic perception front-end and a deterministic control back-end, allows its output to be validated before reaching the vehicle. Therefore, the module is responsible not

only for querying the model, but also for assembling the prompt that guides its behavior and for checking that the returned action complies with the expected format before publishing it.

#### 3.1.2.1 Node structure

The node subscribes to two topics, the scene context on `/scene_context` and the front image on `/carla/hero/rgb_front/image`, and publishes the validated model output on `/llm/output`, which the control module consumes. The image is handled by a dedicated callback that resizes each frame to $320 \times 180$ , compresses it to JPEG, and stores it as a base64 string, so that an image is always available to send when a new context arrives. The reasons for this resolution were discussed in Section 3.1.1.1.

Inference is triggered by the arrival of a new scene context. Each time `scene_context_callback` receives a context, and provided a valid image is available, it sends a call to the model. This ties the decision rate to the perception rate, making it, in principle, one inference per second (1 Hz). In practice, however, a single inference takes several seconds, around 4 to 5 seconds, so contexts arrive faster than they can be processed. Handling this mismatch wrongly can either block the node or pile up a backlog of stuck calls.

The module avoids both with two mechanisms. First, a single processing slot is locked and claimed all at once, ensuring that two calls can not grab it at the exact same time. If an inference is already in process, the incoming context is skipped, so the model is never asked to process a queued context and always reasons over a recent one. Second, because this inference call blocks the thread that runs it, the node is spun by a `MultiThreadedExecutor` with a reentrant callback group, which lets the context and image callbacks keep updating the node's state on other threads while an inference is running. The image and the latest context therefore remain current even during a blocking call.

A direct consequence of skipping contexts is that the system's effective decision rate is not 1 Hz but is limited by the inference latency. In practice, the module completes roughly one decision every few seconds, discarding the contexts that arrive in between. This is a deliberate trade-off that implies stability over raw frequency, and its effect on the number of decisions per scenario is tackled in the results (Section 4).

At startup, the node also issues a "warmup call" to the model with a trivial input, so that the cost of loading and initializing the model is paid once before the first real context arrives rather than penalizing the first decision (Section 3.1.2.6).

#### 3.1.2.2 Prompt construction

On each inference, the node assembles a prompt from three pieces: the behavior specification, the scene context, and the frontal camera image. The behavior specification is sent as the `system` message, while the scene context and the image are sent together as the `user` message, closed with a short instruction that instructs the model to respond (*"Respond with the JSON object now:"*). The scene context is the XML string produced by the perception module (Section 3.1.1.5), and the image is the frame encoded in base64 format, prepared by the image callback.

The behavior specification, referred to as the behavior prompt, is the document that defines how the model must act. It is written in Markdown format, the most token-efficient of the formats

considered (Section 3.1.1.5) and a natural fit for instructional prose, and is organized into the following sections: the model's role and operating context, the set of available actions, an explanation of how those actions are executed by the control module, the format of the input data, the meaning of `target_speed`, the step-by-step reasoning process, the format of the expected output, a set of safety guardrails, and a partial example of the XML input and expected output formats.

Two of these sections are particularly emphasized. First, the behavior prompt explicitly describes how the control module will execute the model's actions: that maneuvers are locked for a minimum time once started, that STOP and SLOW_DOWN override any ongoing maneuver, and that repeating an action continues it. The model is told this so that its reasoning is aligned with the action dispatcher's mechanisms rather than working against them. For instance, it is instructed not to fight an ongoing turn, since a conflicting action would be ignored until the commitment lock expires. Second, a dedicated section clarifies the meaning of `target_speed`. Target speed is the speed to reach, not the current one, a point introduced specifically because it was a recurring source of error during development, the baseline model would hallucinate and use the field wrongly, sometimes repeating the same speed received from the speedometer. These two sections illustrate the general approach, the prompt was shaped against failures observed in practice, not written once in the abstract.

The behavior prompt was refined iteratively by inspecting the model's outputs on a set of representative scenarios and adjusting the instructions accordingly. This refinement was carried out using cloud models, whose stronger instruction-following made the effect of each change easier to observe before fixing the prompt for the full study.

**Chain-of-Thought reasoning.** The reasoning process section imposes a structured, step-by-step procedure that the model must follow before committing to an action. This is an instance of Chain-of-Thought (CoT) prompting, the technique of eliciting a series of intermediate reasoning steps, which has been shown to significantly improve performance on complex reasoning tasks [33]. Rather than letting the model jump directly to a driving action, the prompt walks it through seven steps, shown in Listing 3: it first assesses the ego state, then the traffic lights, the surrounding objects, and the camera image, and only then decides the action, sets its target speed, and reviews the decision. The first three output fields of the response, `scene_analysis`, `risk_assessment`, and `action_rationale` (Section 3.1.2.3), are where this reasoning is recorded, so the chain of thought is not discarded but exposed as part of the structured output, which also makes each decision traceable, aiding with the debug and evaluation tasks.

It is worth noting that many recent models, particularly frontier cloud models, already perform this kind of intermediate reasoning internally. Forcing it explicitly through the prompt is nonetheless deliberate, it standardizes the reasoning procedure across models of very different capability, including the small baseline, which does not reason reliably on its own, and returns more stable and safer outputs by ensuring that the relevant checks are always carried out in the same order before an action is produced.

```
1 Step 1: Analyze the ego vehicle state (speed, acceleration, whether it is turning).
2 Step 2: Analyze the traffic lights (presence, state, and how recent the data is).
3 Step 3: Analyze the surrounding objects (class, distance, speed, collision risk).
4 Step 4: Analyze the camera image (confirm the sensor data, spot anything not sensed).
5 Step 5: Decide the action (default to FOLLOW_LANE; respect ongoing maneuvers).
```

```
Step 6: Determine target_speed (apply the speed guardrails).
Step 7: Evaluate the decision (review the reasoning and assign a certainty level).
```

Listing 3: The seven-step reasoning procedure imposed on the model by the behavior prompt (condensed version, see Appendix **??** for the full text).

#### 3.1.2.3 Output schema and validation

The model is required to answer with a single JSON object and nothing else. JSON is used for the output, rather than the XML of the input, because the response must be parsed by the node. This strict, machine-readable structure format maps directly onto the fields the low-level control module expects, whereas the input prioritized human and model-readable delimitation (Section 3.1.1.5) [32]. The object must contain five required fields and one optional field:

- `scene_analysis`, `risk_assessment`, and `action_rationale`: the textual fields that record the model's reasoning, as introduced in Section 3.1.2.2.
- `action`: exactly one of the eight valid actions.
- `certainty`: the model's confidence in its decision, one of `LOW`, `MEDIUM`, or `HIGH`.
- `target_speed` (optional): the speed to reach, required for the driving actions (`FOLLOW_LANE`, `SPEED_UP`, `SLOW_DOWN`) and omitted for the rest.

An example of a complete, valid response, produced by Claude Haiku 4.5 during the traffic scenario, is shown in Listing 4.

```
{
  "scene_analysis": "Vehicle traveling at 4.2 m/s on a straight road (yaw ~0 deg, angular
    velocity near zero) with a RED traffic light 13.6m ahead and a car 12.4m in front at 4.53
    m/s. A second car is 20m ahead at 7.08 m/s, and another stationary car is 39.7m
    ahead-left.",
  "risk_assessment": "High risk: RED traffic light at 13.6m is within 3x braking_distance
    (13.2m); leading car at 12.4m is very close and approaching the red light. Braking is
    immediately required to avoid collision and traffic violation.",
  "action_rationale": "SLOW_DOWN is mandatory per guardrail 1 (distance 13.6m is between 1x
    braking_distance and 2x braking_distance, requiring SLOW_DOWN with target_speed <= 2.5
    m/s). The leading vehicle and red light both demand immediate speed reduction.",
  "action": "SLOW_DOWN",
  "target_speed": 2.5,
  "certainty": "HIGH"
}
```

Listing 4: Example of a complete, valid model response (Traffic run, Claude Haiku 4.5). The output example does not correspond with the input example in previous sections.

Because the response comes from a probabilistic model, it cannot be trusted to be well-formed, so the node validates it before publishing rather than forwarding it directly to the control module. The validation proceeds as a sequence of checks. The raw response is first cleaned, removing the `<think>` tags around its reasoning, which the baseline may still emit despite its thinking mode being disabled (Section 3.1.2.6), removed here as a safeguard. Any Markdown code delimitation that the models occasionally wrap around the JSON is also cleaned. The cleaned text

is then parsed as JSON, and if it fails, or if the result is not a JSON object, the response is rejected. Next, the node checks that all expected fields are present and that the `action` belongs to the set of eight valid actions. The valid actions are checked using an action whitelist that rejects any invented or misspelled action the model might produce. The `target_speed`, if present, is sanitized, clipping numeric values to a valid range and discarding non-numeric ones. Finally, a certainty guardrail is applied by overriding the action with `SLOW_DOWN` whenever the model reports `LOW` certainty, so that low-confidence decisions resolve to a cautious behavior.

Whenever any of these checks fails, the node does not publish the model's response but a fallback. This is a fixed, safe response with action `SLOW_DOWN` and `LOW` certainty, annotated with the reason for the rejection. This guarantees that the control module always receives a valid, safe action, even when the model returns malformed, incomplete, or unparseable output. Since the fallback is emitted here, in the decision module, the malformed outputs of the weaker model surface as fallbacks rather than as errors in the console. The rate at which this occurs is one of the metrics reported in the results (Section 4). Only when all checks pass is the JSON output published on `/llm/output` for the control module to consume.

#### 3.1.2.4 Providers and API connection

The node is not tied to a single model or vendor. The provider is selected before initialization through an environment variable, and three are supported: a local one, `ollama`, and two cloud ones, `openai` and `openrouter`. Only the way the model is queried depends on this choice, the rest of the node, including prompt construction, validation, and publication, is identical across providers. This is what makes the framework model-agnostic in practice (Hypothesis 2). A different model, served locally or in the cloud, can be swapped in without modifying the architecture, simply by changing the configuration.

Two connection paths are used. The local provider runs the model on the same machine as the framework, where the model weights are downloaded locally and served by Ollama. Ollama uses a runtime application that exposes a local HTTP server. The node reaches it through an HTTP request to that local endpoint, so although the communication uses HTTP, the request and inference remain local. The two cloud providers, by contrast, are accessed over the internet through the official OpenAI client library, with the model running on the provider's servers. This is possible for OpenRouter, which is not OpenAI, because it uses an OpenAI-compatible API and the same client works against it by pointing its base URL to the OpenRouter endpoint. This compatibility is what allowed Claude Haiku 4.5, an Anthropic model, to be integrated through the very same code path as the OpenAI models, with no change beyond the configuration. The reference model of the study was therefore reached with essentially no additional integration effort.

The two paths differ only in specific details concerning the API, the most notable one is about how the image is attached to the request. The local endpoint receives the raw base64 image in a dedicated field, whereas the OpenAI client expects it embedded as a data URL. The cloud path also appends a short user note to the prompt clarifying that the images are simulation renders, for reasons discussed in Section 3.1.2.5. Beyond the model client itself, the node relies on a small set of supporting libraries, including `cv_bridge` and OpenCV to convert and compress the camera image, and a standard environment-loading utility to read the provider and model configuration. The model and provider are read from environment variables and can be overridden through the launch arguments (Section 3.2).

#### 3.1.2.5 Model selection and the vision refusal problem

The study compares two models that act as opposed representative points along capability axis (Section 1.3), a baseline model and a upper reference limit. The baseline is one of the most capable models that is able to run locally on the given available hardware, while the reference is a near-frontier reference model meant to bound the upper end of the capability range. However, reaching the final pair of models was not immediate.

The reference model was initially intended to be GPT-4o. In practice, it proved unusable for this task because, when given the frontal camera image, the model frequently returned a refusal of the form *"I'm sorry, I can't help with that request."* instead of a driving decision. This is consistent with a documented design behavior of GPT-4's vision system, whose safety filters can refuse images on privacy grounds [34]. The exact trigger in this setting was not formally determined as the scenes used in these trials contained no pedestrians, so the refusals cannot be attributed to the presence of people. They appear instead to be rooted in the apparent realism of the driving scene itself, an interpretation supported by the mitigation described below. Critically for a system that must decide on every cycle, the refusals were stochastic, as the same kind of input would sometimes be answered and sometimes refused. Approximately 83% of calls were refused, measured over a set of two trial runs comprising 46 calls.

This motivated the simulation note that the cloud path appends to the prompt (Section 3.1.2.4), which clarifies that the images are CARLA simulation renders and do not depict a real scene. Reformulating the prompt in this way reduced the refusal rate substantially, to approximately 27% over the same trials, but did not eliminate it. That the refusals dropped once the scene was framed as simulated is what suggests the filter was reacting to the apparent realism of the images rather than to any specific content within them. A 27% refusal rate, nonetheless, remains incompatible with a planner that must produce an action at every cycle, so GPT-4o was discarded as the reference model.

The reference ceiling was therefore set to Claude Haiku 4.5, accessed through OpenRouter (Section 3.1.2.4). Over the entire evaluation, it produced no vision refusals. The simulation note was kept on the cloud path, so Haiku also received it. This is a deliberate but asymmetric choice, since the local path does not carry that note, and the resulting prompt asymmetry between the two models is acknowledged as a limitation of the study (Section 5.1).

#### 3.1.2.6 Inference and latency

A few parameters govern each inference call. The maximum number of generated tokens is set to (`num_predict` = 2000, so that the full Chain-of-Thought reasoning and the JSON object fit with margin. The local call is limited by a 60 s timeout, a safety limit that prevents a request from blocking the node indefinitely. For the baseline, the model's internal "thinking" mode is explicitly disabled, because leaving it active inflated the latency dramatically on some cycles. This is not a loss, since the structured reasoning is already imposed externally through the Chain-of-Thought prompt (Section 3.1.2.2), making the model's own reasoning step redundant here. The remaining sampling parameters, such as temperature, are left at each provider's defaults. Therefore, the models sample their output rather than decoding deterministically, so identical inputs may return different responses, a source of variability discussed among the study's limitations (Section 5.1).

As introduced in Section 3.1.2.1, the node also issues a "warmup call" at initialization, which

pays the cost of loading and initializing the model so that the first real decision is not penalized by it.

The latency reported in this work is the elapsed time between consecutive calls, a proxy that combines the inference time with the node's loop period, rather than a pure measurement of model inference. This distinction is noted as a limitation (Section 5.1). Both models complete a decision in a few seconds, with the baseline being slightly slower than the reference.

This fact can be counterintuitive at a first look. The cloud model, queried over the internet, was faster than the local one running on the same machine. The reason is competition for hardware resources. Running the baseline locally makes it share the machine's GPU with the simulator and other processes, and this competition for resources slows its inference. Alternatively, offloading inference to the cloud frees the GPU entirely for CARLA. In return, the cloud path is occasionally delayed by the network. The control layer's watchdog absorbs these sporadic stale-output events (Section 3.1.3). This trade-off between local contention and network latency is taken up again in the discussion (Section 5).

### 3.1.3 Low-level control module

The low-level control module is the final stage of the pipeline. It receives the high-level action chosen by the decision-making module and translates it into concrete vehicle behavior, while enforcing the safety and continuity mechanisms that protect the vehicle from unsafe commands. Its logic is governed by the node `action_dispatcher.py`. While the decision-making module reasons about what to do, this module is in charge of how to carry it out, converting a discrete, abstracted action such as `TURN_LEFT` or `SLOW_DOWN` into continuous actuation of the vehicle. This conversion is not implemented from scratch but delegated to CARLA's native Traffic Manager, which the module drives through its pre-configured functions. The module is therefore the actuator that closes the loop, executing the planner's decisions and acting as a safety layer against the model's errors.

#### 3.1.3.1 Node structure

The node subscribes to two topics, the model output on `/llm/output`, which carries the action to execute, and the scene context on `/scene_context`, from which it extracts the distance to the closest frontal red traffic light and the closest frontal object. The module needs these two distances to decide when a STOP command is genuinely required and when it can be safely relaxed (Section 3.1.3.4). It also publishes the action it commits to on `/action_committed`. Unlike the previous nodes, the dispatcher does not operate at a single rate. It combines event callbacks with two periodic timers, each serving a different purpose:

- An **action callback**, triggered every time a new action arrives on `/llm/output` (that is, at the model's inference rate, once every few seconds). It validates the action, applies the safety overrides, and dispatches it to the Traffic Manager.
- A **speed ramp** timer running at 10 Hz, which continuously moves the vehicle's speed toward the commanded target. This high-frequency loop is what keeps the motion smooth between the sparse decisions of the planner: while the model decides only every few seconds, the speed is adjusted every 100 ms (Figure 10). The ramp rate's selected value (`RAMP_RATE` $= 3\,\mathrm{m/s^2}$) keeps longitudinal acceleration and deceleration within a comfortable, non-abrupt range, smoothing the transitions between the planner's sparse decisions

while remaining quick enough to react to the situations it faces. It has been tuned empirically.

- A **stale-output check** timer running at 1 Hz, which monitors how long it has been since the last model output and forces a safe action if the planner does not output a valid action (Section 3.1.3.4).

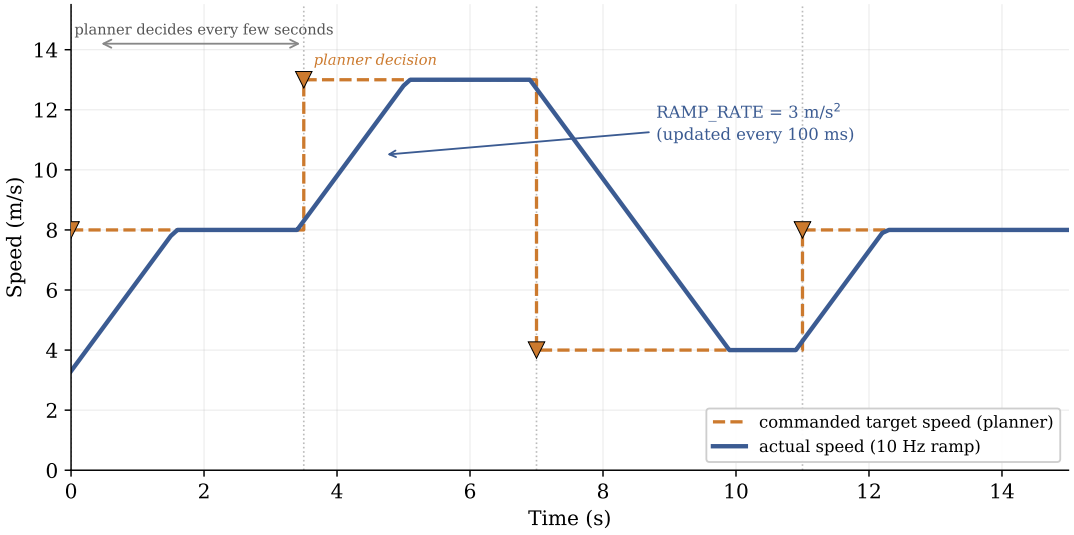


Figure 10: Operation of the speed ramp. The planner issues a new target speed at intermittent decision instants (Inference time and latency are not exactly fixed) (orange markers), producing the step commanded target. The 10 Hz ramp moves the actual speed toward that target gradually, at a maximum rate of `RAMP_RATE` $= 3\ \mathrm{m/s^2}$ and updated every 100 ms, so that the vehicle's motion stays smooth between the infrequent decisions of the planner rather than jumping abruptly at each new command.

This separation matters, the planner's decisions are infrequent and may be delayed, but the vehicle must be actuated continuously and safely in the meantime. The 10 Hz ramp and the 1 Hz watchdog decouple the continuous and critical actuation from the slow or irregular arrival of decisions. There is a final detail regarding initialization robustness. The ego vehicle is spawned in the map, and the dispatcher might be started before it exists. Instead of a single check that would fail, the node queries the simulation until the ego appears, which makes the launch order of the various nodes and CARLA scripts irrelevant.

#### 3.1.3.2 The Traffic Manager and low-level control

The Traffic Manager (TM) is a built-in module in CARLA simulator for driving vehicles autonomously in traffic. In its normal use, it acts as a complete autonomous driver keeping vehicles in their lane, regulating their speed, managing distances to other vehicles, obeying traffic lights, and deciding lane changes, all through an internal controller that produces the low-level steering, throttle, and brake signals. This controller is built on a PID scheme —a classic feedback controller that adjusts the actuation in proportion to the error between the desired and the actual state of the vehicle (for instance, the difference between the target and current speed)— continuously correcting the vehicle's motion toward the commanded targets.

The framework deliberately uses only part of this capability. Rather than letting the TM drive

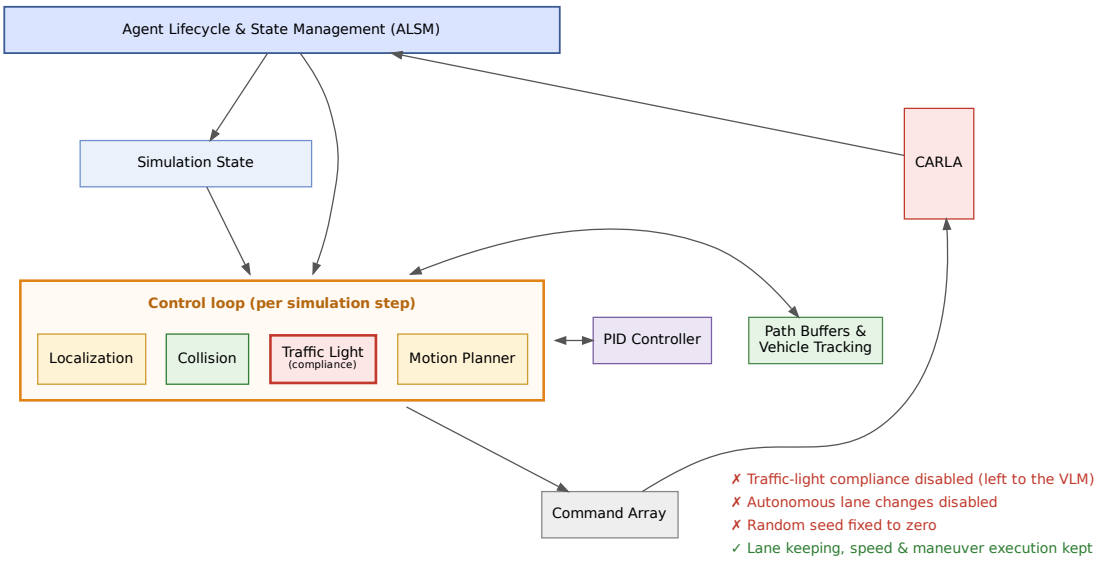


Figure 11: Architecture of CARLA's Traffic Manager and the parts the framework disables. The Traffic Manager runs an iterative control loop (localization, collision, traffic-light, and motion-planning stages) on top of a PID controller, producing the low-level command array sent to the simulator. The framework keeps its low-level execution (lane keeping, commanded-speed following, discrete maneuvers, and collision detection) but disables its decision autonomy. The traffic-light compliance is left to the VLM, autonomous lane changes are turned off, and the random seed is fixed to zero for reproducibility. Adapted from CARLA's Traffic Manager documentation.

the vehicle on its own, it removes the decision-making autonomy of the TM, only keeping its low-level execution. Specifically, when the TM is activated for the ego vehicle, three of its autonomous behaviors are disabled, while its low-level collision braking is deliberately kept. These include its compliance with traffic lights, which is switched off, so that the decision of whether to stop at a red light is left to the VLM rather than the TM; its autonomous lane changes are also disabled, so that lane changes happen only when the planner commands them; and finally, its random-decision seed is fixed, set to zero for reproducibility. What remains active is the TM's ability to keep the lane, follow a commanded speed, and execute the discrete maneuvers it is told to perform. Another low-level behavior is deliberately left active, the TM's collision avoidance. Its collision stage (Figure 11), which brakes the vehicle when an obstacle is too close ahead, is not disabled. This is a deliberate design choice, as the VLM is a high-level planner that reasons over a mid-term horizon and at a slow, latency-bound rate, so it is not suited to react to immediate, close-range hazards. Keeping the TM's collision braking provides a fast, deterministic low-level safety response that complements the planner, handling the immediate situations the planner cannot react to in time, and acting as a last-resort safeguard when the planner's command is late or wrong. This is consistent with the role of the control layer as a reliable substrate around the probabilistic planner, and it is what brakes the vehicle in the pedestrian scenario before the model's own `STOP` takes effect (Section 4.2.4). Its threshold, however, is larger than the framework's own, so the two do not always trigger at the same distance. In this way, the TM is not used as the autonomous driver of the vehicle but as a low-level actuator that turns the high-level actions chosen by the planner into concrete vehicle motion. The "intelligence" of the system resides in the VLM, the TM only carries out its decisions. This division lets the framework keep its low-level control deterministic and verifiable while the high-level reasoning remains probabilistic, consistent with the hierarchical separation mentioned in Section 3.1.

The TM's autonomy, however, cannot be fully suppressed. The TM is not goal-oriented but follows an endless, dynamically generated route, and at each junction a direction must be chosen [35]. When the planner commands a turn, that instruction is written into the vehicle's route through `set_route` (Section 3.1.3.3), and the TM executes it as commanded. This is the normal case, and turns issued by the planner are carried out correctly. Only when the route is empty, when the planner has not commanded any turn, does the TM fall back to its default behavior and choose the direction at the junction on its own. This residual behavior explains a few turns observed in some runs that the planner had not commanded. At those intersections the route happened to be empty, so the TM resolved the junction by itself. Fixing the TM's random seed to zero makes these fallback choices deterministic and repeatable across runs rather than random. The turns commanded by the planner are unaffected and execute normally, only when the planner has not issued an action upon a junction then the TM guides the decision. This is a consequence of the framework's reactive, routeless design and is discussed among its limitations (Section 5.1).

**3.1.3.3 Action translation** Each high-level action is translated into: a target speed, which every action sets, and a discrete maneuver, which only some actions issue. The two are handled by different mechanisms. The target speed is applied gradually by the speed-ramp loop, whereas a maneuver is a single command sent to the Traffic Manager at the moment the action changes.

Every action carries a default target speed, shown in Table 7. When the model provides a `target_speed`, that value is used instead, if it is valid for the action family. Otherwise the default is applied. The validation of this field is detailed in Section 3.1.3.4. Importantly, the dispatcher does not actuate the throttle directly but sets a desired speed that is moved gradually toward

the commanded target by the ramp (Section 3.1.3.1), and the Traffic Manager's own PID controller is what drives the vehicle to reach it. Therefore, the ramp refines the target that the TM is instructed to track, while the TM itself performs the actual longitudinal control. For `STOP`, the target speed is simply set to zero, and the vehicle is brought to a halt by the same ramp mechanism rather than by an abrupt brake.

The maneuvering actions are mapped onto the Traffic Manager's route and lane-change functions. A `TURN_LEFT` or `TURN_RIGHT` writes the corresponding instruction into the vehicle's route, which the TM then executes at the next junction. A `CHANGE_LANE_LEFT` or `CHANGE_LANE_RIGHT` forces a lane change to the corresponding side. An important detail is the sign convention of the lane-change function. In CARLA, the boolean argument is `True` for a change to the right and `False` for a change to the left. This convention was verified empirically in simulation (Section 3.1.3.5) to ensure the vehicle moves to the intended side.

These discrete commands are issued only when the action changes, not on every cycle. Since the planner may repeat the same action across several inference cycles while a maneuver is still in progress, re-sending the command each time would re-queue a turn or re-trigger a lane change that is already under way. A guard compares the new action against the last one dispatched and issues the command only when the vehicle is transitioning, letting the maneuver be completed.

Finally, `FOLLOW_LANE` clears the vehicle's route, removing any pending turn instruction left by a previous, possibly interrupted, maneuver. This has a side effect worth noting, as described in Section 3.1.3.2, an empty route is precisely the condition under which the TM decides the next junction on its own. Moreover, the framework only ever writes `Left` or `Right` into the route, never `Straight`. It can command a turn or clear the route, but it cannot force the vehicle to go straight through an intersection. Both facts are consequences of the reactive, routeless design and are revisited among the limitations (Section 5.1).

Table 7: Translation of each high-level action into a default target speed and a Traffic Manager command. Actions without a discrete command affect only the target speed.

| **Action** | **Default speed** | **Traffic Manager command** |
|---|---|---|
| FOLLOW_LANE | 8.33 m/s (30 km/h) | clear route |
| SPEED_UP | 13.89 m/s (50 km/h) | — (speed only) |
| SLOW_DOWN | 4.17 m/s (15 km/h) | — (speed only) |
| STOP | 0.0 m/s | — (speed only) |
| TURN_LEFT | 5.56 m/s (20 km/h) | route ← `Left` |
| TURN_RIGHT | 5.56 m/s (20 km/h) | route ← `Right` |
| CHANGE_LANE_LEFT | 8.33 m/s (30 km/h) | force lane change (left) |
| CHANGE_LANE_RIGHT | 8.33 m/s (30 km/h) | force lane change (right) |

#### 3.1.3.4 The safety layer

The dispatcher is the last component between the planner and the vehicle, and it is where the framework's safety layer lives. Because the action it receives originates from a model that may be wrong, malformed, late, or unstable, the dispatcher does not execute it blindly but applies a set of safeguards that filter it before it reaches the Traffic Manager. They can be grouped by the kind of failure they protect against.

A first group guards against malformed or invalid output. Even though the decision module already validates the model's response, the dispatcher re-checks that the action belongs to the set of eight valid actions, rejecting anything else before it is dispatched. This whitelist may seem redundant, since in normal operation the decision module has already filtered invalid actions upstream and so this second check never fires. However, as this framework is built from decoupled modules, this mechanism guarantees that no unrecognized command can ever reach the vehicle even if the upstream validation was bypassed or modified. Along the same lines, the model's `target_speed` is sanitized before use. It is accepted only if it is numeric and consistent with the action family, a value in the range $5 < v \leq 14$ m/s for `FOLLOW_LANE` and `SPEED_UP`, or $0 \leq v \leq 5$ m/s for `SLOW_DOWN`. Any value outside its family's range, or any non-numeric value, is discarded and replaced by the action's default speed (Table 7). This prevents an out-of-range or non-numeric value from being applied. For instance, there was a recurring case of the baseline model copying near-zero speeds into `target_speed`, for a `FOLLOW_LANE` action, which would leave the vehicle unable to accelerate.

A second safeguard corrects an unsafe or premature decision. If the model outputs `STOP` in response to a red traffic light that is still far away, halting immediately would be both unnecessary and disruptive to the traffic behind. The dispatcher therefore overrides such a `STOP` into a `SLOW_DOWN`, but only under three strict conditions: a red light must actually be present ahead, it must be farther than the stopping threshold (`TL_STOP_THRESHOLD` = 20 m), *and* no frontal object may be closer than the object threshold (`OBJ_STOP_THRESHOLD` = 10 m). If a red light is within 20 m, or an object is within 10 m ahead, the `STOP` is respected. This keeps the vehicle from braking abruptly for a distant light while never relaxing a stop that could be needed. The two thresholds encode a simple safety range, a frontal object is a more immediate hazard than a distant red light, so the object threshold is tighter. A red light farther than 20 m allows a premature stop to be relaxed into a gradual slowdown without risk, whereas an object within 10 m ahead is close enough that the stop must always stand. Both values were tuned empirically.

A third safeguard, the action commitment, protects against temporal instability. A maneuver takes several seconds to complete, which is longer than the planner's decision cycle, so a model that changes its mind every cycle could interrupt a maneuver halfway or oscillate between actions. To prevent this, once an action is accepted the dispatcher locks it for a minimum duration, longer for turns and lane changes, shorter for ordinary driving (Table 8), and ignores any different action until that time elapses. These durations reflect the real time each maneuver takes to complete: ordinary driving actions are locked only briefly, enough to prevent oscillation between consecutive decisions without freezing the planner, while lane changes and especially turns are locked longer, since they physically occupy several seconds of driving and interrupting them halfway would leave the vehicle in an inconsistent state. There is one deliberate exception. `STOP` and `SLOW_DOWN` are treated as emergency actions and bypass the lock entirely, so that braking is never delayed by a commitment to a previous maneuver, which could cause an accident in an autonomous vehicle context.

A final group ensures continuity. The speed ramp, already described, prevents abrupt speed changes by moving the vehicle's speed gradually toward its target rather than in steps. In addition, a stale-output watchdog guards against a potential silence of the model. If no model output arrives for longer than a fixed timeout (`STALE_TIMEOUT`=8 s), the dispatcher forces a `SLOW_DOWN`, so that a crashed or excessively slow model results in cautious deceleration rather than the vehicle continuing blindly at its last commanded speed. This timeout is set to roughly twice the typical inference latency of 4-5 s (Section 3.1.2.6), so that ordinary inference delays

Table 8: Minimum commitment duration per action, during which a different non-emergency action is ignored.

| Action | Minimum duration (s) |
|---|---|
| FOLLOW_LANE, SPEED_UP, SLOW_DOWN | 2.5 |
| STOP | 4.0 |
| CHANGE_LANE_LEFT, CHANGE_LANE_RIGHT | 5.0 |
| TURN_LEFT, TURN_RIGHT | 8.0 |

and occasional network latency on the cloud path do not trigger it, while a genuine stall from a crashed or excessively slow model is caught before the vehicle continues for too long at its last commanded speed. The watchdog is what absorbs the occasional increased latency of the cloud path noted in Section 3.1.2.6.

Together, these safeguards form the safety layer that makes the framework viable with an unreliable planner. Hallucinated or malformed actions are rejected, premature stops are relaxed, unstable command sequences are damped, and silence or abrupt change is turned into safe, continuous behavior. Their effect is quantified in the results, where the frequency with which each one intervenes is reported per scenario (Section 4).

Table 9: Safety-layer parameters of the action dispatcher.

| Parameter | Value | Description |
|---|---|---|
| `RAMP_RATE` | 3.0 $m/s^2$ | Maximum speed change per second applied by the ramp |
| `STALE_TIMEOUT` | 8.0 s | Time without model output that triggers a forced SLOW_DOWN |
| `TL_STOP_THRESHOLD` | 20.0 m | A STOP for a red light farther than this is overridden to SLOW_DOWN |
| `OBJ_STOP_THRESHOLD` | 10.0 m | A frontal object within this distance always keeps the STOP |

#### 3.1.3.5 Validation

The framework was validated at three levels, from the logic of the individual decisions to the behavior of the complete system in simulation. Each level checks something the previous one cannot.

The first level targets the decision logic of the safety layer. The boolean conditions that govern the most critical safeguards, the `STOP` override, the default speeds filtering, the certainty normalization, the action whitelist, and the traffic-light orientation test, were captured in a set of checks that reproduce the exact expressions used in the nodes and compare their output against the expected result for a range of inputs, including limit cases such as a `STOP` with no traffic light and no object present. These checks run in isolation, without ROS or CARLA, and import the same shared geometry helpers used by the perception node, so that a divergence in that shared code would also be detected. Their purpose is to catch sign errors, boundary mistakes, and bugs in the logic. They do not validate the ROS communication between nodes or the correspondence between the code's assumptions and CARLA's actual conventions. Those require the simulator

and are addressed at the next two levels.

The second level is integration testing of the dispatcher with the rest of the system. Messages were injected manually into the `/llm/output` topic to observe how the dispatcher reacted to specific actions, confirming that the overrides, the whitelist, the speed conversion, and the commitment behaved as expected. Individual mechanisms were also validated in specific runs that isolated them. For instance, the stale-output watchdog was verified by running the dispatcher without the decision module connected, so that the absence of output would trigger the forced `SLOW_DOWN`. In addition, standard runs were used to observe the system's behavior end to end, particularly with the less stable baseline model, logging the outputs for later inspection.

The third level validates the geometric conventions against the simulator, since these cannot be confirmed by logic alone. Two conventions were checked by visual inspection in simulation, aided by running the perception node in debug mode with additional logging. The sign convention of the relative angle, that positive angles correspond to the left of the ego were confirmed by observing the reported position of the scenario's traffic lights. As for the matching direction of the lane-change command (Section 3.1.3.3), it was validated visually by observing how the vehicle executed the action. The anti-parallel orientation criterion for traffic lights (Section 3.1.1.4) was verified using the debug logs, which reported the position and orientation of the traffic lights that were discarded and retained, allowing a check that the lights selected in each scene were indeed the ones governing the ego's lane. Validation revealed that lane-change boolean convention was incorrect and therefore was properly modified.

## 3.2 Experimental setup

### 3.2.1 Models compared

Table 10: Models compared over the five scenarios.

| Role | Model | Access | Identifier |
|---|---|---|---|
| Baseline model | Qwen3.5:2B | Local (Ollama) | qwen3.5:2b |
| Upper reference limit | Claude Haiku 4.5 | Cloud (OpenRouter, OpenAI compatible API) | anthropic/claude-haiku-4.5 |

The model is selected via launch argument (llm_provider, llm_model, Section 3.3.5), which takes precedence over the .env, leaving the model recorded in the command of each run.

### 3.2.2 Reproducibility

Several measures were taken to make the runs reproducible. The guiding principle is that every stochastic element of the environment is seeded to zero, so that the same scenario reproduces the same conditions on each execution. The ego vehicle spawns at a fixed point defined in `objects.json` (Town10HD), using the vehicle `vehicle.dodge.charger_police_2020`. A single Traffic Manager (port 8000) governs the autopilot, with its random seed fixed by the `action_dispatcher`, while the simulation server runs on port 2000. The ambient traffic consists of 15 vehicles whose spawn points and blueprints are drawn from a seeded random generator, with the overall flow capped at 10% below the speed limit for calmer traffic. The scenario events are fired by distance-based triggers rather than time-based ones, so that each event occurs at the same physical point of the route regardless of the model's latency, keeping runs

comparable across models. The visual input is a single front camera, resized to $320 \times 180$ and JPEG-compressed at quality 75, and each inference is bounded by a 2000 token generation limit and a 60 s timeout on the local call, with an initial warmup call absorbing the first-cycle loading cost. It must be noted that, while the environment is deterministic, the planner is not. The models sample their outputs at their default temperature (Section 3.1.2.6), so two runs of the same scenario are not identical. The measures above fix the environment, not the model's responses. This is consistent with the single run per scenario performed in this study, and is acknowledged among its limitations (Section 5.1).

### 3.2.3 Scenarios

The two models were compared across five scenarios of increasing complexity, all sharing the same fixed spawn point and ego vehicle. Each scenario adds a new type of challenge on top of the previous one, increasing the complexity. Moving from an empty environment to dense traffic, then to a static obstacle, a dynamic agent crossing the ego's path, and finally an emergency vehicle sensing edge case. This staged design makes it possible to attribute differences in behavior to specific difficulties rather than to a single, undifferentiated test. The ambient traffic, the seeded actors, and the event triggers are all reproducible, as described above.

- *Free:* the ego drives along the route with no ambient traffic and no events, establishing a baseline of nominal driving behavior.
- *Traffic:* the same route populated with the 15 ambient vehicles, introducing the need to react to surrounding agents.
- *Obstacle:* on top of the traffic, a stopped vehicle (`vehicle.audi.tt`) is placed 40 m ahead in the ego's lane, requiring the planner to detect the blockage and decide between stopping, slowing, or changing lane.
- *Pedestrian:* on top of the traffic, a pedestrian waits at the edge of the road about 35 m ahead and begins to cross when the ego approaches within 18 m, walking across at 1.5 m/s. This tests the reaction to a dynamic agent suddenly entering the ego's path.
- *Ambulance:* on top of the traffic, an emergency vehicle approaches the ego *from behind*, spawned about 30 m to the rear, driving 40% faster than the ambient flow and ignoring traffic lights. Its only distinguishing signal is a set of visual beacons. With a frontal camera and its approach from the rear, the ambulance falls outside what the system can sense, making this scenario an evidence of a known limitation regarding the sensing (Section 5).

Each scenario was run once per model, with one exception: the obstacle scenario, which was run twice. The repetition was motivated by the interest in checking whether the models behaved consistently in the scenario with the most decisions available.

### 3.2.4 Visualization

The runs are monitored in real time with RViz, the standard ROS2 visualization tool. It is configured to display the three sensor streams most useful for following the vehicle's behavior: the front camera image, its semantic-segmentation counterpart, and the LiDAR point cloud. These information channels are specifically used for the visualization of the experiments performed but do not provide any kind of information to the ego vehicle. In addition, the committed action

published on `/action_committed` is shown, so that the action the vehicle is currently executing can be read directly alongside the sensor views. This setup makes it possible to observe, at a glance, what the vehicle perceives and how it is acting.

### 3.2.5 Captured data

Each run produces two files. A debug file (`.md`) records the model's raw response for every inference, before validation, paired with the scene context that produced it. A console log (`.log`) records the dispatcher's activity, including the actions it committed and every intervention of the safety layer (overrides, fallbacks, stale timeouts and commitment blocks).

### 3.2.6 Metrics

The captured data is processed into a set of metrics that characterize and compare the two models. They fall into two groups: metrics of output quality, derived from the raw model responses, and metrics of driving behavior, derived from the dispatcher's actions and the safety-layer events. The metrics of output quality quantify how reliably each model produced a usable decision:

- **Total calls**: the number of inferences performed over the run.
- **Valid-call rate**: the fraction of responses that passed the full validation chain (Section 3.1.2.3) and were published as actions.
- **Fallback rate**: the fraction of responses that failed validation either through a parsing error, a missing field, or an invalid action, and were replaced by a fallback `SLOW_DOWN`.
- **Hallucinated actions**: responses in which the `action` field contains a string outside the set of eight valid actions (for instance `REVERSE` or `BRAKE_AHEAD`). These are a subset of the fallbacks, reported separately because they reflect the model inventing non-existent actions rather than producing malformed text. They are rejected by the whitelist and never executed.
- **Backward confusion**: responses in which the model misinterprets the `yaw` field, an absolute orientation, as the vehicle moving in reverse (for example, describing the ego as "moving backward" when its yaw is near 180°). This is a reasoning error over the semantics of the data, not an invalid action, and it persists despite the behavior prompt explicitly clarifying that `yaw` denotes orientation, not motion. It mainly occurs with the baseline model.
- **target_speed issues**: responses in which the speed field is missing where required, non-numeric, or outside the valid range, and is therefore corrected by the sanitization step (Section 3.1.3.4).
- **Low-certainty rate**: the fraction of responses reporting `LOW` certainty, which are downgraded to `SLOW_DOWN` by the certainty guardrail.

The driving-behavior metrics describe what the vehicle was actually commanded to do:

- **Action distribution**: the frequency of each high-level action over the run, restricted to valid decisions.

- **Safety-layer events**: the number of times each mechanism intervened, including `STOP` overrides, commitment blocks, and stale-output timeouts. They are read from the dispatcher log.

- **Latency**: the elapsed time between consecutive inferences (Section 3.1.2.6), reported as both median and mean per scenario.

All metrics are extracted with `parse_runs.py`, which parses the specific run debug and log files and replicates the node's validation logic offline, so that the reported validity and fallback rates match the system's actual runtime behavior.

## 3.3 Platform and environment setup

The framework is built on top of two main pieces of software. CARLA simulator provides the driving environment and the ground-truth sensor data while ROS2 connects the components of the framework. Both are managed through reproducible environments, as described below.

### 3.3.1 Simulation: CARLA

CARLA is an open-source simulator developed specifically for autonomous-driving research, built on the Unreal Engine [36]. It provides photorealistic urban environments, a library of vehicles and pedestrians, configurable weather and lighting, and a list of sensors, including RGB cameras, LiDAR, radar, GNSS, and IMU, together with pseudo-sensors that expose ground-truth information such as object lists and traffic-light states. It is widely regarded as the open-source standard for high-fidelity driving simulation [37]. CARLA was chosen for several reasons. It offers the ground-truth pseudo-sensors that the perception module relies on (Section 3.1.1), which allow the planner to be evaluated in isolation from a real perception pipeline. Also, it includes the Traffic Manager used as the low-level controller (Section 3.1.3.2). Furthermore, it supports a synchronous mode that guarantees temporal alignment of the sensor data (Section 3.1.1.2). Finally, it exposes a Python API and bridge to communicate through ROS2. The main alternative considered was Gazebo, a general-purpose robotics simulator with strong ROS integration, but it was discarded because, being general-purpose, it lacks the driving assets, traffic infrastructure, and built-in traffic management that CARLA provides natively, all of which are central to this work. The simulation runs on the map Town10HD, a detailed urban map chosen because it concentrates a wide variety of road features relevant to the scenarios such as single- and multi-lane roads, intersections and junctions of different types, traffic lights, road signs, and a dense urban layout. This variety makes it possible to exercise the planner on diverse situations, including lane following, junction handling, traffic-light compliance, within a single, consistent environment.

### 3.3.2 Middleware: ROS2 and the CARLA bridge

ROS2 (Robot Operating System 2) is an open-source framework for building robotic software [38]. Its components, called nodes, communicate through a publish–subscribe model over named topics, on top of the Data Distribution Service (DDS) middleware, which provides a decentralized, peer-to-peer transport with configurable Quality-of-Service policies [38]. This architecture fits the proposed framework naturally, where each of the three modules is an independent node and the data flows between them (Section 3.1). The connection between ROS2 and CARLA is handled by the `carla-ros-bridge`, the officially supported package that exposes CARLA's sensors and actors as ROS2 topics and relays commands back to the simulator. It is the bridge that

Figure 12: Town10HD, the inner-city urban map used for all experiments. It concentrates a variety of road features relevant to the scenarios, including single- and multi-lane roads, intersections and junctions of different types, traffic lights, and a dense urban layout, which makes it possible to exercise the planner on diverse situations within a single, consistent environment.

publishes the pseudo-sensor data consumed by the perception module.

### 3.3.3 Software environments: pixi

CARLA and ROS2 have conflicting dependency requirements, which makes installing them together a source of conflicts. To manage this, the project uses pixi, a package manager built on the conda ecosystem that creates isolated, reproducible environments described in a single manifest and locked to exact versions through a lock file [39]. Pixi natively supports defining multiple environments in one workspace, which is used here to keep CARLA and ROS2 in separate environments while installing ROS2 itself from the community `robostack` channel. This guarantees that the exact set of package versions can be recreated on another machine, which is central to the reproducibility of the study.

### 3.3.4 Hardware and software summary

All runs were performed on a single consumer laptop. Its GPU, with limited video memory, is relevant context for the latency analysis (Section 3.1.2.6). Running the local model on the same GPU as the simulator is what creates the resource contention discussed there. The full configuration is summarized in Table 11.

Table 11: Hardware and software configuration used for all experiments.

| **Component** | **Specification** |
|---|---|
| Laptop | HP OMEN 16-ap0xxx |
| CPU | AMD Ryzen AI 7 350 (16 threads) |
| GPU | NVIDIA GeForce RTX 5050 Laptop, 8 GB VRAM |
| RAM | 32 GB |
| Operating system | Ubuntu 24.04.4 LTS (kernel 6.17) |
| Simulator | CARLA 0.9.16 |
| Map | Town10HD |
| Middleware | ROS2 Humble, `carla-ros-bridge` |
| Environments | pixi (separate CARLA and ROS2 environments) |
| Python | 3.11 |
| CUDA | 12.8 |
| Local serving | Ollama 0.23.3 |
| Cloud access | OpenRouter (OpenAI-compatible API) |

### 3.3.5 Launch configuration file

The framework is brought up through a single ROS2 launch file (`demo_starter.launch.py`) that starts and orchestrates all the components in the required order. First, the `carla-ros-bridge`, then, the spawning of the ego vehicle and its sensors from `objects.json`, and finally, the three framework nodes. The components are started with delays rather than all at once, in order to avoid saturating the GPU during initialization, which on the available hardware would otherwise cause the bridge or the simulator to fail to come up. CARLA itself is launched separately, in its own pixi environment, before the launch file is run. This launch file is also where the model is selected for each run. As introduced above, the provider and model are passed as launch arguments (`llm_provider` and `llm_model`), which override the values in the `.env` file and are

recorded in the launch command, so that every run leaves an explicit trace of which model was used. Switching between the baseline model and the reference model therefore amounts to changing a single argument, with no modification to the code, which is the practical expression of the model-agnostic design discussed in Section 3.1.2.4.

# 4 Results

This chapter presents the results of running the framework with the two models across the five scenarios. The evaluation has two complementary aims. First, to compare the behavior of an on-device baseline model against a near-frontier cloud reference model; and second, to characterize how the framework as a whole behaves when driven by models of very different capability.

The results are presented from general to specific. The first section provides an aggregate comparison across all five scenarios, highlighting the overall differences between the two models. The following sections analyze each scenario individually, combining quantitative results with qualitative observations from the models' reasoning and driving behavior. A separate section examines the impact of the visual modality, and the chapter concludes with the study's limitations. The analysis focuses on describing observed behavior, while its implications for the hypotheses are discussed in Section 5.

Two families of metrics are reported, as defined in Section 3.2: output-quality metrics, which measure how reliably each model produced usable decisions, and driving-behavior metrics, which describe what the vehicle was actually commanded to do and how often the safety layer had to intervene.

Throughout this chapter, a distinction is maintained between the actions a model requests, recorded in the debug files, before the safety layer, and the actions actually committed to the vehicle, recorded in the dispatcher logs. Most quality metrics describe the requested actions, since they characterize the planner itself, whereas the driving behavior reflects the committed ones. Where the two differ, the difference is the work of the safety layer.

## 4.1 Aggregate quantitative comparison

Table 12 compares the two models aggregated over the five scenarios. The baseline model produced a valid, complete response in 91.5% of its calls, falling back to a safe `SLOW_DOWN` in the remaining 8.5%, whereas the reference model passed validation on every single call, with no fallbacks at all. The same asymmetry appears in every quality output related metric. The baseline accounted for all 11 hallucinated actions, all but one of the `target_speed` related issues, and an 11.0% rate of backward confusion, while the reference model did not show any of these. The baseline model is also slower during the inference process, with a median latency of 5.0 s against 4.0 s, consistent with the local GPU contention described earlier (Section 3.1.2.6).

Table 12: Aggregate metric comparison over the five scenarios.

| Metric | Qwen3.5:2B | Claude Haiku 4.5 |
|---|---|---|
| Total calls | 236 | 278 |
| Valid-call rate (%) | 91.5 | 100.0 |
| Fallback rate (%) | 8.5 (20) | 0.0 (0) |
| parse / missing / invalid | 3 / 6 / 11 | 0 / 0 / 0 |
| Hallucinated actions (instances) | 11 | 0 |
| Backward confusion (%) | 11.0 | 0.0 |
| target_speed issues | 13 | 1 |
| Low certainty (%) | 6.4 | 0.0 |
| Latency med / mean (s) | 5.0 / 5.5 | 4.0 / 4.5 |

The fallbacks of the baseline model are not arbitrary. They are dominated by invalid actions. Of the 20 fallbacks, 11 were triggered by the model emitting an action outside the valid set, against 6 missing fields and only 3 parsing errors. These invented actions (REVERSE, Turn Left, KEEP_SPEED_AND_STEER, BRAKE_AHEAD, STAY, HOLD, EXIT, SPEED_DOWN, STAND_STILL and NO_ACTION) are precisely the hallucinated actions counted above. The model, instead of selecting from the eight actions it was given, occasionally invents an action name of its own to describe the movement it wants to execute. Every one of these was caught by the whitelist and replaced by a fallback, so none ever reached the vehicle. The reference model never produced one.

Table 13: Action distribution aggregated over the five scenarios, comparing the actions *requested* by the planner (from the debug files) against those *committed* by the dispatcher after the safety layer (from the dispatcher logs). The obstacle scenario is counted once (second run), consistently with Table 14.

| | Qwen3.5:2B | | Claude Haiku 4.5 | |
|---|---|---|---|---|
| Action | Requested | Committed | Requested | Committed |
| FOLLOW_LANE | 93 | 84 | 202 | 192 |
| SPEED_UP | 8 | 5 | 1 | 1 |
| SLOW_DOWN | 17 | 49 | 57 | 62 |
| STOP | 64 | 55 | 16 | 10 |
| TURN_LEFT | 15 | 7 | 1 | 1 |
| TURN_RIGHT | 7 | 6 | 1 | 1 |
| CHANGE_LANE_LEFT | 8 | 4 | 0 | 0 |
| CHANGE_LANE_RIGHT | 10 | 8 | 0 | 0 |
| Total | 222 | 218 | 278 | 267 |

Table 13 makes the requested and committed duality explicit. The two columns differ precisely by the interventions of the safety layer, premature stops relaxed into slowdowns, unstable maneuvers blocked by the commitment mechanism, and invalid outputs replaced by fallbacks, so the gap between the two columns is itself a direct measure of how much the safety layer reshaped each model's behavior.

That gap is large for the baseline and small for the reference model. The baseline requested 64 `STOP` but only 55 were committed, and it requested 40 lateral maneuvers of which only 25 reached the vehicle. The difference reappears as `SLOW_DOWN`, which rises from 17 requested to 49 committed, absorbing the relaxed stops and the fallbacks. In other words, the safety layer suppressed roughly 40% of the baseline's lateral maneuvers and softened many of its stops. For the reference model, by contrast, the two columns are almost identical, with its 2 lateral maneuvers that were both committed, and only its `STOP` count that changed appreciably (16 requested, 10 committed), reflecting the distant red-light overrides discussed later. The safety layer therefore intervened heavily on the weak planner and only lightly on the strong one, acting in proportion to the instability of each, which is exactly the behavior a general safety substrate should exhibit.

Read at the level of requested actions, the two models still describe very different decision styles. The baseline requests `STOP` far more often than the reference model (64 against 16) and issues a large number of lateral maneuvers (40 against 2), many of them unwarranted, as the per-

scenario analysis will show. The reference model concentrates its decisions on `FOLLOW_LANE` (202 requests) complemented by gradual `SLOW_DOWN`, producing a smooth profile, whereas the baseline spreads its decisions across many more action types. The following section examines, scenario by scenario, both what each model decided and how the safety layer reshaped it into what the vehicle actually did.

## 4.2 Results per scenario

### 4.2.1 Free run

The free run is the simplest scenario, an empty road with no traffic and no events. With no external stimuli, the appropriate behavior is almost always to drive straight ahead, so any deviation reflects the model itself rather than the situation.

The reference model behaved accordingly. Across its 80 calls it issued 55 `FOLLOW_LANE` and 23 `SLOW_DOWN`, one `SPEED_UP`, a single `STOP` and no lateral maneuvers at all, and every response passed validation (Table 14). The control layer barely intervened, a single stale output event and nothing else, so nearly all of the actions issued were actually committed. The result is exactly what an empty road should produce, steady forward driving.

The baseline behaved very differently on the same empty road. Of its 67 calls, 19 were `STOP` and 10 were lateral maneuvers (turns and lane changes), none of which were really needed in the scenario (Figure 13). Of those 10 lateral requests, the commitment mechanism let only 5 through to the vehicle. This tendency appears from the very first call, in which the vehicle, stopped, facing nothing, is commanded to change lane:

> *"The vehicle is stopped at an intersection with no obstacles.* [...] *The ego vehicle has sufficient speed to proceed safely, so 'STOP' should be issued only after the ego vehicle has crossed the intersection and acquired a direction."*
> - Baseline model, call #1, action `CHANGE_LANE_LEFT` at 0.0 m/s

This is a clear example of an hallucination, as the reasoning is internally inconsistent. It states the vehicle has "sufficient speed to proceed" while the vehicle is stationary, and justifies a lane change with an argument about stopping, finishing with issuing an unrelated action to the scene (scenario is empty, there is no need to change lanes). Not only that, but the vehicle is already in the left lane, so issuing a change of lane to the left is dangerous. This kind of rationale is recurrent throughout the baseline's run.

Figure 13: Free-run scenario. Qwen3.5:2b. The model issues a `CHANGE_LANE_RIGHT` action without an appropriate reason. This figure is an example of how the baseline model issued unwarranted lateral maneuvers.

A second recurring pattern is the *backward confusion* introduced in Section 3.2. The baseline repeatedly reads the absolute heading (`yaw` near 180°as the vehicle physically moving in reverse:

> *"The vehicle is moving slowly (6.58,m/s) with a strong braking impulse [...] while pointing backward (yaw 179.49°) [...] the high negative acceleration suggests an urgent stop is needed."*
> - Baseline model, call #11, action `STOP` at 6.58,m/s

Here the model misinterprets a driving orientation as "pointing backward" and concludes that an urgent stop is needed, despite a clear road and a forward speed. This kind of error was recurring in previous baseline model run tests and so, it was specified in the behavior prompt in order to be prevented. The persistence of the error is a clear indicator of the baseline model's limited ability to process the meaning of complex input fields.

However, these erratic decisions did not translate into erratic motion and this is the central observation of the scenario. The unwarranted stops, the 10 requested maneuvers of which half were blocked, and the two hallucinated actions (`Turn Left` and `KEEP_SPEED_AND_STEER`, both rejected by the whitelist) were absorbed by the control layer. The hallucinated actions became fallbacks, the unstable turns were damped by the commitment mechanism (four commitment blocks), and the speed ramp smoothed the abrupt, unreasonable stops into continuous motion. The vehicle navigated the scenario despite a planner that, left unfiltered, would have driven it erratically. This shows how the safety layer is able to narrow the gap in the resulting behavior independently of the model, though the quality of the decisions taken remains the same.

A third pattern concerns the vehicle's response to traffic lights, the only genuine external stimulus present in this otherwise empty scenario. The reference model handled most of them reliably, slowing and stopping in front of the red lights along the route, even sometimes respecting the road markings or the junctions. The baseline model was less consistent, it did detect the red lights and often stopped for them, but its stops were imprecise. In several cases it braked far too early, coming to a halt well before the light, which is exactly the situation the `STOP` override was designed for, and the dispatcher logs record one such override in this run, relaxing a premature stop into a gradual slowdown. Figure 14 shows an imprecise stop from the baseline model, where the vehicle halts well short of the light. It can be contrasted with a correctly placed stop observed in the traffic scenario (Figure 15), from the reference model, where the vehicle halts at an appropriate distance from the light, directly over the road marking.

Two factors help explain this imprecision, neither of which is a reasoning failure in the strict sense. First, the planner is never given the road markings, as the behavior prompt describes traffic lights and objects but says nothing about stop lines, so the model has no reference for where exactly to halt, and stops relative to the light itself rather than to the marked stop line. This also accounts for the opposite error observed elsewhere, where the vehicle overruns the stop line and halts closer to the light than it should. Second, the traffic-light state reaches the planner through a latched topic that refreshes only when the state changes (Section 5.1), so between updates the model may reason over a traffic-light reading that no longer matches the current instant, which the scene context exposes through the `last_updated` field but cannot fully correct. Whether the front camera image contributes to the placement of these stops, by grounding the model in the visual position of the light and the markings, cannot be fully established here and remains part of the open question on the visual modality (Section 5).

The single `SPEED_UP` of the entire study, issued here by the reference model on a clear stretch, is worth a closing remark, as it exposes a redundancy in the action set. `SPEED_UP` and `FOLLOW_LANE`

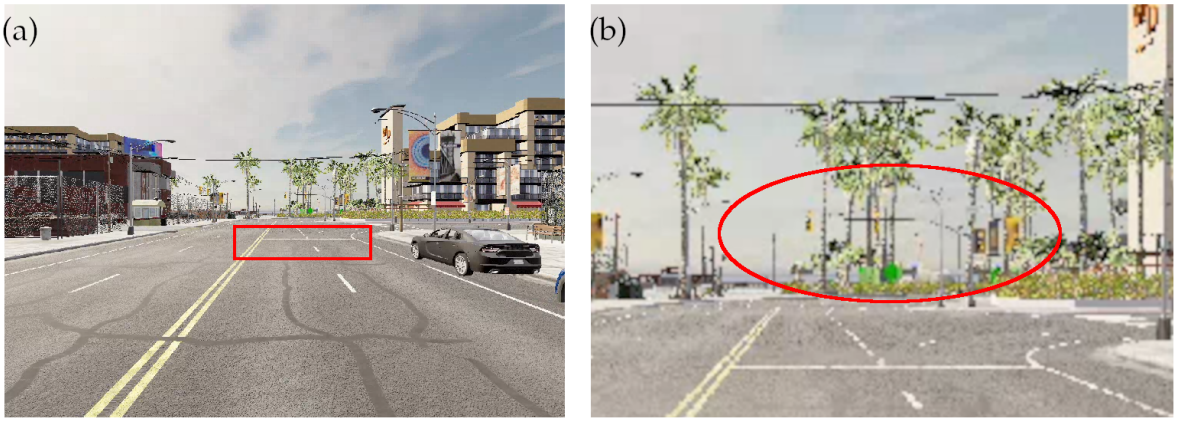


Figure 14: Free-run scenario. Qwen3.5:2b. Imprecise traffic-light stop of the baseline model in the free-run scenario: the vehicle comes to a halt far short of the red light. (a) General view, with the stopping position marked on the road; (b) detail showing that the light is still distant at the moment the vehicle stops.

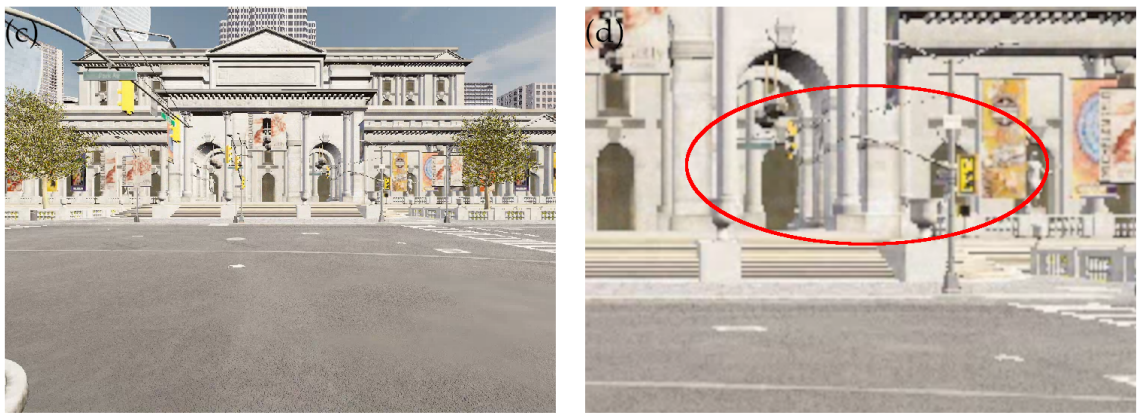


Figure 15: Traffic scenario. Claude Haiku 4.5. Correctly placed traffic-light stop in the traffic scenario: the vehicle halts close to the red light, at an appropriate stopping distance. (c) General view; (d) detail of the light directly ahead.

share the same valid speed range, both accept any cruise speed up to the limit, and differ only in a default that is almost never used, since the planner sets an explicit `target_speed` on nearly every driving call. A `FOLLOW_LANE` at a high target speed and a `SPEED_UP` therefore produce identical behavior, which makes `SPEED_UP` functionally redundant. Being redundant, it adds nothing but an extra option that the model can misuse, as the baseline did, requesting it on the mere assumption that an unseen traffic light was green, without ever enabling a maneuver that `FOLLOW_LANE` could not already express. Removing it from the action set would simplify the prompt and reduce the surface for this kind of error, and it is noted as a refinement for future work (Section 9).

Beyond the `target_speed` issues counted in Table 11, which cover only responses where the field is missing or non-numeric, the baseline exhibited a further misuse of the field that these metrics do not capture. The model frequently emitted a speed that was well-formed and numerically reasonable, yet inconsistent with the action it had chosen. A value inside the `SLOW_DOWN` range paired with a `FOLLOW_LANE`, or a cruise-level value paired with a `SLOW_DOWN`. These outputs are not malformed and would pass the validation chain, but they fall outside the valid range of the selected action family (Section 3.1.3.4), so the dispatcher's sanitization discarded them and applied the action's default speed. The pattern indicates that the baseline does not reliably associate each action with its corresponding speed band, treating `target_speed` as an unconstrained number rather than as a value related to the semantics of the action. It is an error of the same kind as the backward confusion discussed earlier, a misreading of what a field means rather than of its format, and like that error it concentrates in the baseline model. Because the sanitization silently corrects these values to a sensible default, the confusion never propagates to the vehicle's motion, but it is a further illustration of the reasoning gap between the two models that the aggregate metrics only partly reflect.

### 4.2.2 Traffic

The traffic scenario adds 15 ambient vehicles, therefore, in this case, braking and stopping actions are genuinely needed.

The reference model handled the traffic smoothly. Its 71 calls were dominated by `FOLLOW_LANE` (42) and `SLOW_DOWN` (22). It also requested 6 `STOP`, three of them overridden to `SLOW_DOWN`, as discussed below, and a single `TURN_LEFT`, all valid (Table 14). This turn deserves a closer look, because the model's reasoning is partly right and partly wrong. The maneuver is not initiated by the model but by CARLA's Traffic Manager, which is continuing the road as it bends slightly to the right. There is no intersection here. The model correctly detects that a turn is in progress, reading the non-zero `angular_velocity.z` as active turning rather than as noise, and it reasons correctly about not fighting an ongoing maneuver:

> *"A left turn is already in progress (non-zero angular_velocity.z indicates active turning). [...] interrupting an ongoing maneuver is counterproductive unless safety requires it. [...] Repeating TURN_LEFT allows the maneuver to complete naturally while maintaining commitment."*
> - Reference model, call #17, action `TURN_LEFT`

Where it misses is in the direction, as the road curves to the right, yet the model reads the turn as a left one and issues `TURN_LEFT`. It correctly identifies that the vehicle is turning, but misinterprets the sign of the motion, a directional confusion of the same family as the backward confusion discussed earlier, where an absolute directional field is read with the wrong sense. A second, milder issue is that a turn action is used at all: in this framework the turn actions are

meant for intersections, not for following a gently curving road, so applying one here is strictly a misuse, though this owes more to the reactive design and to the absence of an explicit rule for curved roads than to a reasoning failure. In practice neither error changed the vehicle's path, since the actual steering was driven by the Traffic Manager continuing the road to the right regardless of the model's command, as shown in Figure 16.

Figure 16: Traffic scenario. Claude Haiku 4.5. Issuing of the `TURN_LEFT` action. The model correctly identifies that the car is turning but fails to identify the correct orientation. Moreover, it issues the action at a curving road rather than at a junction.

Beyond these isolated cases, the reference model's core behavior in traffic was a correct, graduated longitudinal response to the vehicles ahead. A representative sequence is its approach to a car stopped at a red light, shown in Figure 17. Far from the stopped vehicle the model keeps `FOLLOW_LANE`; as the gap closes it switches to `SLOW_DOWN`, reducing speed in anticipation; when the vehicle is close it commits to `STOP`; and once the leading vehicle sets off again, it releases the stop back into a `SLOW_DOWN` to resume motion. The model modulates its speed progressively with the distance to the dynamic obstacle, rather than switching abruptly between extremes, which is exactly the smooth longitudinal profile expected of a competent driver and the one the speed ramp is designed to preserve.

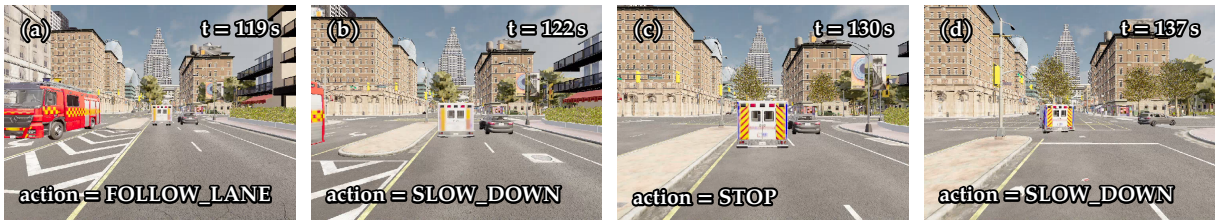


Figure 17: Traffic scenario, Claude Haiku 4.5. Graduated sequential response to a vehicle stopped at a red light: (a) `FOLLOW_LANE` while the leading vehicle is still far; (b) `SLOW_DOWN` as the gap closes; (c) `STOP` when the vehicle is close; (d) `SLOW_DOWN` once the leading vehicle sets off again. The model modulates speed progressively with the distance to the obstacle.

The baseline again produced a far noisier decision stream, with 61 calls including 12 `STOP` requests, four of which the override relaxed to `SLOW_DOWN`, leaving 8 committed, several lateral

maneuvers, four hallucinated or unparseable outputs (`BRAKE_AHEAD`, `STAY`, `HOLD`, and one parse failure), and five low-certainty responses. The same curving stretch that produced the reference model's directional mistake also prompted a `TURN_LEFT` from the baseline, at roughly the same point, which makes the two rationales directly comparable. The baseline reached the same wrong action, but through more confused reasoning:

> *"TURN_LEFT is the only logical action as there is no traffic ahead; the significant yaw rate means a turn is already in progress, but continuing straight would be unsafe. The high angular velocity confirms the vehicle is navigating the intersection, making a new turn impossible."*
> - Baseline model, call #24, action `TURN_LEFT`, certainty HIGH

Like the reference model, the baseline correctly detects an active turn from the yaw rate, and like it, it reads the direction as left when the road curves right. But the rest of the rationale is considerably weaker. It invents an intersection that is not there ("the vehicle is navigating the intersection"), when the vehicle is simply following a curving road; and it is internally contradictory, first declaring that turning would be unsafe ("continuing straight would be unsafe [...] dangerous oversteer") and then committing to the turn as "the only logical action". The decision is moreover issued at `HIGH` certainty, so the model is confidently wrong rather than hesitant. Where the reference model's error was a single wrong sign within otherwise coherent reasoning, the baseline arrives at the same action through a rationale that misreads the scene, fabricates a junction, and contradicts itself. Notably, however, its `STOP` decisions in this scenario were almost all triggered by a genuine red light ahead, the model did detect the red traffic lights. Its problem was one of *calibration* rather than detection, as it tended to brake to a full stop far too early. This is precisely what the `STOP` override was designed for, and the dispatcher logs show it correcting the baseline four times, each for a red light still tens of meters away with no object in front:

```
STOP overridden → SLOW_DOWN: red TL at 63.0 m > 20.0 m, nearest frontal object
inf  STOP overridden → SLOW_DOWN: red TL at 51.0 m > 20.0 m, nearest frontal object
inf  STOP overridden → SLOW_DOWN: red TL at 39.9 m > 20.0 m, nearest frontal object
inf
```

In each case the baseline model committed to a full stop for a red traffic light between 40 to 60 m away, and the override relaxed it to a gradual slowdown, avoiding an abrupt and premature halt.

The reference model's three overrides were of a different nature. They were triggered by red lights much closer, around 20 to 26 m and with a frontal object 10 to 12 m ahead, that is, in tighter situations where the model's `STOP` response was reasonable but slightly conservative, that the override refined into a slowdown. The same safeguard acted on two qualitatively different inputs by correcting the baseline's large misjudgment of distance on one hand, and refining the reference model's already close decision on the other.

The baseline model reasoning also revealed errors that did not change the action but exposed misunderstanding or confusion of traffic rules. In several stops it invoked the right rule for the wrong reason, or contradicted itself within the same field, as in a response that announces a `FOLLOW_LANE` action inside the rationale while emitting `STOP`:

> *"FOLLOW_LANE chosen because the light ahead is RED. Therefore, braking distance from self_data must be utilized to determine appropriate deceleration to stop [...]"*
> - Baseline model, call #42, action `STOP`

Here the action emitted (`STOP`) happened to be appropriate, but the rationale names a different action and offers a non-pertinent justification. This shows how correct decisions from the baseline model do not always mean correct rationale.

The volatility also appeared in the baseline's sequential behavior. Where the reference model modulated its speed smoothly with the distance to the vehicles ahead (Figure 17), the baseline was far more abrupt, switching between many different actions, issuing `SLOW_DOWN` and `STOP` when the situation did not call for them, and frequently pairing these with confusing or self-contradictory rationales. The graduated approach that characterized the reference model, holding, easing, stopping, and releasing in step with the closing gap, did not emerge here. Instead, the baseline produced a more erratic stream of decisions that the control layer had to absorb. As in the free run, these unstable decisions were contained by the control layer, four fallbacks for the invalid outputs, one commitment block, and the four overrides, so that the vehicle's actual motion remained controlled despite the noise in the decision stream.

### 4.2.3 Obstacle

The obstacle scenario places a stationary vehicle in the ego's lane, 40 m ahead of the spawn point, on top of the ambient traffic. It is the first scenario that poses a genuine dilemma with more than one defensible answer. Facing a blockage in the middle of the road, the planner can stop behind it and wait, or attempt to go around it by changing lane. It is also the only scenario run twice per model, which makes it the one place where consistency of each model across repetitions can be observed.

Figure 18: Obstacle scenario. Claude Haiku 4.5. Cautious approach to the stationary vehicle blocking the lane. The model approaches with successive `SLOW_DOWN` actions at a safe distance from the obstacle (a, b) and comes to a controlled `STOP` behind it (c), anticipating the blockage rather than reacting to it at close range.

The reference model resolved the situation identically in both runs. It approached the stopped vehicle, slowing down until it halted behind it, and remained stationary until the end (Table 14; Figure 18). In both runs it requested almost exclusively `FOLLOW_LANE` (63 and 70 in the first and second run), with four `SLOW_DOWN` each, two `STOP` in the second run only, and no lateral maneuvers in either. Almost every action was committed unchanged, consistent with the model's stability. Its behavior is reproducible, as confronted with the blockage, it consistently chose to wait. This produced the most distinctive control-layer figure of the study, five stale-output events in each run, which is not a failure of the model but a consequence of deadlock it created. Once both the ego and the vehicle ahead were stationary, the scene stopped changing, and the model's responses to a static context occasionally exceeded the eight-second watchdog window, prompting the dispatcher to issue a safe `SLOW_DOWN`. The model's own reasoning describes the dead end:

> *"Vehicle is stationary (speed 0.0,m/s) with a car stopped directly ahead at 6.3 m [...] both vehicles are stopped and the front vehicle is at a safe distance."*
> - Reference model, second obstacle run, call #10

The baseline model behaved very differently across the two runs. In the first run, it stopped considerably, requesting `STOP` on 34 of its 54 calls, repeatedly halting in front of the obstacle with only isolated attempts to maneuver. In the second run its behavior shifted toward action, with more lateral maneuvers requested (15 against 7), long stretches of `FOLLOW_LANE`, and a noisier output overall, including four invalid or unparseable responses and the highest fallback count of the study (7). Faced with the same scenario, the baseline produced two different behavioral profiles. This difference in self-consistency is the property that running the scenario twice was meant to reveal. The inconsistency extends beyond the choice of actions to the quality of the outputs themselves. Faced with the same scenario, the baseline's backward-confusion rate fell from 22.2% in the first run to 7.8% in the second, and its share of valid responses dropped from 98.1% to 89.1% (Table 14). Not only did the model decide differently across the two runs, it also reasoned with markedly different reliability. When the baseline model did attempt to go around the obstacle, its reasoning revealed that these maneuvers were not the product of deliberate planning. From the first call of the second run it read the stopped car and decided to change lane:

> *"Change_lane_left is selected to avoid a car ahead in the same lane that has a very low speed of 0.0 m/s."*
> - Baseline model, second obstacle run, call #1, action `CHANGE_LANE_LEFT`

The intent here is reasonable, but elsewhere the same tendency surfaced through incoherent justifications, such as one invoking an urgency-prefixed action it was never given and a false dilemma in which stopping, which is clearly available, is declared unsafe:

> *"URGENT_CHANGE_LANE_RIGHT required to avoid collision with the 8.8 m car ahead, as stopping is unsafe and continuing forward is unsafe due to obstruction."*
> - Baseline model, second obstacle run, call #7, action `CHANGE_LANE_RIGHT`

The action field still resolved to a valid `CHANGE_LANE_RIGHT`, but the rationale fabricates an action name and a false premise, indicating that the baseline's maneuvering is better read as part of its restless, unstable decision-making than as a consistent strategy for clearing the blockage. Two observations stand out from this scenario, both of which are developed in the discussion (Section 5). The first is the reference model's choice to wait rather than overtake, which connects to the absence of a navigation goal. The second is the contrast in self-consistency between the two models across repeated runs, directly relevant to the single-run nature of the rest of the study.

The two models also differed in how they approached the blockage. Where the reference model anticipated it, easing off with `SLOW_DOWN` well before reaching the stopped vehicle (Figure 18), the baseline kept `FOLLOW_LANE` at cruising speed on its approach and only reacted once it was close, braking or maneuvering late rather than in anticipation (Figure 19). The safety layer absorbed the difference, but the reference model's approach was the smoother and safer of the two.

#### 4.2.4 Pedestrian

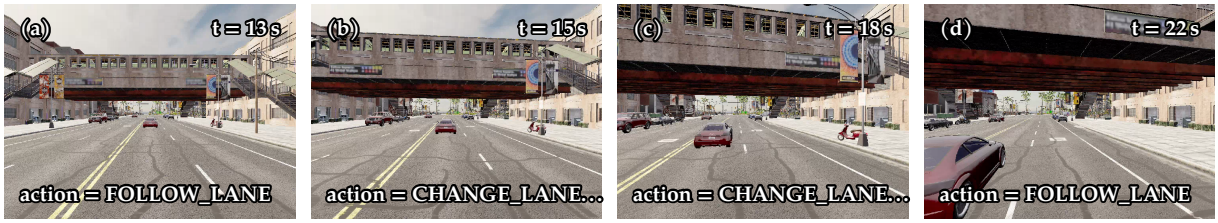


Figure 19: Obstacle scenario. Qwen3.5:2B. Approach to the same blockage: the baseline keeps `FOLLOW_LANE` at speed until close to the obstacle (a), then attempts to go around it with two `CHANGE_LANE_RIGHT` actions (b, c) before returning to `FOLLOW_LANE` (d). The reaction comes late and the maneuver, as the rationale shows, is not part of a deliberate plan.

The pedestrian scenario introduces a pedestrian that crosses the ego's path as it approaches. Unlike the ambulance discussed below, this agent crosses in front of the vehicle, so it does enter the field of the frontal camera and it also appears as a specific `PEDESTRIAN` object in its respective scene context channel. The pedestrian is correctly reported as a `PEDESTRIAN` object in the `FRONT` direction at decreasing distances in both runs. This scenario tests whether each model reacts to it appropriately, because a failure here has the most serious safety implications. This scenario is the most consequential of all.

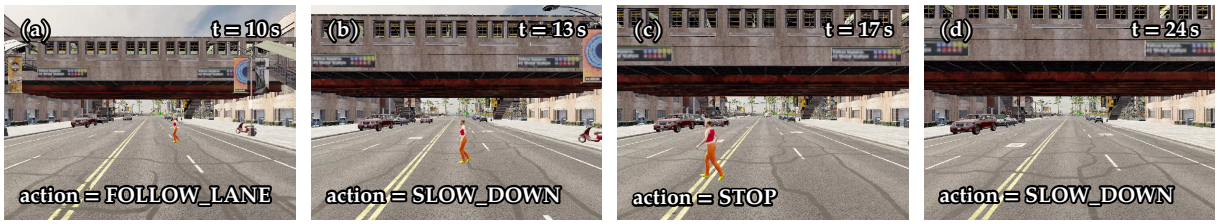


Figure 20: Pedestrian scenario. Claude Haiku 4.5. Graduated response to the crossing pedestrian: (a) `FOLLOW_LANE` while the pedestrian is still far; (b) `SLOW_DOWN` as the pedestrian begins to cross and the distance becomes unsafe; (c) `STOP` when the pedestrian is directly ahead, issued shortly after the Traffic Manager has already begun braking; (d) `SLOW_DOWN` once the pedestrian leaves the lane but remains nearby. The reasoning matches the action at each step.

The reference model reacted correctly and safely, and its behavior is captured in Figure 20. While the pedestrian was still far, it kept `FOLLOW_LANE`; as the pedestrian began to cross and the distance became unsafe, it eased off with `SLOW_DOWN`; and when the pedestrian was directly ahead it committed to `STOP`, referring explicitly to the relevant safety rule:

> *"STOP is mandated by guardrail 2: pedestrian is in FRONT direction at 8.4m [...] represents an unacceptable collision risk given the pedestrian's proximity and the vehicle's current acceleration trajectory."*
> - Reference model, call #6, action `STOP`

Across the critical window, with the pedestrian between roughly 5 and 9 m ahead, the reference model consistently produced `STOP` or `SLOW_DOWN`, citing the pedestrian's position and the braking-distance rule, and once the pedestrian left the lane but remained nearby it returned to a cautious `SLOW_DOWN` rather than resuming full speed. Its rationale tracked the pedestrian accurately at every step, and its behavior in the presence of the vulnerable user was measured and appropriate.

One detail of the reference model's run is worth noting, because it concerns the interaction between the planner and the low-level controller rather than the decision itself. The actual braking in front of the pedestrian was initiated by the Traffic Manager's own collision response, which acts on a larger distance threshold than the framework's, slightly before the model's `STOP` took effect. The inference latency of the planner, several seconds per decision, meant that by the time the model committed to stopping, the Traffic Manager had already begun to brake for the approaching pedestrian. This is not a limitation but an illustration of why a reliable low-level control layer is essential. It handles the immediate, close-range situations that a high-level planner, operating on a slower cycle, cannot always react to in time.

The baseline's behavior in the same scenario is the most concerning result of the study, and it must be read carefully, because at first sight it looks safe. Its committed sequence, once past an erratic opening of unwarranted actions (`SPEED_UP` and `TURN_LEFT` with the pedestrian still 35 m away), settles into `FOLLOW_LANE`, `SLOW_DOWN`, and `STOP`, halting in front of the pedestrian (Figure 21). Taken alone, this looks more cautious than the reference model, since the baseline stops readily. But the appearance is misleading: the baseline stops compulsively throughout the run, issuing `STOP` on 7 of its committed calls, including one at 12.9 m while travelling at 7.7 m/s, well before it was needed. Its readiness to stop is a symptom of its unstable, over-triggered decision-making, not of good judgment, and the same instability produced a genuinely dangerous decision at close range.

Figure 21: Pedestrian scenario. Qwen3.5:2b. Committed sequence in front of the crossing pedestrian: (a) `FOLLOW_LANE` on approach; (b) `SLOW_DOWN`; (c) `STOP` in front of the pedestrian; (d) `FOLLOW_LANE` resuming once the pedestrian is no longer in the lane. Behind this apparently safe sequence, the baseline stopped compulsively throughout the run and, at one point with the pedestrian 6.5 m still ahead, briefly commanded `FOLLOW_LANE` instead of braking or proceeding slowly. Finally when the pedestrian is clearly outside the front cone the vehicle continues with `FOLLOW_LANE`

At the moment the pedestrian was directly in front at 6.5 m, right after the pedestrian had almost crossed, the baseline did not proceed cautiously but commanded the vehicle to keep going forward:

> *"FOLLOW_LANE selected as the car ahead maintains rightward path and vehicle speed is low, making lane following the safest and natural action."*
> - Baseline model, call #8, action `FOLLOW_LANE`

The rationale is both incoherent and unsafe. It refers to a "car ahead" that is in fact the pedestrian, and concludes that continuing forward is the safest action while a pedestrian stands 6.5 m directly in the vehicle's path. The baseline did issue a `STOP` on the following call, once the situation had developed further, but the earlier failure to brake, or at least slow down, at close range is a serious issue in an autonomous driving context. This is an issue specially when the lack of a navigation history is taken into account, as the model should have reasoned over the scenario as

if the pedestrian was about to cross, therefore acting cautiously. It must also be noted that this single dangerous decision was not caught by any safeguard. The scene context contained no red traffic light, so the `STOP` override was not involved, and `FOLLOW_LANE` is a valid action, so the whitelist did not reject it. The safety layer protects against malformed and unstable outputs, but it cannot correct a well-formed decision that is simply wrong, a limitation this scenario makes concrete.

The baseline's reasoning in this run also drifted into perceptual errors. In one call it described the scene as an "overhead pedestrian bridge with a red traffic light ahead". The bridge is real, as the pedestrian crosses beneath an overpass present in the map, but the red traffic light is not, there is no traffic light in this scenario, and none in the scene context. The model thus mixes a correct visual observation with a fabricated one, which is consistent with the uneven reliability of its visual descriptions discussed in Section 5.

This scenario also produced the baseline model's highest rate of backward confusion of the whole study, at 35.3%. It described the vehicle as "moving in reverse" or "facing backward" on the basis of a `yaw` near 180°, even while it was driving forward toward the crossing:

> *"Vehicle is moving in reverse (yaw ~180°) toward pedestrians on the road ahead at 23.6 m distance."*
> - Baseline model, call #6

The concentration of this error in the pedestrian scenario is likely an artifact of the route geometry: the ego happens to travel on a west orientation here, with `yaw` close to 180°throughout, continuously fed to the baseline model. The reference model, given the identical orientations, never made this error.

### 4.2.5 Ambulance

The ambulance scenario was included deliberately as a demonstration of a perception limitation rather than as a test of decision-making (Section 3.2). An emergency vehicle approaches the ego from behind, faster than the surrounding traffic and with its beacons active, and the expected behavior of a competent driver would be to detect it and give up its position in the road, changing lanes or moving away from the ambulance. The result was the same for both models, which never gave up their position in the road because neither of them were ever given the information needed to do so.

The cause is structural. The ambulance approaches from the rear, and the only camera is facing the front of the vehicle, so the ambulance is never in view. Moreover, in the scene context it appears only as a generic `CAR` object in the rear sectors. Across the run it was reported almost exclusively in `BEHIND` directions, never entering the frontal cone as it closed in, with a minimum reported distance of about 8 m directly behind. Furthermore, CARLA's object topic carries no value to identify an emergency vehicle, so the ambulance is reported with the same generic vehicle class as any other car, and its only distinguishing feature, the visual beacons, would only be perceivable through the front camera, where it never is. Consistently with this, none of the models ever referred to an ambulance, an emergency, or flashing lights in its reasoning across the entire run, leaving both models unable to perceive it.

Figure 22 makes this concrete. In the LiDAR view, used purely for visualization and not connected to the planner, a vehicle can be seen directly behind the ego throughout the scene. This is the ambulance, which never overtakes and never enters the frontal camera's field of view. The

evidence that the emergency vehicle is present and persistent is therefore available in the raw sensor space, but not in the channels the framework actually consumes, which is precisely why neither model ever refers to it.

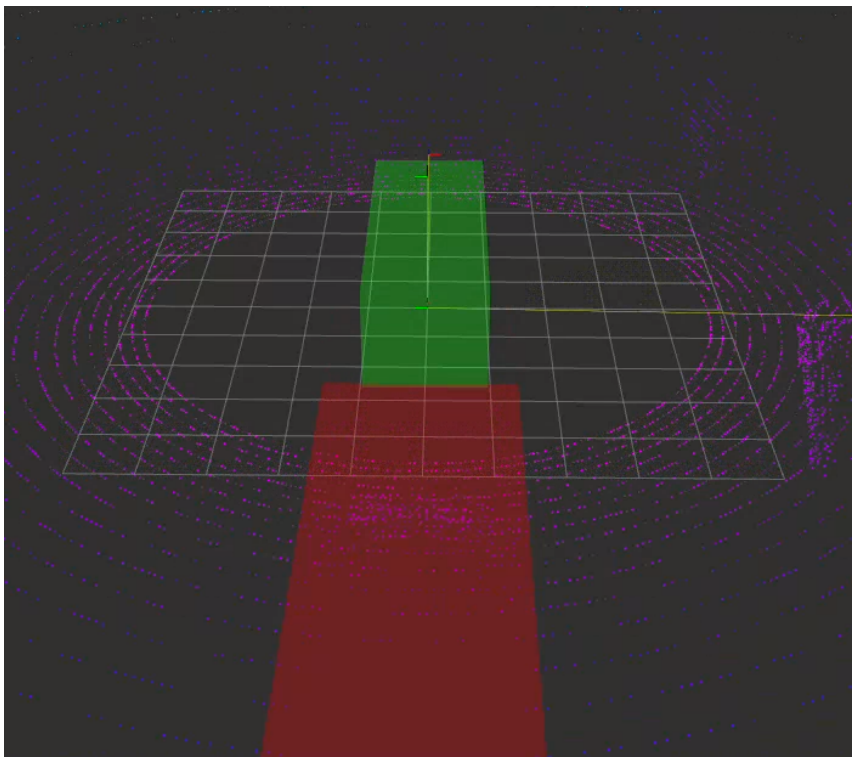

Figure 22: Ambulance scenario, LiDAR point cloud shown for visualization only. The LiDAR is *not* part of the framework and does not feed the planner: the ego relies solely on the pseudo-sensors that build the scene context (Section 3.1.1), and this view is used here merely to make the ambulance visible to the reader. The point cloud shows a vehicle immediately behind the ego, the emergency vehicle, which remains in that position throughout the scene without overtaking. Because it approaches from the rear, outside the frontal camera's field of view, and is reported only as a generic `CAR` in the rear sectors of the object channel, the planner never receives the information a competent driver would need to make way for the ambulance.

This is not considered a failure of the models but a limitation of the framework's perception. The scenario does not show that the models cannot reason about emergency vehicles, only that this particular perception setup cannot provide the evidence that such reasoning would require. This issue is further discussed in the analysis of the framework's limitations (Section 5).

Setting the ambulance aside, the rest of the scenario is consistent with the patterns already established. The reference model drove steadily (24 `FOLLOW_LANE` of 36 calls, with 6 slowdowns, 1 `TURN_RIGHT`, and 5 `STOP` requested, two of them overridden for nearby red lights). The baseline model remained unstable, producing three hallucinated outputs, including two `REVERSE` actions, a particularly clear instance of its backward confusion crossing over into the action field, and four low-certainty decisions, all absorbed by the control layer.

Table 14: Decision quality per scenario and model (Qwen3.5:2B vs Claude Haiku 4.5). The obstacle scenario, run twice, is reported per run. The aggregate row counts each scenario once, using the second obstacle run.

| Scenario | Model | Calls (n) | Valid (%) | Fallback (%) | Halluc. (n) | Backward (%) | TS issues (n) | Lat. med. (s) |
|---|---|---|---|---|---|---|---|---|
| Free | 2B | 67 | 94.0 | 6.0 | 2 | 1.5 | 2 | 6.0 |
| | Haiku | 80 | 100.0 | 0.0 | 0 | 0.0 | 0 | 4.0 |
| Traffic | 2B | 61 | 93.4 | 6.6 | 3 | 9.8 | 5 | 6.0 |
| | Haiku | 71 | 100.0 | 0.0 | 0 | 0.0 | 0 | 5.0 |
| Obstacle (run 1) | 2B | 54 | 98.1 | 1.9 | 1 | 22.2 | 3 | 5.0 |
| | Haiku | 67 | 100.0 | 0.0 | 0 | 3.0 | 1 | 4.5 |
| Obstacle (run 2) | 2B | 64 | 89.1 | 10.9 | 2 | 7.8 | 2 | 5.0 |
| | Haiku | 76 | 100.0 | 0.0 | 0 | 0.0 | 0 | 4.0 |
| Pedestrian | 2B | 17 | 88.2 | 11.8 | 1 | 35.3 | 1 | 5.5 |
| | Haiku | 15 | 100.0 | 0.0 | 0 | 0.0 | 0 | 4.0 |
| Ambulance | 2B | 27 | 88.9 | 11.1 | 3 | 29.6 | 3 | 5.0 |
| | Haiku | 36 | 100.0 | 0.0 | 0 | 0.0 | 1 | 5.0 |
| Aggregate | 2B | 236 | 91.5 | 8.5 | 11 | 11.0 | 13 | 5.0 |
| | Haiku | 278 | 100.0 | 0.0 | 0 | 0.0 | 1 | 4.0 |

### 4.3 Visual modality

A distinctive feature of the framework is that the planner is a vision-language model, receiving the frontal camera image alongside the textual scene context. Therefore, it is natural to ask what is the visual modality contribution to the decisions. The results allow only a partial answer, showing clear evidence that the image is taken into account, but its reliability differs significantly between the two models, and its causal effect on the decisions cannot be isolated in the current study.

That the models attend to the image is not in doubt. Throughout the runs, both models describe elements of the scene that are absent from the textual context but visible in the image. The scene context contains only speeds, distances, relative directions, object classes, and traffic-light states. It says nothing about the appearance of the environment. Yet the models routinely narrate visual features such as an overpass, a bridge, palm trees, road markings, or a zebra crossing, which can only come from the image (Figure 23):

> "[...] *actively turning* [...] *urban road with palm trees and buildings visible* [...]"
> - Reference model, free run, call #17

Contrasting every call against the recorded video is not feasible given their number and the complexity of tracing all of the actions inside the videos with no matching timestamp. This analysis is based on a subset of manually reviewed calls. Within that subset, however, a clear and consistent difference emerges between the two models. The reference model mostly describes the visual elements correctly, both the elements themselves and their purpose, with occasional confusions. For instance, it sometimes misplaces itself in the scene, as in a call that reports a "plaza" while the vehicle is on an ordinary street, but it correctly identifies structures such as an overpass it is passing under, and its descriptions are generally coherent with what the front camera shows. Its visual perception, while not flawless, is largely reliable.

The baseline model behaves differently. It does describe some visual elements correctly, as the reference model does, detecting, for example, a bridge or a double yellow line that are genuinely

Table 15: Control-layer events per scenario and model (Qwen3.5:2B and Claude Haiku 4.5), from the dispatcher logs. The obstacle scenario is reported per run.

| Scenario | Model | stale | STOP override | commitment | fallback |
|---|---|---|---|---|---|
| Free | 2B | 1 | 1 | 4 | 4 |
| | Haiku | 1 | 0 | 0 | 0 |
| Traffic | 2B | 0 | 4 | 1 | 4 |
| | Haiku | 1 | 3 | 1 | 0 |
| Obstacle (run 1) | 2B | 0 | 0 | 0 | 1 |
| | Haiku | 5 | 0 | 0 | 0 |
| Obstacle (run 2) | 2B | 0 | 1 | 3 | 7 |
| | Haiku | 5 | 0 | 0 | 0 |
| Pedestrian | 2B | 0 | 0 | 1 | 2 |
| | Haiku | 0 | 0 | 0 | 0 |
| Ambulance | 2B | 0 | 0 | 0 | 3 |
| | Haiku | 1 | 2 | 0 | 0 |

present, but it frequently distorts the surrounding context or hallucinates elements that are not there. In several reviewed calls it correctly reads part of the image and then misinterprets it. For instance, it detects a real double yellow line but invents an intersection to explain it, or it detects a real bridge but adds a red traffic light and a crosswalk that do not exist. In its most extreme form this drifts into outright hallucination:

> *"There is a red train overhead and a parked car in front and behind."*
> - Baseline, ambulance run, call #7

The pattern in the baseline is thus not a failure to see the image but a failure to integrate it coherently. It attends to the visual input, but the visual information tends to confuse its reading of the situation more than it grounds it, adding fabricated context around real elements rather than clarifying the scene. This extends its general tendency to hallucinate, already seen in the textual domain, into the visual one.

The more fundamental limitation is that, beyond this qualitative difference in reliability, the contribution of the image to the decisions cannot be isolated in this study, for two structural reasons. First, the information most relevant to the driving decisions, the state of the traffic lights and the position and distance of objects, is already provided as ground truth in the textual context. The image is therefore largely redundant with the text for decision-making, and in no observed case did a visual observation override what the numeric data already implied. Second, the design includes no ablation study, as the models were never run on the text alone, so there is no contraposition against which to measure what the image adds. Establishing the visual contribution would require such a study, running each scenario with and without the image, as discussed in the future work (Section 5).

### 4.4 Study limitations

Several limitations were identified, limiting the scope of this study.

Figure 23: Front camera frame corresponding to the reference model's call cited above (free run, call #17). The palm trees and the surrounding buildings that the model reports in its analysis are both present in the image, neither of which appears in the textual scene context, confirming that the model attends to and incorporates the camera image rather than reasoning from the text alone.

The most important one concerns the statistical weight of the results. Each scenario was run once per model, with the only exception of the obstacle scenario. This is not a statistical study as the variance between runs was not characterized, and therefore, the resulting values should be read as indicative only. The obstacle repetition is a reminder of this limitation, since the baseline produced two different behaviors across its two runs. With a single sample per cell, the results support statements about the behavior observed in these specific runs and in general across the five scenarios, but further samples should be obtained to make absolute claims.

A second limitation is the prompt asymmetry between the two models. The cloud path prepends a note clarifying that the images are simulation renders, which the local path does not carry (Section 3.1.2.5). This was necessary to obtain a usable output from the cloud model, but it means the two models were not prompted identically. Although the difference is small and probably negligible it has to be noted.

A third limitation concerns the visual modality, as discussed above. The causal contribution of the image could not be isolated and an ablation study is needed.

Finally, the study is limited in generalization. All runs were performed on a single map (Town10HD) from a single fixed spawn point, and the reported latency is an elapsed value rather than a measurement (Section 3.1.2.6). Generalization to other maps has not been evaluated.

# 5 Discussion

The purpose of this work was to evaluate whether a hierarchical framework, with a vision-language model used as a high-level planner and a deterministic layer handling everything else, can drive safely even when the model of the planner is weak. The results, read together, answer that question directly, and the hypotheses that shaped it.

The main finding concerns the safety substrate, supporting H1. The control layer absorbed the baseline model instability without letting it reach the vehicle. Malformed responses became fallbacks, hallucinated actions were blocked by the whitelist, premature stops were relaxed, and conflicting commands were damped by the commitment mechanism. Nonetheless, the same safeguards also acted on the reference model, correcting its distant red-light stops and covering its occasional silences, which shows that the layer is not tuned to absorb the specific issues of one model but it can be generalized to others. The most telling evidence is that the models outcomes converge, having both completed the five scenarios and having stopped in front of the obstacle, despite a large gap in the quality of their decisions. That convergence is precisely the point. The value of this architecture is reflected by how much it narrows the outcome gap despite the model gap, lifting a weak planner to viable behavior. The only notable difference is in the smoothness, but not on safety. This claim, however, must be honestly delimited because, as the pedestrian scenario shows, the safety layer guards against malformed or unstable outputs, but it cannot correct a decision that is well-formed and genuinely wrong. When the baseline drove on toward a pedestrian while emitting a valid `FOLLOW_LANE`, nothing intervened, because nothing about the output was malformed. H1 therefore holds but within a clear boundary. The safety layer limits the damage from erratic behavior, but it is not a substitute for a reliable planner, which is exactly why the capability of that planner matters.

That capability difference, in turn, supports H2. The gap between the two models is notable and consistent across every quantitative metric, but the more interesting difference is qualitative. The reference model reasons about the numbers, comparing distances against the braking distance, differentiating governing traffic light from cross-traffic ones, tracking its own state, whereas the baseline model largely relies on matching superficial patterns, producing hallucinations irregularly, the most relevant one when reading the yaw parameter as heading backwards instead of a driving orientation, despite the specific clarification in the system's prompt. With only two models this can not be stated as an absolute but it is a consistent observation at two representative points of the capability range. Within a fixed architecture, a more capable model returns cleaner and safer decisions with no change to the system around it, which is the intended goal of the model-agnostic design.

The visual modality (RQ) is where the study is most limited. The image is clearly noted, as both models describe elements absent from the textual context, but its contribution cannot be isolated, because the relevant decision information is already provided as ground truth in the text, and because no ablation study was run to compare with only text in the prompt. Nonetheless, the results do add one qualitative finding on this front. the two models attend to the image with markedly different reliability, the reference model describing the scene largely correctly while the baseline frequently distorts or fabricates visual elements, extending its textual tendency to hallucinate into the visual domain. This tells us how each model reads the image, but not whether that reading changes the decision, which remains the open question an ablation study would settle.

## 5.1 Framework limitations

Beyond the limitations of the study itself, the framework has structural limitations that bound its current scope and point toward future work.

The first is the freshness of the traffic-light information. It reaches the planner through a latched topic that only refreshes when the state changes, so between updates the vehicle may reason over old data, and traffic-light compliance is reliable only when the information is recent. A dedicated computer-vision pipeline that detects traffic lights from the image, real-time, and estimates their state and distance at a constant, low-latency rate would remove this dependency.

The second concerns the low-level control. It is delegated to the Traffic Manager through its pre-configured functions, which are built on a PID controller. This is robust and deterministic, but rigid. A learned controller such as reinforcement learning, which has already been shown effective as the coordinating layer of hierarchical driving frameworks [26], would be a more flexible alternative, and the use of reinforcement learning or world models is likely the path toward scalability of the framework to more complex scenarios and the transfer from simulation to the physical world.

The third is the absence of a navigation goal, which the obstacle scenario exposed. Because the planner is purely reactive, the reference model, confronted with the blockage, simply stopped and waited. With no destination to reach, it had no reason to overtake. The same issue is why an empty route lets the Traffic Manager choose junctions on its own (Section 3.1.3.2). Introducing goal-oriented navigation, which the GNSS and map sensors, deliberately left unused, would support, would give the planner the intent that its current reactive design lacks.

The fourth is the handling of emergency vehicles, which the ambulance scenario was designed to assess. The framework has no pathway to perceive one, the object channel exposes only a generic vehicle class with no emergency category, and the sole distinguishing cue (the beacons) is visual and lies outside the single front camera when the vehicle approaches from behind. Emergency-vehicle handling would require either a richer sensor set or a rear field of view, and is left as future work.

Finally, the baseline model behavior is a reminder that small models raise the rate of hallucination and lower decision quality. While the safety layer contained this in the tested scenarios, a heavier reliance on the planner, or more complex situations, would expose the limits of what a 2-billion-parameter model can support, reinforcing that model capability and architectural safeguards are complementary, not interchangeable.

# 6 Planning and Scheduling

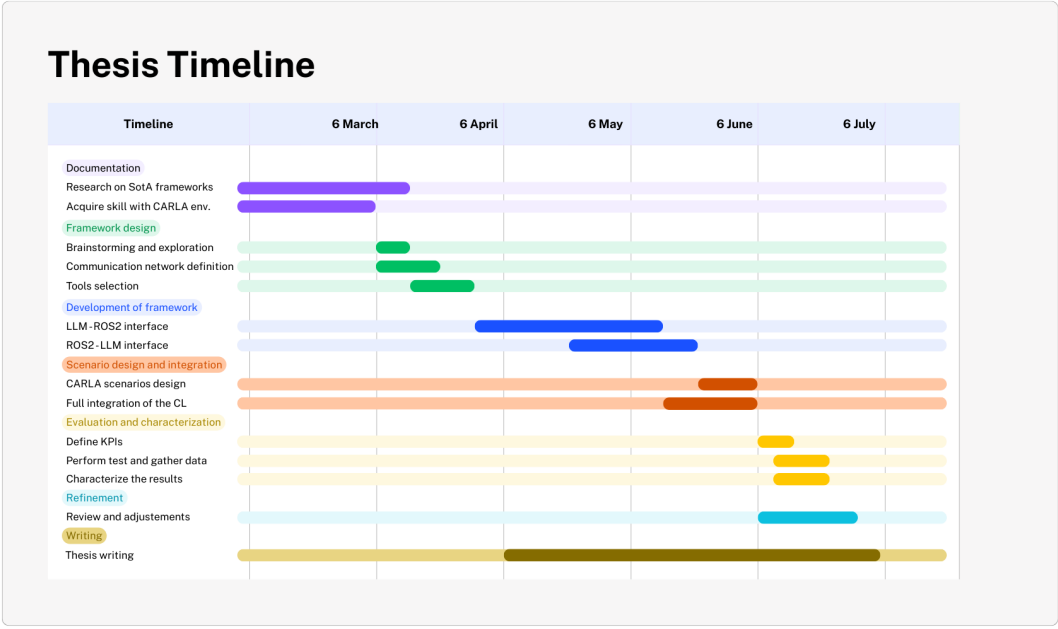


Figure 24: The Gantt chart shows the initial planning and scheduling for the development of the thesis.

Figure 24 shows the initial planning and scheduling stated for the development of the thesis. Although some of the sections, subsections and schedules were adapted or changed along the project, following arising necessities, the chart approximately reflects what has come to be as this final version of the thesis.

# 7 Economical and environmental impact

## 7.1 Economical study

The economic cost of this project was modest, and it is dominated by the hardware. Because the local model must run on-device and shares the GPU with the simulator, a machine with a dedicated GPU is a requirement in order to be able to run the framework. The laptop used (Section 3.2) was acquired specifically for this work at approximately 1,200 €, and it represents the overwhelming majority of the budget. The rest of the software stack incurred no licensing cost. CARLA, ROS2, the carla-ros-bridge, pixi, and Ollama are all open-source and free to use, and the local model runs at no monetary cost beyond the electricity it consumes. The only recurring expense was the cloud API. Over the entire project, the API usage cost 2.18$ on OpenRouter (Claude Haiku 4.5, billed at 1$ and 5$ per million input and output tokens respectively) and 0.33$ on the OpenAI API during the discarded GPT-4o trials, for a combined total of 2.51$ . This reflects the light token footprint of the task. To this must be added the electricity consumed locally. A gaming laptop of this class draws on the order of 150 to 200 W under the sustained full load of running CARLA and the local model together. The exact number of hours run over the project is hard to pin down, but even at a generous estimate of a few hundred hours of full-load operation, the total energy is on the order of tens of kWh, which at the average Spanish electricity price (around 0.15 €/kWh in 2026) amounts to a few euros.

Table 16: Approximate project budget.

| **Item** | **Cost** |
|---|---|
| Laptop (HP OMEN 16, RTX 5050) | 1 200 € |
| Cloud API usage (OpenRouter + OpenAI) | ≈ 2.3 € |
| Local electricity (estimated) | ≈ 3–5 € |
| Open-source software (CARLA, ROS2, Ollama, pixi) | 0 € |
| **Total** | ≈ 1 207 € |

## 7.2 Environmental study

The direct environmental footprint of this project is small, but it is worth situating it in the wider context of the technology it relies on, because the local versus cloud choice studied here has an environmental dimension as well as a technical one. Locally, the impact is limited to the electricity consumed by the laptop, on the order of tens of kWh over the whole project. Using the Spanish grid's average carbon intensity, roughly 150 to 200 $gCO_2$/kWh in recent years, a relatively low figure thanks to a high share of renewable energy and nuclear. This corresponds to only a few kilograms of $CO_2$, comparable to a short car journey. The cloud usage adds an indirect footprint that is harder to quantify, since it involves the provider's data centers rather than local infrastructure. The inference performed by the cloud model does not consume the user's electricity, but it does consume the data center's, and the environmental cost of large-scale AI inference is substantial and growing. The International Energy Agency projects that global data-center electricity demand will more than double by 2030, to around 945 TWh, slightly above the total consumption of Japan [40]. A significant and rising share of this is driven by AI, being inference, not training, the dominant contributor. Google has reported that about 60% of its AI-related energy use comes from serving queries rather than training models [41]. Even a single short query to a large model carries a measurable cost. One benchmark estimates a short GPT-4o query at around 0.4 Wh, roughly 40% more than a web search [42], a figure

that becomes considerable when multiplied across billions of daily requests. In this light, the framework's design has a favorable environmental impact. By confining the model to sparse, high-level decisions rather than continuous inference, it keeps the number and size of queries low, whether they are served locally or in the cloud. The most token consuming element, the image, is deliberately downscaled to $320 \times 180$.

# 8 Conclusions

This thesis proposed to design, develop, and validate a VLM-based high-level planner for an autonomous vehicle within a hierarchical framework with a dedicated safety layer. That objective was met, the framework was built as three decoupled ROS2 modules, spanning perception, decision-making, and control, integrated and validated in the CARLA simulator under reproducible conditions, and evaluated across five scenarios of increasing complexity with two models of very different capability. The remaining conclusions concern what that evaluation revealed about the two hypotheses and the one research question that were stated at the beginning of the work.

The first hypothesis (H1) is supported. In every scenario the vehicle completed the navigation without executing dangerous maneuvers, and the low-level control layer absorbed the planner's instabilities and inference delays rather than letting them reach the vehicle. Malformed and hallucinated outputs were rejected, premature stops were relaxed, unstable command sequences were damped, and the model's silences and latency were covered by the watchdog and the speed ramp. The strongest evidence is that the driving outcomes of a weak baseline and a near-frontier model converged to a similar performance in the global evaluation of the framework despite a large gap in the quality of their decisions, which is precisely what the safety layer was meant to achieve. There is, however, a clear boundary that the pedestrian scenario made explicit. The safety layer absorbs malformed or unstable outputs, but it cannot correct a decision that is well-formed and simply wrong. Therefore, H1 holds for the class of failures the layer was designed to absorb, not as a guarantee of safe behavior under any planner error.

The second hypothesis (H2) is also accomplished, within the limits of the evidence. The framework proved model-agnostic in practice, switching from the baseline model to the reference model required changing only a single launch argument, with no modification to the architecture. And the quality of the decisions scaled clearly with the capability of the model, consistently across every metric and in the reasoning itself, where the reference model reasoned about the numeric scene while the baseline fell back on simple matching of patterns and hallucination. With only two models this is a consistent observation at two points of the capability range not a demonstrated statement. However, the data obtained clearly points in that direction.

The research question (RQ) remains, as anticipated, only partially answerable. The results confirm that the visual input is attended to but it cannot establish what it contributes to the decisions, because the relevant decision information is already provided as ground truth in the text and no comparison with only text framework was run for contrast. Whether the image improves the planning is left open, and answering it is one of the directions for the work that follows.

Taken together, the results support the main assumption of the thesis, that a hierarchical design, isolating a probabilistic vision-language model to issue high-level decisions behind a deterministic safety layer, can turn even an unreliable planner into viable driving behavior. The value of this architecture is exactly how much it bridges the gap between a weak and a strong model at the executed actions level. The limitations encountered along the way, the scope of the safety layer, the absence of a navigation goal, the perception of emergency vehicles, and the unresolved role of the visual modality, define the directions in which the framework can be extended, which are developed in the following section.

# 9 Future work

The framework opens several directions for future development, ranging from substantial research lines to more incremental refinements.

The most promising is the introduction of goal-oriented navigation. The current design, as explained, is purely reactive. The obstacle scenario showed that the reference model, having nowhere to go, simply waited behind the blockage and almost never issued a turn or lane change. Providing a route planning layer, supported by the GNSS and map sensors deliberately left unused (Section 3.1.1.1), would give the planner the intent its reactive design lacks, likely improving the decisions of the more capable models and, by grounding the smaller ones in a concrete objective, possibly reducing their hallucinations. Closely related is the addition of a navigation memory, a short history of previously committed actions with timestamps, which would give the model an awareness of how the scene has progressed, giving continuity to the context and logic to the chain of reasoning.

A second line concerns the models themselves. The two models used were selected for their capability and accessibility and were run zero-shot, without any adaptation to the driving task, so their performance is bounded by their general pre-training. Fine-tuning a model on driving data, or adopting a model specialized in embodied reasoning, such as Gemini Robotics-ER 1.6, which is designed to act as a high-level planner with enhanced spatial reasoning, task planning, and success detection [43], could improve the grounding and quality of the decisions well beyond what a general model offers zero-shot.

A third line targets the open question of the visual modality. A dedicated ablation study, running each scenario with and without the image, and at different resolutions, would finally isolate what the image contributes and whether its quality matters, answering the research question this study left open. This could be further improved by replacing the idealized ground-truth pseudo-sensors by realistic perception pipelines so that when the textual context is no longer a perfect ground truth, the image could have genuine information to add, and its contribution could be measured meaningfully.

Finally, several smaller refinements would strengthen the framework and its evaluation. Increasing the empirical base with more runs per scenario and additional maps besides Town10HD, to move from indicative figures toward statistically grounded conclusions. Beyond this, the commitment mechanism could be refined in how it protects an ongoing maneuver from intermediate commands, and the sampling temperature, exposed by the cloud APIs, could be tuned to trade off determinism against flexibility. The migration of the low-level control toward a learned approach, discussed among the framework's limitations (Section 5.1), belongs to this horizon as well, as the long-term improvement path from simulation toward the physical world.

# 10 Appendix

## 10.1 Source code

The complete source code of the framework is publicly available at:

https://github.com/marjimvic02/vlm-high-level-planner-framework

The repository contains the three ROS2 packages that implement the perception, decision-making, and low-level control modules (`scene_context_node.py`, `prompt_manager.py`, and `action_dispatcher.py`, respectively), together with the launch configuration, the scenario setup script (`demo_escenario.py`), the behavior prompt, and the parsing tools used to extract the metrics reported in Chapter 4. The code is released for reproducibility, so that the experiments described in this work can be repeated under the conditions detailed in Section 3.2.

# Bibliografia


[1] Epic Games. *Behavior Tree in Unreal Engine - Overview*. Image referenced: behavior-tree-overview-intro.png. 2026. URL: https://dev.epicgames.com/documentation/en-us/unreal-engine/behavior-tree-in-unreal-engine---overview (visited on 06/29/2026).

[2] J. Hu et al. "A survey of decision-making and planning methods for self-driving vehicles". In: *Frontiers in Neurorobotics* (2025).

[3] A. Rizehvandi. "Evolution of Decision-making Systems in Autonomous Vehicle Technology: A Comprehensive Review". In: *IEEE Transactions on Intelligent Vehicles* (2024).

[4] M. Colledanchise and P. Ögren. *Behavior Trees in Robotics and AI: An Introduction*. Chapman & Hall/CRC Artificial Intelligence and Robotics Series. Preprint also available as arXiv:1709.00084. CRC Press, Taylor & Francis Group, 2018.

[5] M. Buehler, K. Iagnemma, and S. Singh, eds. *The DARPA Urban Challenge: Autonomous Vehicles in City Traffic*. Vol. 56. Springer Tracts in Advanced Robotics. Springer, 2009.

[6] C. Urmson et al. "Autonomous Driving in Urban Environments: Boss and the Urban Challenge". In: *Journal of Field Robotics* 25.8 (2008), pp. 425–466.

[7] M. Bojarski et al. "End to End Learning for Self-Driving Cars". In: *arXiv* (2016). arXiv: 1604.07316 [cs.CV].

[8] B. R. Kiran et al. "Deep Reinforcement Learning for Autonomous Driving: A Survey". In: *IEEE Transactions on Intelligent Transportation Systems* 23.6 (2021), pp. 4909–4926. DOI: 10.1109/TITS.2021.3054625.

[9] S. Ross, G. J. Gordon, and D. Bagnell. "A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning". In: *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics* (*AISTATS*). Vol. 15. Proceedings of Machine Learning Research. PMLR, 2011, pp. 627–635.

[10] Y. J. Ma et al. "Eureka: Human-Level Reward Design via Coding Large Language Models". In: *arXiv* (2024). arXiv: 2310.12931 [cs.RO].

[11] R. Hazra et al. "REvolve: Reward Evolution with Large Language Models using Human Feedback". In: *Proceedings of the International Conference on Learning Representations* (*ICLR*). 2025. arXiv: 2406.01309 [cs.LG].

[12] D. Ha and J. Schmidhuber. "World Models". In: *arXiv* (2018). DOI: 10.5281/zenodo.1207631. arXiv: 1803.10122 [cs.LG].

[13] A. Hu et al. "GAIA-1: A Generative World Model for Autonomous Driving". In: *arXiv* (2023). arXiv: 2309.17080 [cs.CV].

[14] T. B. Brown et al. "Language Models are Few-Shot Learners". In: *Advances in Neural Information Processing Systems* (*NeurIPS*). Vol. 33. 2020, pp. 1877–1901.

[15] J. Wei et al. "Emergent Abilities of Large Language Models". In: *Transactions on Machine Learning Research* (*TMLR*) (2022). arXiv: 2206.07682 [cs.CL].

[16] A. Vaswani et al. "Attention Is All You Need". In: *Advances in Neural Information Processing Systems* (*NeurIPS*). Vol. 30. 2017, pp. 5998–6008.

[17] D. Driess et al. "PaLM-E: An Embodied Multimodal Language Model". In: *Proceedings of the 40th International Conference on Machine Learning* (*ICML*). 2023. arXiv: 2303.03378 [cs.LG].

[18] M. Ahn et al. "Do As I Can, Not As I Say: Grounding Language in Robotic Affordances". In: *Proceedings of the 6th Conference on Robot Learning* (*CoRL*). 2022. arXiv: 2204.01691 [cs.RO].

[19] J. Liang et al. "Code as Policies: Language Model Programs for Embodied Control". In: *2023 IEEE International Conference on Robotics and Automation (ICRA)*. 2023, pp. 9493–9500. arXiv: 2209.07753 [cs.RO].

[20] A. Brohan et al. "RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control". In: *arXiv* (2023). arXiv: 2307.15818 [cs.RO].

[21] G. Zhou, Y. Hong, and Q. Wu. "NavGPT: Explicit Reasoning in Vision-and-Language Navigation with Large Language Models". In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 38. 2024, pp. 7641–7649. arXiv: 2305.16986 [cs.CV].

[22] R. Schumann et al. "VELMA: Verbalization Embodiment of LLM Agents for Vision and Language Navigation in Street View". In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 38. 2024, pp. 18924–18933. arXiv: 2307.06082 [cs.CL].

[23] V. Bhat et al. "Grounding LLMs For Robot Task Planning Using Closed-loop State Feedback". In: *arXiv* (2024). arXiv: 2402.08546 [cs.RO].

[24] P. Sharma et al. "Correcting Robot Plans with Natural Language Feedback". In: *Proceedings of Robotics: Science and Systems (RSS)*. 2022. DOI: 10.15607/RSS.2022.XVIII.065.

[25] L. Li et al. "LLM-augmented hierarchical reinforcement learning for human-like decision-making of autonomous driving". In: *Expert Systems with Applications* 294 (2025), p. 128736. DOI: 10.1016/j.eswa.2025.128736.

[26] J. Li et al. "A Safe Hierarchical Planning Framework for Complex Driving Scenarios based on Reinforcement Learning". In: *2021 IEEE International Conference on Robotics and Automation (ICRA)*. 2021, pp. 2660–2666. arXiv: 2101.06778 [cs.RO].

[27] S. Kato et al. "Autoware on Board: Enabling Autonomous Vehicles with Embedded Systems". In: *2018 ACM/IEEE 9th International Conference on Cyber-Physical Systems (ICCPS)*. IEEE, 2018, pp. 287–296. DOI: 10.1109/ICCPS.2018.00035.

[28] F. Shi et al. "Large Language Models Can Be Easily Distracted by Irrelevant Context". In: *Proceedings of the 40th International Conference on Machine Learning (ICML)*. Vol. 202. Proceedings of Machine Learning Research. PMLR, 2023, pp. 31210–31227. arXiv: 2302.00093 [cs.CL].

[29] N. F. Liu et al. "Lost in the Middle: How Language Models Use Long Contexts". In: *Transactions of the Association for Computational Linguistics (TACL)* 12 (2024), pp. 157–173. DOI: 10.1162/tacl_a_00638. arXiv: 2307.03172 [cs.CL].

[30] Open Robotics. *Quality of Service settings — ROS 2 Documentation: Humble*. 2026. URL: https://docs.ros.org/en/humble/Concepts/Intermediate/About-Quality-of-Service-Settings.html (visited on 04/18/2026).

[31] Anthropic. *Prompting best practices - Claude Platform Docs*. 2026. URL: https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/claude-prompting-best-practices (visited on 04/05/2026).

[32] J. T. Licardo, N. Tanković, and D. Etinger. "BPMN Assistant: An LLM-Based Approach to Business Process Modeling". In: *arXiv* (2025). arXiv: 2509.24592 [cs.SE].

[33] J. Wei et al. "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models". In: *arXiv* (2022). arXiv: 2201.11903 [cs.CL].

[34] OpenAI. *GPT-4V(ision) System Card*. 2023. URL: https://cdn.openai.com/papers/GPTV_System_Card.pdf (visited on 05/21/2026).

[35] CARLA Team. *Traffic Manager - CARLA Simulator*. 2026. URL: https://carla.readthedocs.io/en/latest/adv_traffic_manager/ (visited on 05/29/2026).

[36] A. Dosovitskiy et al. "CARLA: An Open Urban Driving Simulator". In: *arXiv* (2017). arXiv: 1711.03938 [cs.LG].

[37] CARLA Team. *CARLA Simulator Documentation - Release 0.9.16*. 2026. URL: https://carla.readthedocs.io/en/0.9.16/ (visited on 04/01/2026).

[38] S. Macenski et al. "Robot Operating System 2: Design, Architecture, and Uses In The Wild". In: *arXiv* (2022). arXiv: 2211.07752 [cs.RO].

[39] prefix.dev. *Pixi Documentation*. 2026. URL: https://pixi.prefix.dev/latest/ (visited on 03/30/2026).

[40] International Energy Agency. *Energy and AI*. IEA, 2025. URL: https://www.iea.org/reports/energy-and-ai (visited on 07/02/2026).

[41] T. Vartziotis et al. "Carbon Footprint Evaluation of Code Generation through LLM as a Service". In: *arXiv* (2025). arXiv: 2504.01036 [cs.CY].

[42] N. Jegham et al. "How Hungry is AI? Benchmarking Energy, Water, and Carbon Footprint of LLM Inference". In: *arXiv* (2025). arXiv: 2505.09598 [cs.CY].

[43] Google DeepMind. *Gemini Robotics-ER 1.6 Model Card*. 2026. URL: https://deepmind.google/models/model-cards/gemini-robotics-er-1-6/ (visited on 06/28/2026).